

INSTITUT AFRICAIN D'INFORMATIQUE

NOM ENTREPRISE (EN FRANÇAIS)

AFRICAN INSTITUTE OF COMPUTER SCIENCES

NOM ENTREPRISE (EN ANGLAIS)



Représentation du Cameroun

BP: 13719 Yaoundé (Cameroun)

Tél: (237) 22 21 11 43 / (237) 99 99 82 49

Site web: www.iaicameroun.com

Adresse de l'entreprise

Ville

Tél:

Site web:

RAPPORT DE FIN DE STAGE

VOTRE THEME ICI

Stage effectué du 1er juillet 2025 au 30 septembre 2025

En vue de l'obtention du **Diplôme de Technicien Supérieur (DTS) en Informatique option Génie Logiciel**

Rédigé par :

- **NOM & PRENOM**, étudiant au niveau II à l'Institut Africain d'Informatique

Sous la supervision de :



Conception et développement d'une application de gestion des missions d'audit interne : SIGMA-ADC



ENCADRANT ACADEMIQUE

Nom de l'encadrant académique

Niveau académique

Niveau professionnel

&

ENCADRANT PROFESSIONNEL

Nom de l'encadrant professionnel

Niveau académique

Niveau professionnel

Année académique 2024-2025



Conception et développement d'une application de gestion des missions d'audit interne : SIGMA-ADC



DÉDICACE

À
La famille
NDEME
ASSEYI

REMERCIEMENTS

Nous tenons à remercier toutes les personnes qui ont contribué de près ou de loin au bon déroulement de notre stage, à la réalisation de ce rapport jusqu'à la présentation de notre travail. Nos sincères remerciements s'adressent :

- ✓ Au représentant résident de l'Institut africain d'Informatique et représentation du Cameroun M. Armand Claude ABANDA ;
- ✓ Au directeur Général des ADC S.A du Cameroun M. Thomas OWONA ASSOUMOU ;
- ✓ A mon encadrant professionnel Mme ONDOBO EBELLA
- ✓ A mon encadrant académique Mme ADA Nadege
- ✓ Un grand merci à nos parents pour leurs sacrifices et dévouements ;
- ✓ Nos camarades de promotions et toutes personnes de près ou de loin ayant participés à notre évolution.
- ✓ Au comité des étudiants « IAI United » pour l'aide aux étudiants durant toute l'année ainsi qu'à la période allant du stage aux soutenances.

SOMMAIRE

DÉDICACE	II
REMERCIEMENTS.....	III
SOMMAIRE	IV
LISTE DES TABLEAUX.....	V
LISTE DES FIGURES	VI
SIGLES ET ABBREVIATIONS.....	VIII
GLOSSAIRE.....	IX
RESUME	XI
ABSTRACT.....	XII
INTRODUCTION GÉNÉRALE	1
1ère PARTIE : PHASE D'INSERTION	2
2ème PARTIE : PHASE TECHNIQUE	14
DOSSIER 1 : L'EXISTANT	16
DOSSIER 2 : CAHIER DES CHARGES.....	24
DOSSIER 3 : DOSSIER D'ANALYSE.....	46
DOSSIER 4 : DOSSIER DE CONCEPTION	91
DOSSIER 5 : DOSSIER DE RÉALISATION.....	108
DOSSIER 6 : TEST DE FONCTIONNALITÉS	121
DOSSIER 7 : GUIDE D'INSTALLATION ET GUIDE D'UTILISATEUR.....	131
CONCLUSION GÉNÉRALE.....	138
BIBLIOGRAPHIE	2
WEBOGRAPHIE	4
ANNEXES.....	5
TABLE DES MATIÈRES	6

LISTE DES TABLEAUX

Tableau 1 ressources matérielles de ADC S.A.....	11
Tableau 2 ressources logicielles de ADC S.A.....	12
Tableau 3 les limitations de l'existant et leurs solutions	21
Tableau 4 listes des intervenants.....	36
Tableau 5 ressources matérielles.....	38
Tableau 6 ressources logicielles.....	39
Tableau 7 ressources humaines.....	41
Tableau 8 comparaison entre UML et MERISE	49
Tableau 9 composant d'un diagramme de cas d'utilisation	54
Tableau 10 types de relations.....	55
Tableau 11 formalisme d'une description textuel pour le diagramme de cas d'utilisation	59
Tableau 12 description textuelle du cas : S'authentifier.....	61
Tableau 13description textuelle du cas : Gérer l'exécution des missions d'audit.....	63
Tableau 14 description textuelle du cas : Gérer la documentation de la mission	65
Tableau 15 description textuelle du cas : Gérer les missions d'audit.....	68
Tableau 16 formalisme du diagramme de communication	71
Tableau 17 formalisme du diagramme de séquence	76
Tableau 18 tableau de formalisme d'activé	83
Tableau 19 formalisme du diagramme de classe	94
Tableau 20 description textuelle.	97
Tableau 21 formalisme d'un diagramme de transition	102
Tableau 22 formalisme du diagramme de paquetage.....	105
Tableau 23 environnement logiciel utilisée	110
Tableau 24 technologies desktop utilisée.....	111
Tableau 25 technologies mobile utilisée	111
Tableau 26 technologies backend utilisée.....	112
Tableau 27 formalisme du diagramme de déploiement	116
Tableau 28 formalisme du diagramme de composant.....	118

LISTE DES FIGURES

Figure 1 localisation de ADC S.A.....	7
Figure 2 organigramme de ADC S.A.....	9
Figure 3 diagramme de gantt	37
Figure 4 2TUP.....	53
Figure 5 exemple de diagramme de cas d'utilisation	55
Figure 6 diagramme cas d'utilisation global	56
Figure 7 diagramme spécifique du cas d'utilisation : Gérer les missions d'audit.....	57
Figure 8 diagramme spécifique tu cas d'utilisation : Gérer la documentation de la mission	58
Figure 9 cas d'utilisation : Gérer l'exécution des missions d'audit.....	59
Figure 10 diagramme de communication du cas s'authentifier.....	72
Figure 11 diagramme de communication du cas : Gérer les missions d'audit	73
Figure 12 diagramme de communication du cas : gérer l'exécution des missions d'audit	74
Figure 13 diagramme de communication du cas : gérer la documentation de la mission.	75
Figure 14 exemple de diagramme de séquence.	77
Figure 15 diagramme de séquence : s'authentifier	78
Figure 16 diagramme de séquence : Gérer les missions d'audit	79
Figure 17 diagramme de séquence : gérer l'exécution des missions.....	80
Figure 18 diagramme de séquence : Gérer la documentation de la mission.....	81
Figure 19 formalisme d'un diagramme d'activité.....	84
Figure 20 diagramme d'activité du cas s'authentifier.....	85
Figure 21 diagramme d'activité du cas : Gérer les missions d'audit	87
Figure 22 diagramme d'activité du cas : Gérer l'exécution des missions d'audit.....	88
Figure 23 diagramme d'activité du cas : Gérer la documentation de la mission.....	89
Figure 24 exemple du diagramme de classe.	95
Figure 25 diagramme de classe de notre système	96
Figure 26 exemple de diagramme d'état transition	103
Figure 27 diagramme d'état transition du cas de la connexion	103
Figure 28 diagramme d'état transition du cas d'un utilisateur	104
Figure 29 diagramme de paquetage.	106
Figure 30 architecture physique n-tiers.....	114
Figure 31 exemple de diagramme de déploiement	117
Figure 32 diagramme de déploiement.....	117
Figure 33 premier test sur les rôles	127
Figure 34 test d'appel d'api.....	128
Figure 35 deuxième test sur le rôle s'identifier	128
Figure 36 tests sur les modèles	129
Figure 37 page d'accueil du site d'Oracle pour télécharger Java.	133
Figure 38 page de connexion	135



Conception et développement d'une application de gestion des missions d'audit interne : SIGMA-ADC



Figure 39 page d'accueil de l'administrateur.....	136
Figure 40 page d'accueil du chef de division.....	136

SIGLES ET ABBREVIATIONS

- **2TUP**: Two Tracks Unified Process
- **ADC**: Aéroports Du Cameroun
- **API** : Application Programming Interface (interface de programmation applicative)
- **COSO**: Committee of Sponsoring Organizations of the Treadway Commission
- **CRUD**: Create, Read, Update, Delete (créer, lire, modifier, supprimer)
- **CSS**: Cascading Style Sheets
- **FXML** : JavaFX Extensible Markup Language
- **HTTP** : HyperText Transfer Protocol
- **IFACI** : Institut Français de l'Audit et du Contrôle Interne
- **IHM**: Interface Homme-Machine
- **IIA**: Institute of Internal Auditors
- **ISO**: International Organization for Standardization
- **JPA**: Java Persistence API
- **JSON**: JavaScript Object Notation
- **KPI**: Key Performance Indicator (indicateur clé de performance)
- **MCD** : Modèle Conceptuel de Données
- **MERISE** : Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise
- **MLD** : Modèle Logique de Données
- **MPD** : Modèle Physique de Données
- **MVC** : Modèle-Vue-Contrôleur (*Model-View-Controller*)
- **ORM**: Object-Relational Mapping (mapping objet-relationnel)
- **PAA** : Plan Annuel d'Audit
- **PDF** : Portable Document Format
- **REST**: Representational State Transfer
- **S.A.**: Société Anonyme
- **SGBD** : Système de Gestion de Bases de Données
- **UML**: Unified Modeling Language
- **XML**: Extensible Markup Language

GLOSSAIRE

- **Audit interne** : Activité indépendante et objective qui donne à une organisation une assurance sur le degré de maîtrise de ses opérations, lui apporte des conseils pour les améliorer et contribue à créer de la valeur ajoutée. Dans ce projet, l'application est destinée à outiller la Division de l'Audit Interne d'Aéroports Du Cameroun S.A.
- **Mission d'audit** : Intervention planifiée portant sur un processus, un service ou une activité, comprenant la préparation, la réalisation des travaux sur le terrain, la restitution des constats et la production d'un rapport. L'application gère l'ensemble du cycle de vie d'une mission.
- **Plan Annuel d'Audit (PAA)** : Document qui programme les missions d'audit à réaliser sur l'année, en fonction des risques identifiés et des ressources disponibles. L'application permet de l'élaborer et de le suivre.
- **Cartographie des risques** : Représentation structurée des risques d'une organisation, classés selon leur probabilité et leur impact. Elle sert de point de départ à la planification des missions dans l'application.
- **Risque significatif** : Risque dont la probabilité ou l'impact est suffisamment élevé pour menacer l'atteinte des objectifs de l'organisation et justifier une mission d'audit.
- **Recommandation** : Action corrective ou d'amélioration proposée par l'auditeur à la suite d'un constat. L'application assure le suivi de leur mise en œuvre.
- **Traçabilité** : Capacité à retrouver l'historique et l'origine des informations et des travaux réalisés. Elle constitue l'un des principaux enjeux du projet, qui centralise des données auparavant dispersées.
- **Tableau de bord** : Outil de visualisation synthétique d'indicateurs clés destiné au pilotage. Dans l'application, il permet de suivre la performance de la fonction d'audit interne.
- **Application desktop** : Logiciel installé et exécuté sur le poste de travail de l'utilisateur, par opposition à une application web. L'application développée est un client desktop connecté à un serveur central.

- **Architecture client-serveur** : Modèle où des postes clients envoient des requêtes à un serveur qui centralise les données et les traitements. Dans le projet, plusieurs clients desktop accèdent à une base de données centralisée.
- **PostgreSQL** : Système de gestion de bases de données relationnelles open source, réputé pour sa fiabilité et sa robustesse. Il stocke de manière centralisée les données de l'application (missions, risques, recommandations).
- **MERISE** : Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise. Dans le projet, elle a servi à la conception de la base de données de l'application.
- **Java** : Langage de programmation orienté objet, portable et largement utilisé en entreprise. Il est utilisé pour développer à la fois le client desktop et le serveur de l'application.
- **JavaFX** :
Framework Java pour la création d'interfaces graphiques d'applications desktop. Dans le projet, JavaFX sert à construire l'interface utilisateur du client (vues FXML et contrôleurs).
- **Spring Boot** :
Framework Java qui simplifie la création d'applications serveur en fournissant une configuration automatique et un serveur embarqué. Dans le projet, il constitue le serveur d'application : logique métier, sécurité et accès à la base de données.
- **API REST (Representational State Transfer)** :
Style d'architecture permettant à un client et à un serveur d'échanger des données via le protocole HTTP. Dans le projet, le client JavaFX communique avec le serveur Spring Boot par ce biais.
- **MVC (Modèle-Vue-Contrôleur)** : Patron d'architecture qui sépare les données (Modèle), l'interface (Vue) et la logique de traitement des actions (Contrôleur). Il rend l'application plus modulaire et plus facile à maintenir.

RESUME

Ce rapport présente la conception et le développement d'une application desktop de gestion des missions d'audit interne, réalisée dans le cadre d'un stage académique au sein de la Division de l'Audit Interne d'Aéroports Du Cameroun S.A. Il répond aux difficultés rencontrées par les auditeurs internes : la dispersion de l'information sur des outils bureautiques hétérogènes, le manque de traçabilité des travaux et un suivi des recommandations peu structuré. Il propose une solution articulée autour de trois piliers : une couverture fonctionnelle complète du cycle de vie d'une mission d'audit, allant de la cartographie des risques significatifs à la production des rapports, en passant par l'élaboration du Plan Annuel d'Audit et la gestion des missions ; une architecture client-serveur fiable, reposant sur une base de données PostgreSQL centralisée et un client desktop développé en JavaFX selon le patron d'architecture MVVM ; et un pilotage de la performance de la fonction d'audit interne à l'aide de tableaux de bord dédiés, ainsi que le suivi de la mise en œuvre des recommandations. Le document retrace la démarche suivie, de la phase d'insertion en entreprise et de l'analyse des besoins jusqu'à la conception et la réalisation de l'application. Face à la nécessité de centraliser l'information et de sécuriser la traçabilité des travaux, cette solution apporte une réponse concrète aux exigences de rigueur, de transparence et de suivi de la fonction d'audit interne, et se positionne comme un outil d'aide à la décision pour la Division de l'Audit Interne

- Audit interne
- Gestion des missions
- Cartographie des risques
- Suivi des recommandations
- Tableaux de bord
- Springboot – JavaFX – PostgreSQL

ABSTRACT

This report presents the design and development of a desktop application for managing internal audit missions, carried out during an academic internship within the Internal Audit Division of Aéroports Du Cameroun S.A. It addresses the difficulties faced by internal auditors: information scattered across heterogeneous office tools, a lack of traceability of audit work, and poorly structured follow-up of recommendations. It proposes a solution built around three pillars: full functional coverage of the audit mission lifecycle, from the mapping of significant risks to report production, including the drafting of the Annual Audit Plan and mission management; a reliable client-server architecture based on a centralized PostgreSQL database and a desktop client developed in JavaFX following the MVVM architectural pattern; and performance monitoring of the internal audit function through dedicated dashboards, together with tracking of the implementation of recommendations. The document describes the approach followed, from the onboarding phase within the company and the analysis of needs through to the design and implementation of the application. Faced with the need to centralize information and secure the traceability of audit work, this solution provides a concrete response to the requirements of rigor, transparency, and follow-up of the internal audit function, and positions itself as a decision-support tool for the Internal Audit Division.

- Internal audit
- Mission management
- Risk mapping
- Recommendation follow-up
- Dashboards
- Springboot - JavaFX - PostgreSQL

INTRODUCTION GÉNÉRALE

L'informatique s'est imposée, au fil des dernières décennies, comme un levier essentiel de transformation des organisations. En automatisant les tâches répétitives, en centralisant les données et en facilitant leur exploitation, les systèmes d'information améliorent la performance, la fiabilité et la rapidité des processus de travail, quel que soit le secteur d'activité. Cette évolution touche aujourd'hui aussi bien les fonctions opérationnelles que les fonctions de contrôle et de pilotage. Parmi ces dernières, l'audit interne occupe une place centrale dans le dispositif de gouvernance des entreprises modernes. Il apporte une assurance raisonnable sur la maîtrise des risques et l'efficacité des processus, et contribue à l'amélioration continue du contrôle interne. Pour remplir pleinement ce rôle, la fonction d'audit doit s'appuyer sur des outils qui garantissent la fiabilité de l'information, la traçabilité des travaux et le suivi rigoureux des recommandations formulées. C'est le cas chez Aéroports Du Cameroun S.A., société assurant la gestion de sept aéroports sur l'ensemble du territoire national. Sa Division de l'Audit Interne est chargée de conduire les missions permettant d'évaluer et d'améliorer les dispositifs de contrôle interne des différentes directions. Or, la gestion actuelle de ces missions repose principalement sur des outils bureautiques génériques (tableurs, documents texte). Cette situation limite la centralisation des informations, complique le suivi des recommandations et ne garantit pas une traçabilité suffisante des travaux réalisés. C'est dans ce contexte que s'inscrit notre projet de fin de stage : la conception et le développement d'une application desktop dédiée à la gestion des missions d'audit interne, afin de centraliser l'information et d'outiller la Division sur l'ensemble du cycle de vie d'une mission. Le présent document retrace la démarche suivie tout au long du stage. Il s'ouvre sur la phase d'insertion en entreprise, qui nous a permis de nous familiariser avec l'environnement professionnel et la structure d'accueil, puis présente l'analyse des besoins, la conception de la solution et enfin sa réalisation technique.

1ÈRE PARTIE : PHASE D'INSERTION

Résumé :

Le dossier d'insertion est le tout premier document rédigé durant la période de stage académique. C'est la partie du stage qui présente la structure d'accueil, son fonctionnement, ainsi que l'accueil de l'étudiant en entreprise. Cette phase d'insertion dure généralement deux semaines et permet au stagiaire de se familiariser avec son nouvel environnement.

Aperçu :

Introduction

- I. Accueil et intégration en entreprise.
- II. Présentation de l'entreprise.

Conclusion

INTRODUCTION

La phase d'insertion constitue la première étape de notre stage académique au sein d'Aéroports Du Cameroun S.A. Elle a pour objectif de nous familiariser avec l'environnement professionnel, de comprendre le fonctionnement de la structure d'accueil et de créer les liens nécessaires avec les équipes de la Division de l'Audit Interne, avant d'aborder les aspects techniques du projet. Cette partie présente successivement les conditions de notre accueil et de notre intégration au sein de l'entreprise, ainsi qu'une présentation générale d'Aéroports Du Cameroun S.A.

I. ACCUEIL ET INTÉGRATION EN ENTREPRISE

1. Accueil en entreprise

Le mardi 09 juin 2026 à 7h30, nous avons été accueillis par le Chef du Service de la Formation, **M. ALIN NGOA**, et ses collaborateurs, qui ont pris le soin de nous informer non seulement sur le fonctionnement de la structure, mais aussi sur l'objectif à atteindre durant les trois (03) prochains mois de stage qui nous ont été accordés. Après cet entretien, nous avons été conduits vers **Mme EFFALA MEVONO, épouse HERTZ**, Chef de Division de l'Audit Interne, qui nous a affectés dans un service au sein de la Division de l'Audit Interne (**DI**), à savoir celui de **l'Audit des Systèmes d'Information**, tout en nous attribuant un encadrant professionnel, **Mme ONDOBO EBELLA, épouse NGUELE**, Chef de Cellule l'Audit des Marchées et des Systèmes d'information, par la même occasion. Cette première journée de stage a aussi marqué le début des activités des stagiaires, avec quelques rappels de certaines notions acquises tout au long de l'année académique au sein de l'IAI-Cameroun Centre d'Excellence Technologique Paul BIYA.

2. Intégration en entreprise

Après l'accueil chaleureux que nous avons reçu le premier jour de stage est venue une phase critique : celle de l'insertion. Notre première semaine de stage a été consacrée au développement des relations professionnelles et à l'acclimatation à notre nouvel environnement. Cette semaine a été fructueuse et bénéfique, car nous avons eu le privilège de mettre en pratique nos connaissances et de les améliorer. Chaque matin, nous étions informés des tâches à réaliser ainsi que des horaires de travail. Nos encadrants nous ont également fait passer quelques tests de performance afin d'évaluer notre niveau et de combler les éventuelles lacunes en connaissances technologiques. L'intégration s'est faite facilement, et la familiarité est venue rapidement : étant tous jeunes, nos collègues ont instauré une atmosphère détendue au sein de l'entreprise. Cependant, en ce qui concernait le travail à effectuer, tout restait professionnel, et nous étions toujours guidés par **Madame EFFALA MEVONO, épse HERTZ**, Chef de Division de l'Audit Interne, et encadrés par **Madame ONDOBO EBELLA, épse NGUELE**, Chef de la Cellule de l'Audit des Marchés et des Systèmes d'Information.

II. PRÉSENTATION DE L'ENTREPRISE

1. Historique

Le début des années 90 marque, pour l'État camerounais, le désir de se désengager de la gestion des aéroports et la volonté de mettre en place une stratégie de gestion autonome des principaux aéroports du Cameroun. C'est ainsi que le 06 octobre 1992, l'autorité camerounaise et la société ADP conviennent de créer une société qui va gérer les aéroports dénommés Aéroports du Cameroun, en abrégé ADC.SA. La société ADC.SA voit donc le jour le 13 décembre 1993.

L'année suivante, plus précisément le 12 août 1994 à Yaoundé, une convention de concession est signée entre l'État du Cameroun et la société camerounaise ADC.SA, Aéroports du Cameroun, société au capital de 463 millions de francs CFA. Cette société assure, depuis le 1er octobre 1994, la gestion effective des sept principaux aéroports du Cameroun, à savoir Douala, Garoua, Yaoundé-Nsimalen, Maroua-Salak, Ngaoundéré, Bamenda et Bertoua. Aéroports du Cameroun est une entreprise camerounaise dotée d'un Conseil d'Administration ; elle est régie par l'Acte Uniforme OHADA au regard de ses statuts, de ses lois et règlements en vigueur. Placée sous l'autorité d'un Directeur Général, cette société aménage et exploite des plateformes aéroportuaires, dont les sept principales sont classées en catégories A et B au Cameroun. C'est le premier groupe aéroportuaire en Afrique centrale. Les services aéroportuaires incluent l'exploitation des aéroports, notamment les terminaux aéroportuaires, les activités des consignataires aériens, les services d'entretien-maintenance des avions (hors réparation) et le contrôle de l'espace aérien au voisinage des aéroports.

ADC est détenteur et gestionnaire de sept aérodromes d'aviation générale. Les revenus sont principalement issus de la rémunération de ses services aéroportuaires. Ils comprennent les redevances aéronautiques (atterrissage, stationnement, carburant, balisage lumineux, passagers), les redevances spécialisées (tri des bagages), les taxes payées par les compagnies aériennes et les passagers, les loyers des boutiques et des terrains, ainsi que les recettes des parcs de stationnement. En 2016, ADC reste la propriété exclusive de l'État du Cameroun. L'entreprise n'est pas cotée en bourse au Cameroun. La répartition de l'actionnariat d'ADC a été établie le 07 juillet 2016.

2. Situation géographique

La société Aéroports du Cameroun est une société qui gère l'aéroport l'ensemble des sept 07 aéroports du Cameroun. Sa direction générale au Cameroun est située à environ 7 kilomètres de la ville de Yaoundé dans l'arrondissement de MFOU au quartier Nsimalen.

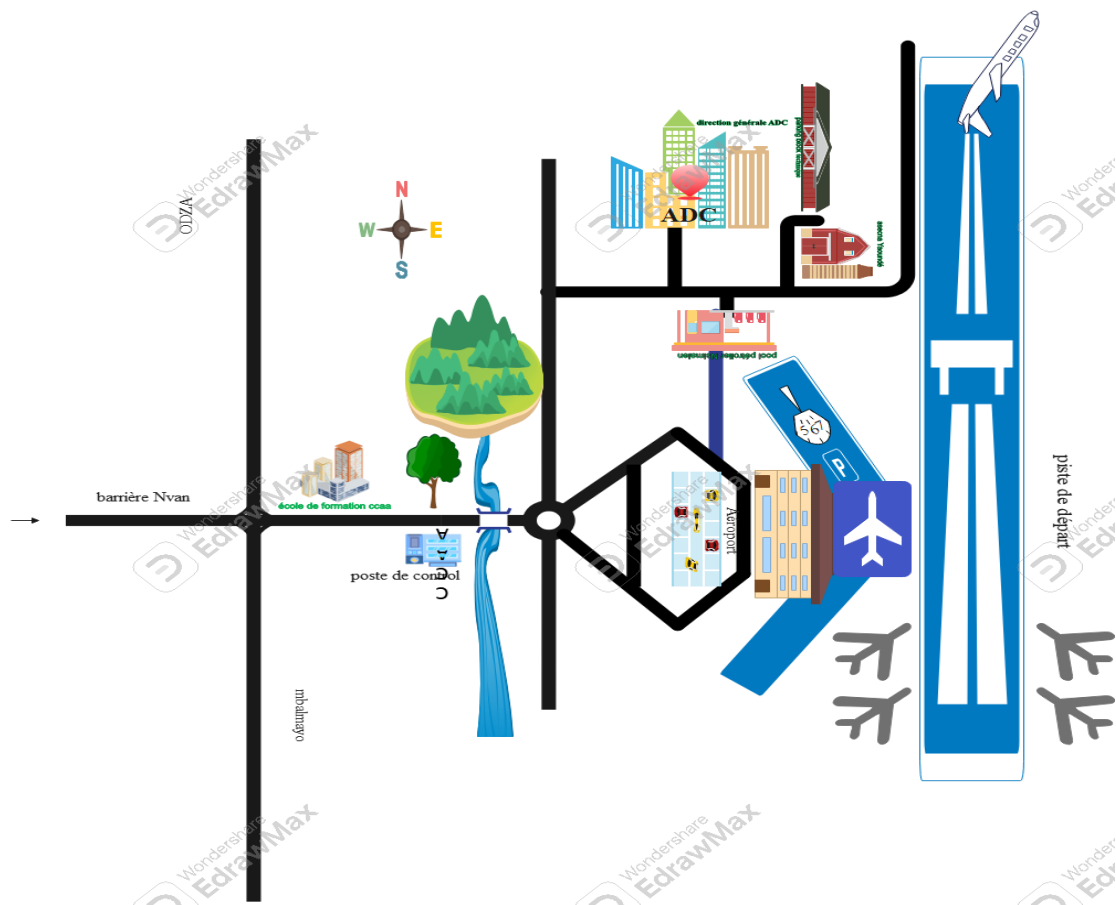


Figure 1 localisation de ADC S.A.

3. Organigramme

À l'entrée de l'enceinte de la société Aéroports du Cameroun se trouve une guérite climatisée qui fait office de lieu d'accueil et de réception. Un esprit de courtoisie et de cordialité anime le personnel d'accueil. Pour l'exécution de ses missions, la société Aéroports du Cameroun S.A comprend :

1. **La Direction Générale** : elle est coiffée par le Directeur Général, qui incarne le top management de la structure. Il coordonne toutes les orientations politiques, administratives, financières et sociales de la société, et est secondé par un Directeur Général Adjoint.
2. **Les services rattachés à la Direction Générale** : les Conseillers Techniques, l'Attaché de Direction, l'Inspection Générale des Services, la Division des Contrôles, le Département des Systèmes d'Information, la Division de la Communication, de la Documentation et de la Traduction, le Service Courrier et Liaison, le Service Suivi et Relance, le Pool du Système d'Information Géographique et le Service de Sécurité Informatique.
3. **Les Directions Centrales** : la Direction de la Comptabilité et des Finances, la Direction des Études et des Projets, la Direction de la Maintenance, la Direction des Ressources Humaines, la Direction des Affaires Générales, la Direction Sécurité, Qualité et Environnement, et la Direction de l'Exploitation.
4. **Les Directions d'Aéroports** : la Direction de l'Aéroport International de Douala, la Direction de l'Aéroport International de Yaoundé-Nsimalen, la Direction de l'Aéroport International de Garoua, la Direction de l'Aéroport International de Maroua-Salak, la Direction de l'Aéroport International de Ngaoundéré, la Direction de l'Aéroport International de Bamenda et la Direction de l'Aéroport International de Bertoua.

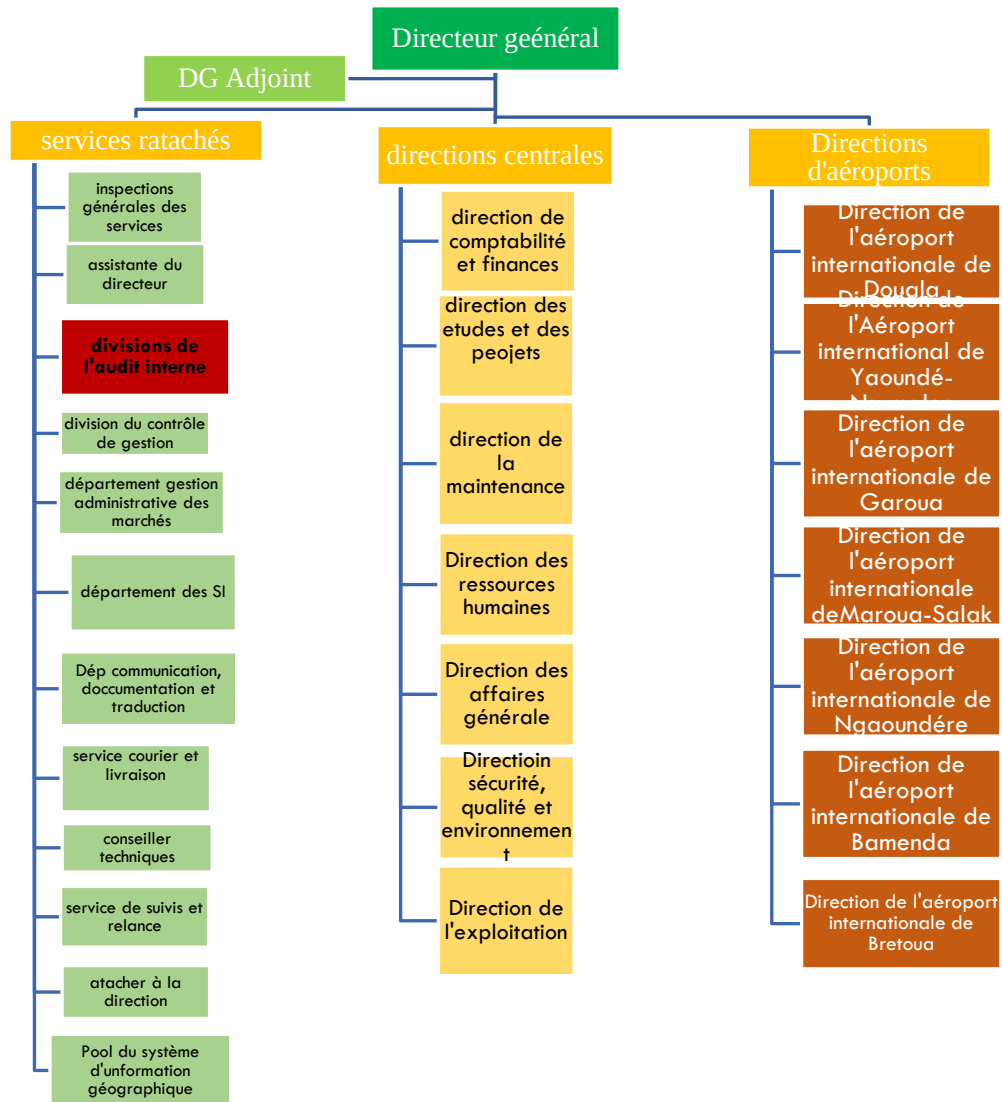


Figure 2 organigramme de ADC S.A.

4. Mission

ADC a pour missions principales : La gestion, l'exploitation, l'entretien, le renouvellement, la réhabilitation et le développement des infrastructures des aéroports. La fourniture des services en escale. Au titre de **la convention de la concession du 13 février 1994**, ADC S.A assure la gestion et l'exploitation de sept aéroports : **Yaoundé-Nsimalen, Douala, Garoua, Maroua, Ngaoundéré, Bamenda, Bertoua.**

5. Activités

Les activités des ADC, c'est-à-dire **Aéroports du Cameroun** (ADC S.A.) au Cameroun, incluent l'aménagement, l'exploitation et l'entretien des sept aéroports du Cameroun, l'amélioration de l'assistance aux voyageurs, aux bagages et aux aéronefs, ainsi que la gestion des installations aéroportuaires. L'entreprise fournit également des services spécifiques aux passagers PMR (Personnes à Mobilité Réduite), comme l'assistance à l'enregistrement et aux formalités de police, le transfert en fauteuil roulant et l'embarquement. En résumé, ADC gère l'ensemble des services opérationnels et de transit des sept aéroports du Cameroun, en mettant l'accent sur la fluidité et la qualité de l'expérience des passagers.

III. RESSOURCES MATÉRIELLES ET LOGICIELLES

1. Ressources matérielles

Dans le cadre de l'exécution de son plan d'actions pour l'année en cours, le Directeur Général de la société Aéroports Du Cameroun S.A, a doté cette structure d'outils informatiques qui sont à la pointe de la technologie

Tableau 1 ressources matérielles de ADC S.A.

N°	Equipements	Caractéristique	Quantité
1	Ordinateurs desktop	Desktop HP, Lenovo, Dell 	600
2	Ordinateur portable	Laptop HP, Lenovo, Dell, Asus, Msi,  	350
3	Routeur	Cisco Catalyst 1941 	4
4	Switch	Switch Cisco Catalyst 24 et 48 ports 	100
5	Internet	Fibre optique	1
6	Modem	Box internet TP Link, Orange, MTN, Cisco 	15
7	Cable Ethernet	Cable Ethernet à paire torsadés CAT6	12Km

2. Ressources logicielles

De nombreux logiciels permettent aux employés des Aéroports du Cameroun de mener à bien l'exécution de leur tâche dans le but de remplir les objectifs semestriels et trimestriels de cette prestigieuse structure et comme logiciels nous avons :

Tableau 2 ressources logicielles de ADC S.A.

N°	Logicielles	Descriptions
01	Suite Microsoft Office 	Pour la création de documents, feuilles de calcul, présentations et autres contenus professionnels.
02	Windows 11 	Système d'exploitation pour machine
03	Visual Studio Code 	IDE (Environnement de développement) gratuit et open source,
04	Angry ip scanner 	Permet de gérer les conflits d'adresse IP et de consulter les adresse Ip disponibles dans un réseau local.
05	Windows server 2022, 2019, 2007 	Pour la mise en place de server
6	VMWare 	Pour virtualiser nos différents serveurs et OS de test et VM
7	Amadeus	Pour la gestion aéroportuaire
8	Metasploit 	Pour la supervision du réseau ADC.SA

CONCLUSION

Cette phase d'insertion nous a permis de découvrir le fonctionnement d'Aéroports Du Cameroun S.A. et de nous familiariser avec l'environnement de la Division de l'Audit Interne, au sein de laquelle s'inscrit notre projet de fin de stage. L'accueil bienveillant de nos encadrants et l'atmosphère de travail détendue nous ont permis de nous intégrer rapidement et de mieux appréhender les enjeux liés à la gestion des missions d'audit interne. Cette immersion constitue le socle sur lequel s'appuiera la suite de notre travail, consacrée à l'analyse des besoins et à la conception de l'application de gestion des missions d'audit interne.

2ÈME PARTIE : PHASE TECHNIQUE

Résumé :

Il a été question pour nous dans la phase d'insertion de nous familiariser avec notre structure d'accueil. L'accent sera mis dans cette phase sur les différentes caractéristiques, spécificités et attentes du sujet soumis à notre étude.

Aperçu :

Introduction

- Dossier 1 : L'Existant
- Dossier 2 : Le cahier des charges
- Dossier 3 : Dossier d'Analyse
- Dossier 4 : Dossier de Conception
- Dossier 5 : Dossier de Réalisation
- Dossier 6 : Test de fonctionnalités
- Dossier 7 : Guide d'installation et guide d'utilisation

Conclusion

INTRODUCTION

La phase technique est la partie du rapport de stage ayant pour but principal la description des besoins d'utilisateurs ainsi que les conditions nécessaires à la réussite du projet pour éviter la production des produits inadéquats. Une fois installé au sein des ADC S.A., et ayant des ressources à notre disposition, nous avons pour devoir d'apporter des solutions et réalisation à ce projet.

DOSSIER 1 : L'EXISTANT

Résumé :

Le dossier de l'existant analyse en détail le système actuel, décrivant ses fonctionnalités, ses performances et ses limitations. Il identifie les besoins non satisfaits et les problèmes à résoudre, fournissant ainsi une base solide pour guider le développement de la nouvelle application. Ce dossier permet de comprendre les forces et faiblesses du système en place et d'établir des recommandations pour améliorer et optimiser la solution future.

Aperçu :

Introduction

1. Présentation du thème/projet
2. Etude de l'existant
3. Critique de l'existant
4. Problématique
5. Proposition de solution

Conclusion

INTRODUCTION

Avant de proposer une solution informatique adaptée aux besoins de la Division de l'Audit Interne, il convient d'abord d'analyser la situation actuelle de gestion des missions d'audit au sein d'Aéroports Du Cameroun S.A., ainsi que les solutions déjà disponibles sur le marché. Ce dossier présente le thème du projet, décrit le mode de fonctionnement actuel, en relève les limites, formule la problématique qui en découle, et propose une orientation de solution.

I. PRÉSENTATION DU THÈME/PROJET

Le thème de notre projet de fin de stage porte sur la conception et le développement d'une application desktop de gestion des missions d'audit interne pour Aéroports Du Cameroun S.A. Ce projet a été confié à la Division de l'Audit Interne, dans le but de doter cette dernière d'un outil centralisé permettant de couvrir l'ensemble du cycle de vie d'une mission d'audit : de la cartographie des risques significatifs jusqu'au suivi de la mise en œuvre des recommandations, en passant par l'élaboration du Plan Annuel d'Audit, la planification et l'exécution des missions, ainsi que la production des rapports.

II. ETUDE DE L'EXISTANT

1. Situation actuelle au sein d'Aéroports Du Cameroun S.A.

À ce jour, la gestion des missions d'audit interne au sein de la Division de l'Audit Interne d'ADC S.A. repose entièrement sur des méthodes manuelles. La planification des missions, le suivi de leur exécution, la rédaction des rapports ainsi que le suivi des recommandations sont réalisées à l'aide d'outils bureautiques classiques (feuilles de calcul Excel, documents Word), sans base de données centralisée ni système dédié. Chaque auditeur ou chef de mission conserve ses propres fichiers de suivi, ce qui rend difficile la consolidation des informations au niveau de la Division et complique le pilotage global de l'activité d'audit.

2. Recherche sur les solutions existantes sur le marché

Afin de mieux situer notre projet, nous avons mené une recherche sur les solutions logicielles existantes dédiées à la gestion de l'audit interne. Plusieurs éditeurs proposent des plateformes reconnues dans ce domaine, parmi lesquelles :

- **TeamMate+** (Wolters Kluwer) : plateforme mature de gestion de l'audit interne, couvrant la planification, l'exécution, la documentation des missions et le suivi des recommandations, utilisée par de nombreuses grandes organisations à travers le monde.
- **AuditBoard** : solution cloud moderne axée sur la gestion connectée des risques, de l'audit et de la conformité, appréciée pour son ergonomie mais dont le coût de licence annuel est particulièrement élevé.
- **Pentana Audit** (Ideagen) : outil orienté secteur bancaire, financier et public, permettant de modéliser l'organisation de l'entreprise et de gérer un nombre illimité de missions d'audit.

- **Diligent** et **MetricStream** : plateformes de gouvernance, risques et conformité (GRC) intégrant des modules d'audit interne, principalement destinées aux grandes entreprises avec des besoins de reporting vers les instances de gouvernance.

Ces solutions, bien que riches fonctionnellement, présentent des caractéristiques communes qui limitent leur adéquation avec le contexte d'ADC S.A. : elles sont pour la plupart proposées en mode SaaS (hébergement dans le cloud de l'éditeur), avec des coûts de licence annuels élevés, souvent inadaptés à la taille d'une division d'audit interne unique, et ne répondent pas à l'exigence d'un hébergement sur les serveurs locaux de l'entreprise.

III. CRITIQUE DE L'EXISTANT

La gestion manuelle actuelle, ainsi que les solutions du marché étudiées, présentent un certain nombre de limites au regard des besoins spécifiques de la Division de l'Audit Interne d'ADC S.A.

L'analyse de la situation actuelle et des solutions existantes fait ressortir les critiques suivantes :

Tableau 3 les limitations de l'existant et leurs solutions

Critiques	Limites	Solutions
Gestion manuelle sur fichiers Excel/Word dispersés	Absence de base de données centralisée, risque de perte ou de version obsolète des documents, difficulté de consolidation	Mettre en place une base de données unique centralisant l'ensemble des informations d'audit
Absence de traçabilité systématique des opérations	Impossible de savoir précisément qui a créé, modifié ou validé une information et à quel moment	Intégrer un journal d'audit historisant automatiquement toutes les actions des utilisateurs
Suivi des recommandations peu structuré	Risque d'oubli ou de retard dans la mise en œuvre des recommandations, faute de rappels et d'indicateurs de suivi	Développer un module dédié au suivi des recommandations avec échéances et statuts
Solutions du marché (TeamMate+, AuditBoard, Pentana...) coûteuses et hébergées dans le cloud	Coûts de licence élevés et hébergement externe incompatible avec l'exigence d'un serveur local ADC S.A.	Concevoir une application sur mesure, hébergée sur les serveurs locaux de l'entreprise, sans coût de licence récurrent

IV. PROBLÉMATIQUE

Face à ces constats, la question qui guide notre projet peut être formulée ainsi : **comment doter la Division de l'Audit Interne d'Aéroports Du Cameroun S.A. d'un outil permettant de centraliser, sécuriser et tracer l'ensemble des informations relatives aux missions d'audit, depuis la cartographie des risques jusqu'au suivi des recommandations, tout en respectant la contrainte d'un hébergement sur les infrastructures locales de l'entreprise, et sans recourir aux solutions du marché dont le coût et le mode d'hébergement en cloud ne correspondent pas aux besoins et aux contraintes d'ADC S.A. ?**

V. PROPOSITION DE SOLUTION

Pour répondre à cette problématique, nous proposons la conception et le développement d'une application desktop sur mesure, dédiée à la gestion des missions d'audit interne : **SIGMA-ADC**, répondant spécifiquement aux besoins et contraintes d'ADC S.A. Cette application reposera sur une architecture client-serveur avec une base de données PostgreSQL unique et centralisée, hébergée sur les serveurs locaux de l'entreprise, garantissant ainsi la maîtrise totale des données sans dépendance à un hébergement externe. Elle couvrira l'ensemble du cycle de vie d'une mission d'audit à travers des modules dédiés : cartographie des risques significatifs, Plan Annuel d'Audit, gestion des missions, gestion documentaire, rapports d'audit, suivi des recommandations et tableaux de bord de pilotage.

CONCLUSION

L'étude de l'existant a permis de mettre en évidence les limites de la gestion manuelle actuelle des missions d'audit interne au sein d'ADC S.A., ainsi que l'inadéquation des solutions du marché avec les contraintes spécifiques de l'entreprise, notamment en matière d'hébergement local et de coût. Ces constats justifient pleinement le développement d'une application sur mesure, dont les besoins fonctionnels détaillés seront présentés dans le dossier suivant : le cahier des charges.

DOSSIER 2 : CAHIER DES CHARGES

Résumé :

Le cahier des charges est un document contractuel établi entre le maître d'œuvre et le maître d'ouvrage qui énonce les besoins du client. Il étudie et présente avec exactitude les exigences formulées par les utilisateurs en ce qui concerne le projet, son déroulement et les résultats.

Aperçu :

Introduction

- I. Contexte et justification de l'étude /du projet
- II. Les objectifs de l'étude/projet
- III. Expressions des besoins de l'utilisateur
- IV. Planification du projet/ étude / Gant project
- V. Estimation du coût du projet/étude
- VI. Les contraintes du projet/ étude
- VII. Les livrables

Conclusion

INTRODUCTION

Tout projet informatique d'envergure repose sur un accord préalable et sans ambiguïté entre celui qui exprime le besoin et celui qui le réalise. Le cahier des charges est le document qui formalise cet accord : il énonce le contexte dans lequel s'inscrit le projet, les raisons qui le justifient, les objectifs poursuivis, les attentes précises des futurs utilisateurs, les moyens humains, matériels et logiciels à mobiliser, ainsi que les délais et les contraintes à respecter. Le présent dossier constitue donc le document de référence du projet **SIGMA-ADC** (*Système Intégré de Gestion des Missions d'Audit — ADC*), application destinée à la Division de l'Audit Interne d'Aéroports du Cameroun S.A. Il fait suite au dossier de l'existant, dont il exploite les conclusions pour traduire les insuffisances relevées en exigences vérifiables. Il servira de base contractuelle à toutes les phases ultérieures : analyse, conception, réalisation et recette. Nous y présentons successivement le contexte et la justification du projet, ses bénéficiaires et sa cible, ses objectifs, l'expression détaillée des besoins fonctionnels et non fonctionnels, la planification des travaux, l'estimation du coût, les contraintes identifiées et enfin les livrables attendus.

I. CONTEXTE ET JUSTIFICATION DE L'ÉTUDE DU PROJET

1. Contexte

Aéroports du Cameroun S.A. (ADC S.A.) est une société anonyme de droit camerounais créée le 13 décembre 1993, régie par l'Acte Uniforme OHADA, dont l'État du Cameroun est l'actionnaire unique. Liée à l'État par une convention de concession signée le 12 août 1994, elle assure depuis le 1er octobre 1994 la gestion, l'exploitation, l'entretien et le développement des sept aéroports du territoire national : Douala, Yaoundé-Nsimalen, Garoua, Maroua-Salak, Ngaoundéré, Bamenda et Bertoua. Premier groupe aéroportuaire d'Afrique centrale, elle tire ses revenus des redevances aéronautiques et spécialisées, des taxes acquittées par les compagnies et les passagers, ainsi que des loyers commerciaux et des parcs de stationnement.

Rattachée à la Direction Générale, la **Division de l'Audit Interne** exerce une activité indépendante et objective qui donne à l'entreprise une assurance sur le degré de maîtrise de ses opérations et lui apporte ses conseils pour les améliorer. Son activité s'organise autour d'un cycle rigoureux : élaboration d'une cartographie des risques significatifs, construction d'un Plan Annuel d'Audit, conduite des missions (orientation, registre des risques, programme de travail, exécution des travaux, feuilles de couverture, FRAP), production des rapports provisoires puis définitifs après contradictoire, et enfin suivi des recommandations jusqu'à leur mise en œuvre effective. Ce déroulement est encadré par la Charte d'Audit Interne, par la Note de service N°106-22/ADC/DG/DI qui fixe la classification et le référencement des pièces du dossier de mission, ainsi que par les normes professionnelles de l'audit interne.

C'est dans ce cadre que s'inscrit notre stage académique de fin de cycle, effectué au sein de la Cellule de l'Audit des Marchés et des Systèmes d'Information, et dont le thème porte sur la conception et le développement d'une application de gestion des missions d'audit interne.

2. Justification

L'étude de l'existant a révélé que l'ensemble de ce cycle est aujourd'hui conduit à l'aide d'outils bureautiques génériques - classeurs Excel et documents Word — stockés sur les postes individuels des auditeurs et sur des supports non centralisés. Il n'existe ni base de données commune, ni référentiel documentaire unique, ni mécanisme automatique de traçabilité.

Cette organisation engendre plusieurs difficultés :

- **La dispersion de l'information** : les pièces d'une même mission se trouvent réparties entre plusieurs postes et plusieurs formats, ce qui rend la reconstitution d'un dossier de mission longue et incertaine ;
- **L'absence de traçabilité** : rien ne permet d'établir avec certitude qui a produit, modifié ou révisé une pièce du dossier, alors même que la traçabilité est une exigence fondamentale de la profession ;
- **La lourdeur du suivi des recommandations** : le suivi s'effectue par ressaisie manuelle d'un tableau d'un exercice à l'autre, avec un risque élevé d'omission et sans alerte sur les échéances dépassées ;
- **L'absence d'indicateurs de pilotage** : le taux d'exécution du Plan Annuel d'Audit, le taux de couverture des risques significatifs ou le taux de mise en œuvre des recommandations ne peuvent être obtenus qu'au prix d'un dépouillement manuel ;
- **La fragilité de la conservation** : la perte ou la corruption d'un fichier local suffit à compromettre une partie du dossier d'audit, dont la valeur probante est pourtant essentielle.

À ces difficultés s'ajoute une contrainte propre au travail de terrain. L'auditeur conduit l'essentiel de ses travaux hors de son bureau : entretiens dans les services audités, visites sur les plateformes aéroportuaires, réunions d'ouverture et de restitution. Il n'y dispose ni de son poste de travail, ni toujours d'une connexion au réseau de l'entreprise. Les comptes rendus d'entretien sont donc pris à la main puis ressaisis au retour, avec une perte d'information proportionnelle au délai écoulé, et la consultation d'un rapport antérieur ou d'une recommandation en cours suppose de différer la question

jusqu'au retour au bureau. Une application de bureau, si complète soit-elle, ne couvre pas ce moment du travail d'audit.

L'examen des solutions disponibles sur le marché - TeamMate+ (Wolters Kluwer), AuditBoard, Pentana Audit (Ideagen), Diligent, MetricStream - a montré qu'elles sont toutes proposées en mode SaaS, hébergées dans le nuage et assorties de licences dont le coût annuel est élevé. Or ADC S.A. impose que les données d'audit, par nature sensibles, demeurent sur ses propres serveurs. Ces solutions sont donc, en l'état, incompatibles avec les exigences de l'entreprise.

Le développement d'une application sur mesure, déployée sur l'infrastructure locale d'ADC et strictement calquée sur la méthodologie de la Division, constitue par conséquent la réponse la plus appropriée. Tel est l'objet du projet SIGMA-ADC.

II. BÉNÉFICIAIRE ET CIBLE

1. Bénéficiaire

Le bénéficiaire du projet est **Aéroports du Cameroun S.A.**, et plus précisément sa **Division de l'Audit Interne**, structure rattachée à la Direction Générale et maître d'ouvrage de l'application. Au-delà de cette division, l'entreprise dans son ensemble bénéficie indirectement du projet, dans la mesure où l'amélioration du dispositif d'audit interne renforce la maîtrise des risques, la qualité du contrôle interne et, in fine, la gouvernance de la société.

2. Cible

La cible désigne les utilisateurs directs de l'application. Elle est constituée de cinq profils :

- **L'Administrateur**, chargé de la gestion des comptes utilisateurs, du paramétrage de l'application et de la mise à jour du référentiel des risques, sans aucun accès au contenu métier des missions ;
- **Le Chef de Division** (Responsable de l'Audit Interne), qui élabore et valide le Plan Annuel d'Audit, oriente les missions, valide en dernier ressort les rapports et les recommandations et prononce la clôture des missions ;
- **Le Chef de Cellule**, qui supervise l'exécution des missions relevant de sa cellule et assure les validations intermédiaires ;
- **L'Auditeur**, qui exécute les missions, alimente le dossier, renseigne les feuilles de couverture, rédige les FRAP et les projets de rapport ;
- **La Direction Générale**, qui consulte en lecture seuls les tableaux de bord et les rapports validés.

III. LES OBJECTIFS DE L'ÉTUDE PROJET

1. OBJECTIF GÉNÉRAL

L'objectif général du projet est de **concevoir et développer une application sécurisée, centralisée et évolutive, hébergée sur les serveurs locaux d'ADC S.A., couvrant l'intégralité du cycle de vie d'une mission d'audit interne, de la programmation annuelle jusqu'au suivi des recommandations et à l'archivage du dossier.**

2. OBJECTIFS SPÉCIFIQUES

De cet objectif général découlent les objectifs spécifiques suivants :

- Centraliser dans une base de données unique l'ensemble des informations et des documents relatifs aux missions d'audit ;
- Automatiser l'élaboration et le suivi du Plan Annuel d'Audit à partir de la cartographie des risques significatifs ;
- Dématérialiser le dossier de mission en respectant la classification officielle en dossier synthétique, dossier de travail et dossier administratif, ainsi que les codes de référencement en vigueur ;
- Encadrer le circuit de validation successive des travaux, de l'auditeur au Chef de Division, en garantissant la séparation des responsabilités ;
- Structurer le suivi des recommandations issues des rapports définitifs validés, avec statut, taux d'avancement, échéance, responsable métier et référent audit ;
- Assurer la traçabilité intégrale des opérations au moyen d'un journal d'audit applicatif ;
- Produire automatiquement les tableaux de bord et indicateurs de pilotage de l'activité d'audit, avec exports au format PDF et Excel ;

- Garantir la confidentialité et l'intégrité des données par une gestion fine des habilitations fondée sur les profils utilisateurs.
- Doter l'auditeur d'un accès mobile aux informations de ses missions, exploitable en situation de mobilité et en l'absence de connexion ;
- Assister la prise de notes lors des entretiens et des réunions par un enregistrement audio suivi d'une transcription automatique, sous réserve du consentement préalable des personnes présentes ;
- Permettre l'interrogation en langage naturel de l'historique d'audit de l'entreprise, chaque réponse citant obligatoirement les documents sur lesquels elle se fonde.

IV. EXPRESSIONS DES BESOINS DE L'UTILISATEUR

1. BESOINS FONCTIONNELS

Les besoins fonctionnels décrivent les services que l'application doit rendre à ses utilisateurs. Ils se répartissent en neuf modules.

Module 1 - Gestion des utilisateurs et des habilitations. Créer, modifier, activer et désactiver les comptes utilisateurs ; affecter un profil à chaque compte ; authentifier l'utilisateur par identifiant et mot de passe ; restreindre l'accès aux fonctionnalités selon le profil ; consulter le journal des opérations.

Module 2 - Cartographie des risques significatifs. Consulter le référentiel des risques ; rechercher un risque par processus, direction ou niveau de criticité ; coter chaque risque selon la probabilité et l'impact et en déduire automatiquement la criticité ; rattacher un ou plusieurs risques à une mission ; visualiser les risques couverts et non couverts par le Plan Annuel d'Audit.

Module 3 - Plan Annuel d'Audit. Élaborer le plan d'un exercice à partir des risques prioritaires ; y inscrire les missions en précisant leur nature (Gouvernance, Assurance, Conseil, Suivi) ; valider le plan, un seul plan validé étant admis par exercice ; suivre son taux d'exécution.

Module 4 - Gestion des missions. Créer une mission planifiée ou hors plan ; désigner le chef de mission et l'équipe ; formaliser l'orientation de la mission ; constituer le registre des risques de la mission ; élaborer le programme de travail à partir de ce registre ; organiser les réunions d'ouverture et de restitution ; saisir les feuilles de couverture de test et d'entretien ; établir les FRAP avec constat, causes, risques, recommandations proposées et réponse de l'audité ; suivre l'avancement ; clôturer la mission.

Module 5 - Gestion documentaire. Déposer, consulter, télécharger et archiver les documents d'une mission ; classer chaque document par catégorie et code de référence ; marquer un document comme confidentiel ; conserver l'historique des versions,

chaque réimport donnant lieu à une nouvelle version sans écrasement de la précédente ; interdire toute modification ou suppression d'un document archivé.

Module 6 - Rapports d'audit. Élaborer le rapport provisoire ; intégrer les observations issues de la procédure contradictoire ; produire le rapport définitif ; formuler l'opinion d'audit ; conserver l'historique complet des versions.

Module 7 - Suivi des recommandations. Enregistrer une recommandation exclusivement à partir d'un rapport définitif validé ; désigner un responsable métier et un référent audit ; suivre le statut, le taux d'avancement, l'échéance et les justificatifs fournis ; statuer sur l'acceptation ou la non-acceptation motivée ; conduire des campagnes de suivi successives.

Module 8 - Annotations et notifications. Permettre au Chef de Cellule ou au Chef de Division de laisser une note sur un élément en cours de revue ; notifier automatiquement les membres de l'équipe concernée ; consulter et marquer comme lues les notifications reçues.

Module 9 - Tableaux de bord et évaluation. Calculer et afficher le taux d'exécution du plan, le taux de couverture des risques, la répartition des missions, l'état des recommandations et le taux de leur mise en œuvre ; recueillir et consolider les fiches d'évaluation post-mission ; exporter les tableaux de bord aux formats PDF et Excel.

Module 10 — Assistance mobile à l'auditeur. Consulter en mobilité les missions auxquelles l'auditeur est affecté, leur documentation et les recommandations dont il est référent ; consulter ces informations hors connexion à partir d'un cache local chiffré ; ouvrir une séance d'entretien ou de réunion en déclarant les participants et en recueillant explicitement leur consentement à l'enregistrement ; basculer en mode notes lorsque le consentement est refusé ; enregistrer la séance et mettre l'enregistrement en file d'attente d'envoi vers le serveur dès le retour de la connexion ; obtenir la transcription automatique de l'enregistrement, découpée en segments horodatés ; corriger l'attribution des locuteurs ; exporter un brouillon d'entretien à partir de la transcription ; interroger en langage naturel l'historique des missions, des rapports et des recommandations ; consulter les sources citées à l'appui de chaque réponse ; purger du terminal les fichiers audio après leur transmission.

2. Besoins non fonctionnels

Les besoins non fonctionnels expriment les qualités attendues de la solution.

- **Sécurité** : authentification obligatoire sur les deux applications ; mots de passe stockés sous forme hachée ; contrôle des habilitations effectuées systématiquement côté serveur ; accès aux missions restreint aux seuls auditeurs qui y sont affectés.
- **Confidentialité et localisation des données** : hébergement exclusif sur les serveurs locaux d'ADC S.A., à l'exclusion de tout service en nuage. Cette exigence s'étend au traitement de la parole et à l'inférence du modèle de langage, qui s'exécutent l'un et l'autre sur l'infrastructure de l'entreprise. Aucune donnée d'audit, aucun enregistrement, aucune requête ne transite par un service externe.
- **Chiffrement et rémanence** : le cache local du terminal mobile et les fichiers audios en attente d'envoi sont chiffrés ; les enregistrements sont effacés du terminal dès leur transmission confirmée ; le cache est vidé à la déconnexion.
- **Fonctionnement hors connexion** : l'application mobile reste utilisable en consultation et en enregistrement sans accès au réseau, la synchronisation s'effectuant automatiquement au rétablissement de la liaison.
- **Intégrité du dossier d'audit** : l'assistant mobile accède au système d'audit exclusivement en lecture. Aucune information produite par transcription ou par génération automatique n'est portée au dossier de mission sans ressaisie et validation explicites par l'auditeur.
- **Traçabilité des réponses générées** : toute réponse produite par l'assistant cite les documents, segments d'entretien ou rapports sur lesquels elle se fonde, l'auditeur pouvant remonter à la source.
- **Performance** : temps de réponse de l'authentification inférieur à 2 secondes ; recherche multicritère restituée en moins de 2 secondes ; génération d'un export inférieure à 5 secondes ; transcription d'une heure d'enregistrement rendue en moins de trente minutes en traitement différé.
- **Fiabilité et intégrité** : les règles de gestion critiques sont garanties au niveau de la base de données, indépendamment des applications clientes ; aucune suppression physique des données historisées.

- **Traçabilité** : toute opération de création, de modification ou de validation est horodatée et rattachée à son auteur.
- **Ergonomie** : interfaces en langue française, homogènes entre le poste de travail et le terminal mobile, conformes au vocabulaire métier de l'audit interne et exploitables après une formation courte.
- **Disponibilité** : les applications doivent être opérationnelles durant l'ensemble des heures ouvrables de l'entreprise.
- **Maintenabilité et évolutivité** : architecture en couches et code documenté, permettant l'ajout ultérieur de nouveaux modules sans remise en cause de l'existant.
- **Portabilité** : le poste client fonctionne sur l'environnement de travail standard d'ADC S.A. ; l'application mobile fonctionne sur les terminaux Android en service dans la Division.
- **Conformité** : respect de la Charte d'Audit Interne, de la Note de service N°106-22/ADC/DG/DI et des standards techniques du Département des Systèmes d'Information.

V. PLANIFICATION DU PROJET / GANTT PROJECT

1. Intervenants

Tableau 4 listes des intervenants

Intervenant	Qualité	Rôle dans le projet
Aéroports du Cameroun S.A. - Division de l'Audit Interne	Maître d'ouvrage	Exprime le besoin, valide les livrables et réceptionne l'application
NKONO NDEME Miguel	Maître d'œuvre	Analyse, conception, développement, tests et documentation
Mme EFFALA MEVONO épouse HERTZ	Chef de la Division de l'Audit Interne	Encadrement de proximité, expertise fonctionnelle
Mme ONDOBO EBELLA épouse NGUELE	Chef de la Cellule de l'Audit des Marchés et des Systèmes d'Information	Encadrement professionnel, validation métier
Encadreur académique	Superviseur académique	Suivi méthodologique et validation scientifique

2. Diagramme de Gantt

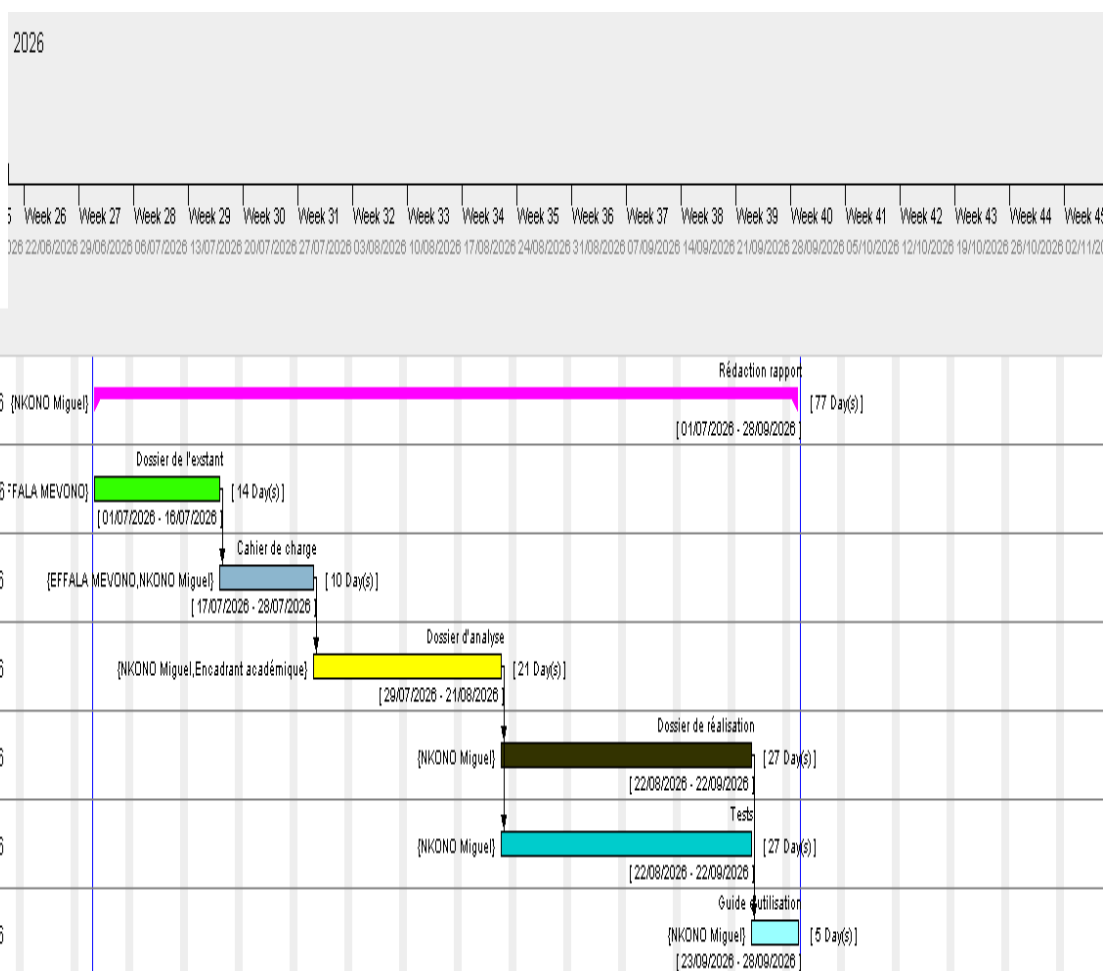


Figure 3 diagramme de gantt

VI. ESTIMATION DU COÛT DU PROJET

1. Ressources Matérielles

Tableau 5 ressources matérielles

Ressource	Quantité	Cout unitaire (FCFA)	Cout total (FCFA)
Ordinateur portable de développement (Core i7, 16 Go RAM, SSD 512 Go)	1	450 000	450 000
Serveur d'application et de base de données	1	1 200 000	1 200 000
Serveur d'inférence (8 cœurs, 32 Go RAM, SSD 1 To)	1	1 800 000	1 800 000
Terminaux Android de terrain (6 Go RAM, 128 Go)	5	150 000	750 000
Onduleur 1000 VA	2	85 000	170 000
Disque dur externe 1 To (sauvegardes)	1	45 000	45 000
Connexion Internet (3 mois)	3	25 000	75 000
Total			4 490 000

2. Ressources logicielles

Tableau 6 ressources logicielles

Ressource	Logo/Description	Cout unitaire (FCFA)
Java Development Kit 21	 Environnement d'exécution et de compilation - licence libre	0
Spring Boot	 Cadriciel du serveur applicatif - licence Apache 2.0	0
Flutter	Cadriciel de développement de l'application mobile - licence BSD	0
FastAPI	Serveur d'application du volet mobile - licence MIT	0
Whisper	Moteur de transcription automatique de la parole - licence MIT	0
Ollama	Serveur d'exécution locale du modèle de langage - licence MIT	0
Modèle de langage quantifié	Modèle à poids ouverts exécuté localement	0
JavaFX + AtlantaFX	  Bibliothèque d'interface graphique du client - licence libre	0
PostgreSQL 16	 Système de gestion de base de données relationnelle - licence libre	0

IntelliJ IDEA Community Edition	 Environnement de développement intégré	0
Astah UML	 Modélisation UML - licence académique	0
GanttProject	 Planification du projet - licence libre	0
Git	 Gestion de versions du code source	0
Suite bureautique Microsoft Office	 Rédaction du rapport et des livrables documentaires	45000
Total		45 000

3. Ressources humaines

Tableau 7 ressources humaines

Fonction	Quantité	Durée (Jour/semaine)	Cout(jour/semaine)	Cout total (FCFA)
Chef de projet	1	13	50 000	650 000
Analyste- concepteur	1	7	75 000	525 000
Développeur applicatif (poste de travail)	1	8	75 000	600 000
Développeur mobile	1	5	75 000	375 000
Testeur	1	4	50 000	200 000
Total				2 350 000

4. Cout total

Figure 4 cout totla

Ressources	Total (FCFA)
Ressources matérielles	4 490 000
Ressources logicielles	45 000
Ressources humaines	2 350 000
Total provisoire	6 885 000
Imprévus (10 %)	688 500
Total	7 573 500

VII. LES CONTRAINTES DU PROJET

Le projet est soumis aux contraintes suivantes :

Contraintes techniques. L'ensemble des composants doit être déployé sur les serveurs locaux d'ADC S.A. ; le recours à toute solution hébergée dans le nuage est formellement exclu. Cette exclusion s'étend au traitement automatique de la parole et à l'inférence du modèle de langage, ce qui impose des composants exécutables localement plutôt que des interfaces de programmation externes. Les applications doivent s'intégrer à l'environnement de travail existant, fonctionner sur les postes Windows et les terminaux Android en service dans la Division, et respecter les standards techniques définis par le Département des Systèmes d'Information.

Contraintes de capacité. L'exécution locale du modèle de langage et du moteur de transcription mobilise des ressources mémoire et processeur importantes. En l'absence de processeur graphique dédié, la transcription s'effectue en traitement différé et non en temps réel : cette limitation est structurante et doit être intégrée à la conception de l'application mobile.

Contraintes fonctionnelles. La solution doit épouser fidèlement la méthodologie d'audit en vigueur : classification et référencement des pièces conformes à la Note de service N°106-22/ADC/DG/DI, circuit de validation successive, séparation stricte des responsabilités entre celui qui exécute et celui qui valide. Aucune simplification du processus métier n'est admise.

Contraintes de sécurité et de confidentialité. Les données manipulées portent sur des constats d'audit et des informations sensibles relatives au fonctionnement de l'entreprise. Leur confidentialité, leur intégrité et leur non-répudiation doivent être garanties à tout moment. L'assistant mobile n'accède au système d'audit qu'en lecture, sans aucune possibilité d'écriture, et aucune information produite automatiquement n'est portée au dossier de mission sans validation explicite de l'auditeur.

Contraintes déontologiques et juridiques. L'enregistrement d'un entretien ou d'une réunion est subordonné au consentement préalable et explicite de l'ensemble des

personnes présentes. L'application doit rendre ce recueil obligatoire et proposer un mode de repli sans enregistrement lorsque le consentement est refusé.

Contraintes de délai. L'ensemble des travaux doit être conduit dans la durée du stage académique, ce qui impose un rythme soutenu et une priorisation rigoureuse des modules.

Contraintes humaines. L'équipe de réalisation se réduit à un seul développeur, ce qui limite la parallélisation des tâches et rend la planification sensible à tout retard.

Contraintes budgétaires. Le budget mobilisable étant limité, le recours à des technologies libres et gratuites constitue une exigence et non une simple préférence.

VIII. LES LIVRABLES

Au terme du projet, les livrables suivants seront remis à la Division de l'Audit Interne d'ADC S.A. :

Livrables logiciels

- L'application de bureau SIGMA-ADC : module client et module serveur, sous forme exécutable ;
- L'application mobile SIGMA-ADC, sous forme de paquet d'installation Android ;
- Le serveur d'application du volet mobile et sa configuration de déploiement ;
- La configuration du serveur de transcription et du serveur d'inférence ;
- Le code source complet et documenté de l'ensemble des composants ;
- Les scripts de création des deux bases de données, leurs migrations versionnées, les vues de consultation en lecture seule et le rôle applicatif restreint associé ;
- Un jeu de données de démonstration permettant la prise en main des deux applications.

Livrables documentaires

- Le dossier de l'existant ;
- Le cahier des charges (le présent dossier) ;
- Le dossier d'analyse ;
- Le dossier de conception ;
- Le dossier de réalisation ;
- Le dossier de tests de fonctionnalités ;
- Le guide d'installation et de déploiement ;
- Le guide de l'utilisateur de l'application de bureau ;
- Le guide de l'utilisateur de l'assistant mobile ;
- Le rapport de stage.

CONCLUSION

Ce cahier des charges a permis de circonscrire précisément le projet SIGMA-ADC. Après avoir situé le projet dans le contexte d'Aéroports du Cameroun S.A. et justifié sa nécessité au regard des insuffisances de la gestion manuelle actuelle, nous avons identifié ses bénéficiaires et sa cible, défini son objectif général et ses objectifs spécifiques, puis détaillé les besoins fonctionnels répartis en dix modules ainsi que les exigences non fonctionnelles attendues de la solution. La planification retenue étale les travaux sur cent cinq jours, du 1er juillet au 26 octobre 2026, pour un coût global estimé à 7 573 500 francs CFA. Les contraintes techniques, fonctionnelles, sécuritaires, déontologiques et temporelles ont enfin été explicitées, de même que la liste des livrables attendus.

DOSSIER 3 : DOSSIER D'ANALYSE

Résumé :

Le dossier d'analyse a pour but d'affiner le choix de la méthode à utiliser tout au long du projet. Il permet dans la même lancée de faire comme son nom l'indique une analyse détaillée du projet à mettre en œuvre.

Aperçu :

Introduction

1. Méthodologie

1.1. Etude comparative UML et MERISE

1.2. Etude comparative des processus unifiés

2. Modélisation

2.1. Diagramme des cas d'utilisation

2.2. Descriptions textuelles

2.3. Diagramme de communication

2.4. Diagramme de séquence

2.5. Diagramme d'activité

Conclusion

INTRODUCTION

La réalisation d'un bon projet repose principalement sur son analyse. Ainsi, un projet mal analysé ne pourrait donner de bons résultats. Les résultats de l'analyse ne dépendent d'aucune technologie particulière. Le but principal de cette partie est d'indiquer comment le travail sera organisé et mené. Pour ce faire, le choix d'une démarche d'analyse permettant un suivi aisé du développement de notre plateforme est primordial. A cet effet, le choix s'est posé sur le processus 2TUP du langage UML. Il est donc question pour nous dans cette partie, de présenter de façon détaillée la démarche d'analyse, de justifier le choix de cette démarche par rapport à notre champ d'étude, de choisir le processus unifié et enfin de les appliquer pour la modélisation des premiers diagrammes.

I. MÉTHODE ET LANGAGE D'ANALYSE

L'analyse est une étape fondamentale dans la conception d'un logiciel. C'est la base de tous travaux de réalisation de système d'information. Un système d'information est un système organisé de ressources, de personnes, et de structures qui évoluent dans une organisation et dont le comportement coordonné vise à atteindre un but commun. Plusieurs méthodes de langages ont été développées pour faciliter et normaliser l'analyse et la conception des systèmes d'information parmi lesquels nous avons principalement UML et MERISE.

1. Etude comparative UML et MERISE

MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise) est une méthode d'analyse, de conception structurelle et de réalisation des systèmes d'informations très utiles notamment dans les entreprises françaises. Elle est basée sur la séparation des données et des traitements à effectuer en plusieurs modèles conceptuels et physiques. Son but principal est d'arriver à concevoir un système d'information (SI). C'est une méthode systémique d'analyse et de conception de SI qui propose de considérer le système réel selon deux points de vue : une vue statique (données) et une vue dynamique (traitement).

UML (Unifie Modelling Language) quant à lui, est un langage de modélisation des systèmes standards, qui utilise des diagrammes pour présenter chaque aspect d'un système en s'appuyant sur la notion d'orienté objet qui est un véritable atout pour ce langage. UML propose donc une approche différente de celle de MERISE en ce sens qu'il associe les données aux traitements. En effet, avec UML, centraliser les données d'un type de traitement associé permet de limiter les points de maintenance dans le code et faciliter l'accès à l'information en cas d'évolution de logiciel. De plus, UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.

Tableau 8 comparaison entre UML et MERISE

MERISE	UML
Méthode d'étude et de réalisation informatique pour les systèmes d'entreprises.	Unified Modeling Language
Est une méthode systémique d'analyse et de conception de systèmes d'information. C'est-à-dire qu'elle utilise une approche systémique.	N'est cependant pas une méthode, mais plutôt un langage de modélisation objet qu'il faut associer une démarche pour en faire une méthode. C'est le cas de la méthode 2TUP ; RUP et XP.
Propose de considérer le système réel selon deux points de vue : Une vue statique (données). Une vue dynamique (traitements). C'est-à-dire qu'avec la méthode Merise, nous avons une étude séparée des données et des traitements.	Propose une approche différente de celle de Merise en ce sens qu'il associe les données et les traitements. Car avec UML, centraliser les données d'un type et les traitements associés permet de limiter les points de maintenance dans le code et facilite l'accès à l'information en cas d'évolution du Logiciel. De plus ; UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.

2. Choix et présentation de la démarche d'analyse

Après cette étude comparative entre les méthodes UML et MERISE, notre choix sera porté sur la méthode UML.

UML se définit comme étant un langage de modélisation graphique et textuel destiné à l'architecture, la conception et la mise en œuvre de systèmes logiciels complexes par leurs structures aussi bien que leurs comportements, concevoir des solutions et communiquer des points de vue. L'UML utilise les points forts de ces trois approches pour présenter une méthodologie plus cohérente et plus facile à utiliser. Il représente les meilleures pratiques de création et de documentation des différents aspects de la modélisation des systèmes logiciels et d'entreprise. UML unifie à la fois les notions et les concepts orientés objets. Il ne s'agit pas d'une simple notation graphique, car les concepts transmis par un diagramme ont une sémantique précise et sont porteurs de sens au même titre que les mots d'un langage. C'est un langage standard de modélisation des systèmes d'informations à objet.

La version d'UML utilisée ici est la version 2.5.1 qui compte quatorze (14) diagrammes, et une division du système en deux (02) grandes vues :

- ❖ La vue statique composée des diagrammes UML structurels qui représente le système physiquement et comporte six (07) diagrammes
 - Diagramme de classe
 - Diagramme de composants
 - Diagramme de structures composite
 - Diagramme de déploiement
 - Diagramme d'objets
 - Diagramme de packages
 - Diagramme de profil
- ❖ La vue dynamique constituée des diagrammes UML comportementaux qui représente les interactions effectuées dans le système, elle comporte sept (07) diagrammes :
 - Diagramme d'activité
 - Diagramme de communication

- Diagramme global d'interaction
- Diagramme de séquence
- Diagramme d'états-transition
- Diagramme de temps

- Diagramme de cas d'utilisation

UML étant un langage formel et normalisé, il est un bon support de communication dans l'élaboration d'une solution informatique, sa polyvalence et sa souplesse font de lui un langage universel. Pour développer des solutions, on a besoin de lui associer une méthode générique qui s'attache à ses diagrammes, par exemple la méthode Agile. Il en existe plusieurs : Scrum, XP (Extreme Programming), RUP (Rational Unified Process), FDD (Feature Driven Development), etc...

3. Etude comparative des processus unifiés

La maîtrise des processus de développement implique une organisation et un suivi des activités : c'est ce à quoi s'attachent les différentes méthodes qui s'appuient sur l'utilisation du langage UML pour modéliser un système d'information. **UP (Unified Process)** est une méthode générique de développement de logiciel. Générique signifie qu'il est nécessaire d'adapter UP au contexte du projet, de l'équipe, du domaine et/ou de l'organisation (exemple : **R.U.P, U.P, X.P, 2 TUP...**).

Parmi les PU les plus populaires, on trouve :

- **Rational Unified Process (RUP)** : Développé par Rational Software, RUP est l'un des PU les plus complets et les plus structurés. Il est basé sur une approche en cascade avec des phases distinctes et des livrables bien définis.
- **Unified Process (UP)** : UP est une version plus légère et plus adaptable de RUP, créée par l'Object Management Group (OMG). Il offre plus de flexibilité aux équipes pour adapter le processus à leurs besoins spécifiques.

- **Extrême Programming (XP)** : XP est une méthodologie agile qui met l'accent sur la collaboration, les tests fréquents et les livraisons incrémentielles. Il est particulièrement adapté aux projets avec des exigences changeantes.
- **Scrum** : Scrum est une autre méthodologie agile qui utilise des cycles de développement courts appelés sprints. Il met l'accent sur la communication et la visibilité entre les membres de l'équipe.

✦ Choisir le processus unifié adéquat

Le choix du PU le plus adapté à un projet dépend de plusieurs facteurs, tels que la taille et la complexité du projet, les compétences de l'équipe, les exigences du client et la culture de l'entreprise.

- **RUP** est un bon choix pour les projets de grande envergure et critiques où une structure et une planification détaillées sont importantes.
- **UP** est un bon choix pour les projets qui nécessitent plus de flexibilité et d'adaptation.
- **XP** est un bon choix pour les projets avec des exigences changeantes et une équipe expérimentée.
- **Scrum** est un bon choix pour les projets qui nécessitent une livraison rapide et une communication efficace entre les membres de l'équipe.

4. Choix et présentation du processus unifié

Pour des raisons d'adaptation du processus unifié (UP), nous optons pour le processus **2TUP (Two Tracks Unified Process)** qui est construit sur le langage UML. Le processus unifié répète un certain nombre de fois une série de cycles. Tout cycle se conclut par la livraison d'une version du produit aux clients et s'articule en 4 phases :

Création, élaboration, construction et transition, chacune d'entre elles se subdivisant à son tour en itérations.

Dans le processus **2TUP**, les activités de développement sont organisées suivant 5 workflows qui décrivent :

- La capture des besoins ;
- L'analyse ;
- La conception ;
- L'implémentation ;
- Et le test.

Il commence par une étude préliminaire qui consiste essentiellement à identifier les acteurs qui vont interagir avec le système à construire, les messages qu'échangent les acteurs et le système, par la suite de produire le cahier des charges et à modéliser le contexte.

Ce processus vise à améliorer l'efficacité du développement en dissociant les aspects techniques des aspects fonctionnels du système. Il s'articule autour de 3 phases essentielles à savoir : **une branche technique, une branche fonctionnelle et une phase de réalisation.**

Le schéma ci-dessous est l'illustration du processus **2TUP** :

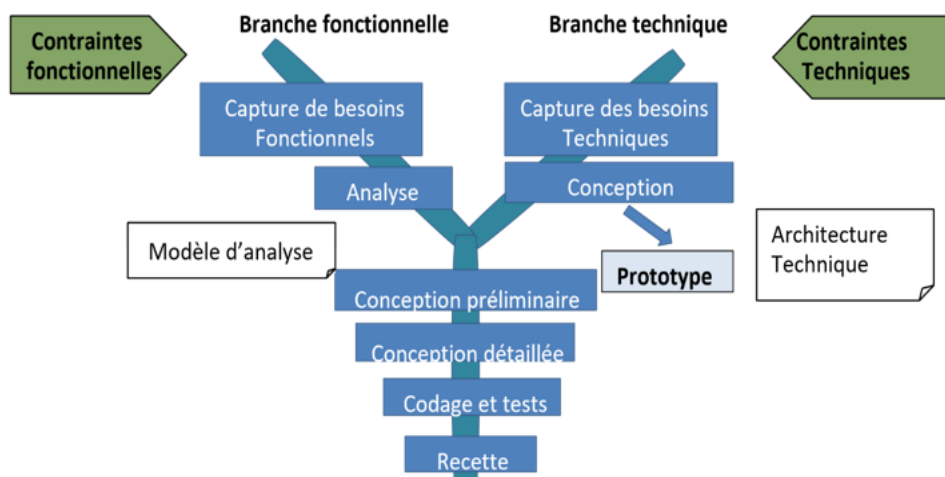


Figure 5 2TUP

II. MODÉLISATION

1. Diagramme des cas d'utilisation

Les diagrammes de cas d'utilisation identifient les fonctionnalités fournies par le système, ces fonctionnalités sont appelées ici cas d'utilisations, les utilisateurs qui interagissent avec le système (acteurs), et les interactions entre ces derniers. C'est une description de l'ensemble des opérations que l'utilisateur pourra effectuer dans le système.

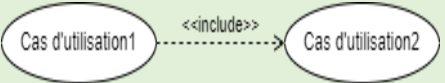
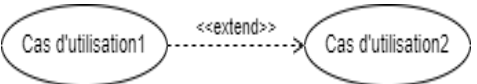
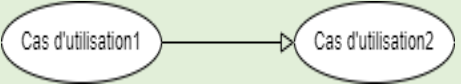
Les cas d'utilisations sont représentés par une ellipse contenant le nom du cas d'utilisation. Un acteur et un cas d'utilisation sont mis en relation par une association représentée par **une ligne**.

Tableau 9 composant d'un diagramme de cas d'utilisation

Composant du diagramme	Représentation Graphique
Acteur	 Acteur
Cas d'utilisation	 Cas d'utilisation
Association	

Trois types de relations sont pris en charge par la norme UML. Nous pouvons les distinguer dans le tableau ci-dessous :

Tableau 10 types de relations

Type de relation	Description	Représentation graphique
Inclusion	Un cas 1 est inclus dans un cas 2 si l'exécution du cas 1 passe obligatoirement par l'exécution du cas 2.	
Extension	On dit qu'un cas 1 étend un autre cas 2 si la réalisation du cas 1 ajoute un ou plusieurs comportements à la réalisation du cas 2.	
Généralisation (Héritage)	Un cas 1 hérite d'un cas 2 si le cas 1 en plus de réaliser complètement les actions du cas 2 réalise ou non d'autres nouvelles actions.	

Exemple de diagramme de cas d'utilisation :

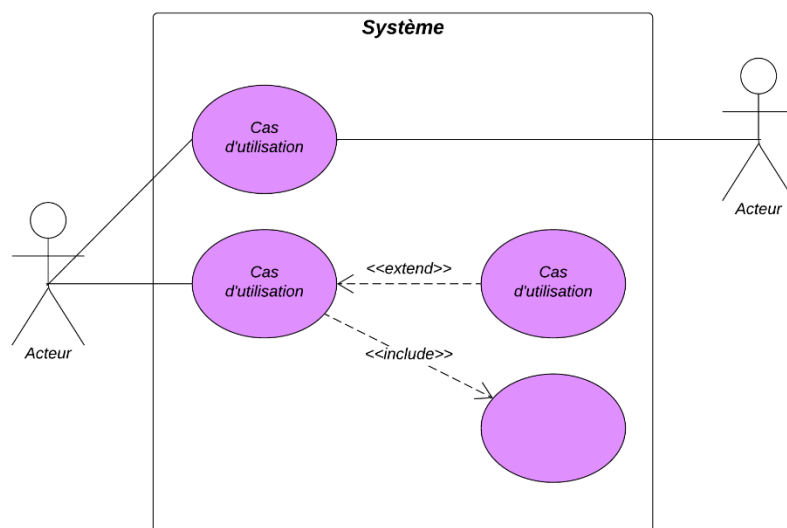


Figure 6 exemple de diagramme de cas d'utilisation

1. Diagramme de cas d'utilisation globale

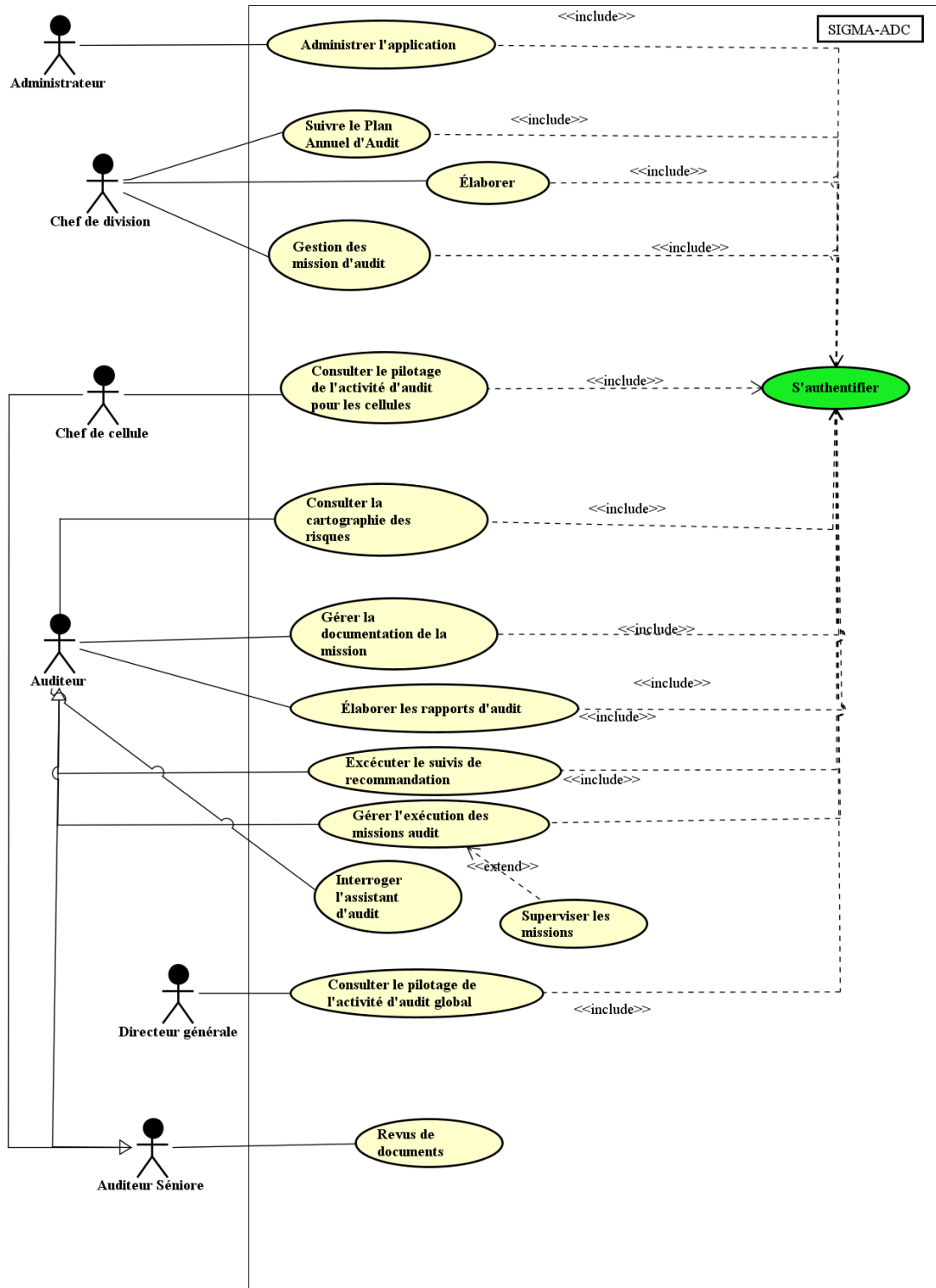


Figure 7 diagramme cas d'utilisation global

2. Diagrammes de cas d'utilisation spécifique

a. Cas d'utilisation : Gérer les missions d'audit

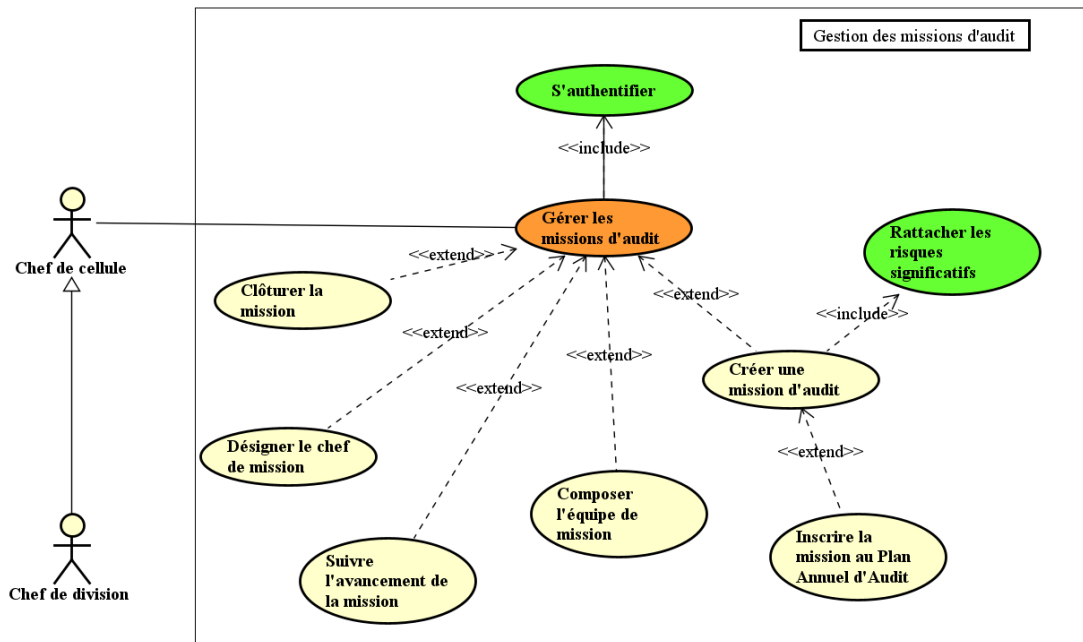


Figure 8 diagramme spécifique du cas d'utilisation : Gérer les missions d'audit

b. Cas d'utilisation : Gérer la documentation de la mission

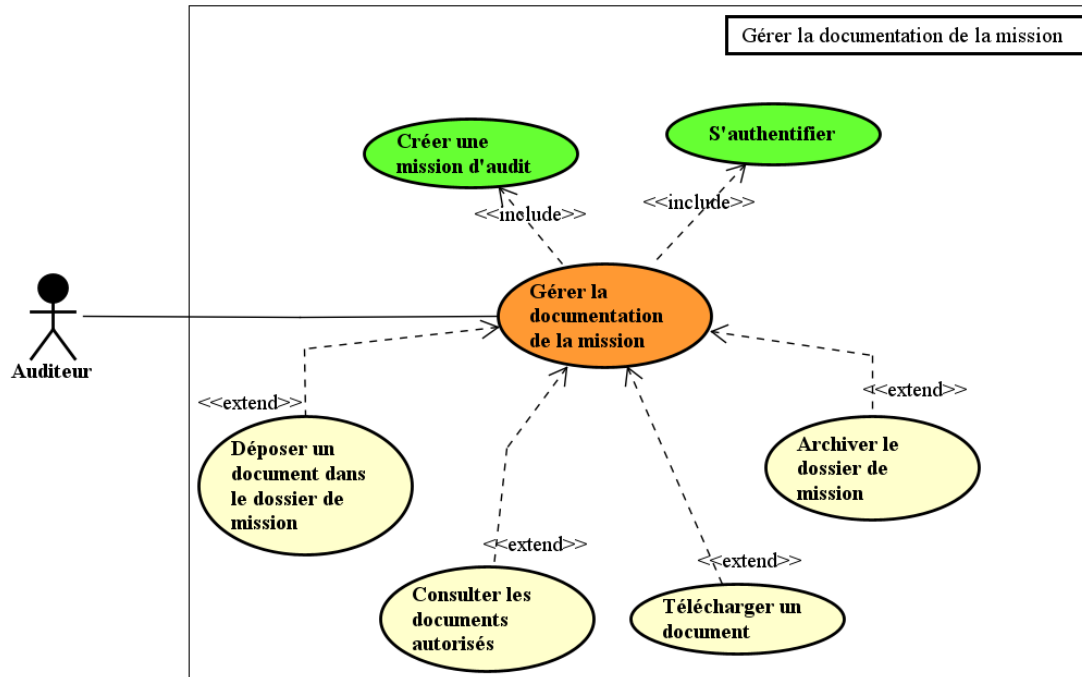


Figure 9 diagramme spécifique tu cas d'utilisation : Gérer la documentation de la mission

c. Cas d'utilisation : Gérer l'exécution des missions d'audit

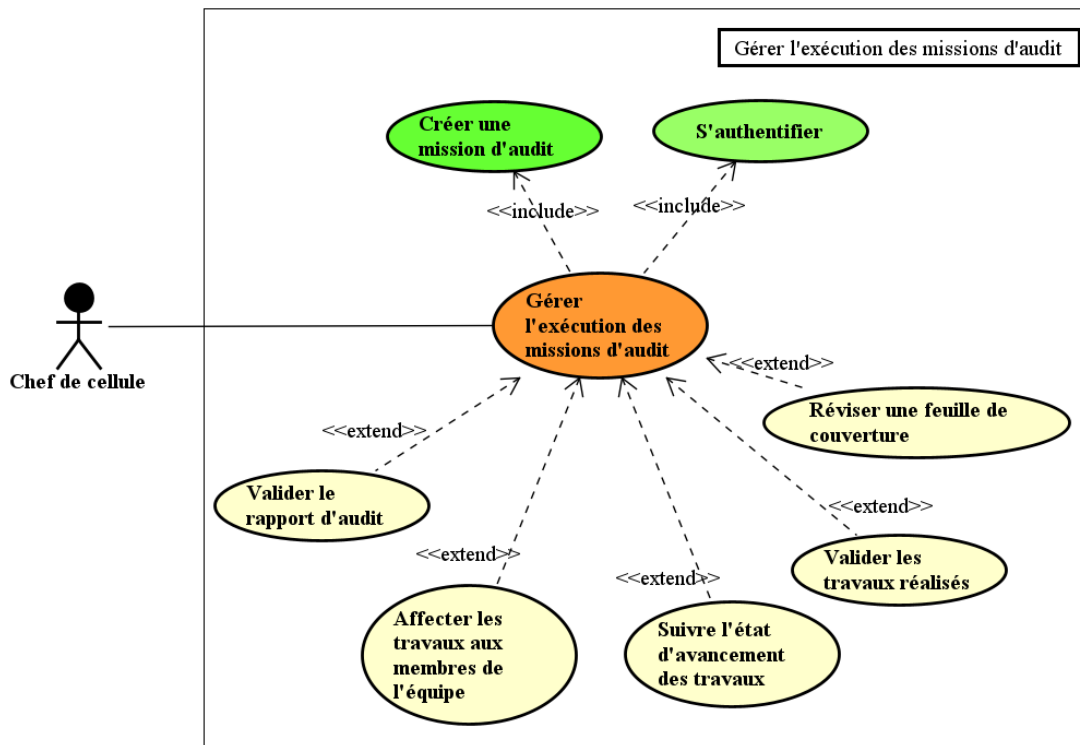


Figure 10 cas d'utilisation : Gérer l'exécution des missions d'audit

3. Descriptions textuelles

Chaque cas d'utilisation, est associé à une série d'actions représentant la fonctionnalité voulue, ainsi que les stratégies à utiliser dans l'alternative où la validation échoue, ou des erreurs se produisent.

Le formalisme d'une description textuelle est le suivant :

Tableau 11 formalisme d'une description textuel pour le diagramme de cas d'utilisation

Le titre	
Résumé	Brève description de l'objectif du cas d'utilisation.
Acteurs	Entités (humaines ou systèmes) qui interagissent avec le système.
Date de création	Date à laquelle le cas d'utilisation a été rédigé.

Responsable	Personne en charge du cas d'utilisation.
Version	Numéro de version de l'application.
Précondition	Conditions à remplir avant le déclenchement du cas d'utilisation.
Scénario nominal	Séquence principale des étapes d'exécution.
Scénarios alternatifs	Variantes ou chemins alternatifs du scénario principal.
Scénario d'exception	Cas où une anomalie ou une erreur survient.
Post condition de succès	Résultat attendu si tout se passe bien.
Post condition d'échec	Conséquence en cas d'échec.
Exigences non fonctionnelles	Contraintes techniques (performance, sécurité, etc.)

d. S'authentifier

Tableau 12 description textuelle du cas : S'authentifier

S'authentifier	
Résumé	Permet à l'utilisateur de se connecter à l'application avec son identifiant et son mot de passe afin d'accéder aux fonctionnalités autorisées selon son profil.
Acteurs	Administrateur, Chef de Division, Chef de Cellule, Auditeur, Direction Générale
Date de création	08 juillet 2026
Responsable	NKONO NDEME Miguel
Version	1.0
Précondition	Disposer d'un compte utilisateur actif créé par l'administrateur.
Scénario nominal	1. L'utilisateur lance l'application. 2. Le système affiche l'écran de connexion. 3. L'utilisateur saisit son identifiant et son mot de passe. 4. L'utilisateur valide le formulaire. 5. Le système vérifie la conformité des champs saisis. 6. Le système envoie une requête de vérification à la base de données. 7. La base de données vérifie les identifiants et le statut du compte. 8. La base de données renvoie le résultat au système. 9. Le système analyse le résultat. Le système affiche l'écran principal correspondant au profil de l'utilisateur.
Scénarios alternatifs	5.a Un champ obligatoire est vide. 5.b Le système affiche un message d'erreur et reste sur l'écran de connexion.
Scénario d'exception	8.a Les identifiants saisis sont incorrects. 8.b Le système affiche un message « identifiant ou mot de passe incorrect » et retourne à l'étape 3.

	8.c Le compte de l'utilisateur est désactivé (statut « Inactif »). 8.d Le système affiche un message indiquant que le compte est désactivé et invite l'utilisateur à contacter l'administrateur.
Post condition de succès	L'utilisateur accède à l'écran principal avec les droits correspondant à son profil.
Post condition d'échec	L'utilisateur reste sur l'écran de connexion.
Exigences non fonctionnelles	Le mot de passe doit être stocké sous forme hachée (BCrypt) ; le temps de réponse de l'authentification ne doit pas excéder 2 secondes.

e. Description textuelle du cas : Gérer l'exécution des mission d'audit

Tableau 13 description textuelle du cas : Gérer l'exécution des missions d'audit

Gérer l'exécution des missions d'audit	
Résumé	Permet au Chef de Cellule de piloter le déroulement des travaux d'une mission : affectation des travaux aux membres de l'équipe, suivi de leur état d'avancement, révision des feuilles de couverture produites, validation des travaux réalisés et validation du rapport d'audit.
Acteurs	Chef de Cellule
Date de création	29 juillet 2026
Responsable	NKONO NDEME Miguel
Version	1.0
Précondition	Être authentifié avec un profil Chef de Cellule. La mission doit avoir été créée, dotée d'une équipe, et son programme de travail doit être validé.
Scénario nominal	1. L'utilisateur sélectionne une mission de sa cellule en cours d'exécution. 2. Le système affiche le programme de travail et la liste des tests qui le composent. 3. L'utilisateur affecte chaque test à un membre de l'équipe et en fixe l'échéance. 4. Le système enregistre les affectations et notifie les auditeurs concernés. 5. L'utilisateur consulte le tableau d'avancement des travaux. 6. Le système affiche pour chaque test son exécutant, son statut et son échéance. 7. L'utilisateur sélectionne une feuille de couverture achevée par un auditeur. 8. Le système affiche la feuille, les travaux effectués, les anomalies relevées et la conclusion.

	9. L'utilisateur révise la feuille et pose son visa. 10. Le système enregistre le visa, l'horodate et fait passer le test au statut « Validée ». 11. Au terme des travaux, l'utilisateur examine le projet de rapport d'audit. 12. L'utilisateur valide le rapport et le transmet au Chef de Division.
Scénarios alternatifs	9.a L'utilisateur estime les travaux insuffisants. 9.b L'utilisateur pose une annotation motivée sur la feuille et la retourne à son auteur. 9.c Le système notifie l'auditeur concerné et ramène le test au statut « En cours ». 12.a L'utilisateur relève des insuffisances dans le projet de rapport. 12.b L'utilisateur le retourne à son rédacteur avec ses observations, sans le transmettre.
Scénario d'exception	3.a L'auditeur pressenti n'appartient pas à l'équipe de la mission. 3.b Le système refuse l'affectation et signale que seuls les membres de l'équipe peuvent se voir confier un test. 9.a L'utilisateur est également l'auteur de la feuille de couverture examinée. 9.b Le système refuse le visa : le rédacteur d'une pièce ne peut en être le réviseur. 1.a La mission sélectionnée ne relève pas de la cellule de l'utilisateur. 1.b Le système refuse l'accès et ne présente que les missions de sa cellule.
Post condition de succès	Les travaux sont affectés, exécutés puis validés ; le rapport d'audit est transmis au Chef de Division pour validation définitive.
Post condition d'échec	Les travaux demeurent dans leur état antérieur ; aucun visa n'est enregistré.

Exigences fonctionnelles	non	Le rédacteur d'une pièce ne peut jamais en être le réviseur ni le valideur. Tout visa est horodaté, rattaché à son auteur et ne peut être ni modifié ni supprimé. Le Chef de Cellule n'accède qu'aux missions relevant de sa cellule.
--------------------------	-----	---

f. Description textuelle du cas : Gérer la documentation de la mission

Tableau 14 description textuelle du cas : Gérer la documentation de la mission

Gérer la documentation de la mission	
Résumé	Permet à l'auditeur de constituer et d'exploiter le dossier documentaire d'une mission : dépôt de pièces classées selon la nomenclature en vigueur, consultation des documents auxquels son profil lui donne accès, téléchargement, et archivage du dossier à la clôture de la mission.
Acteurs	Auditeur
Date de création	29 juillet 2026
Responsable	NKONO NDEME Miguel
Version	1.0
Précondition	Être authentifié avec un profil Auditeur et être affecté à la mission concernée. La mission doit avoir été créée.
Scénario nominal	1. L'utilisateur sélectionne une mission à laquelle il est affecté. 2. Le système affiche le dossier de mission organisé en dossier synthétique, dossier de travail et dossier administratif. 3. L'utilisateur demande le dépôt d'un document. 4. Le système affiche le formulaire de dépôt. 5. L'utilisateur choisit le fichier, sa catégorie et son type, et précise son caractère confidentiel le cas échéant. 6. Le système vérifie le format et la taille du fichier. 7. Le système engendre le code de référence conformément à

	la nomenclature de la Note de service N°106-22/ADC/DG/DI. 8. Le système enregistre le document en version 1 et le marque comme version courante. 9. L'utilisateur consulte la liste des documents auxquels il a accès. 10. L'utilisateur télécharge un document pour exploitation. 11. Le système enregistre l'opération de téléchargement dans le journal applicatif. 12. À la clôture de la mission, le système archive l'ensemble du dossier.
Scénarios alternatifs	5.a Le document déposé porte le même code de référence qu'un document existant. 5.b Le système enregistre le nouveau fichier comme version suivante, le rattache au document racine et transfère le marquage de version courante, sans écraser la version antérieure. 9.a Le dossier comporte des documents marqués confidentiels dont l'utilisateur n'est pas destinataire. 9.b Le système ne présente que les documents autorisés pour son profil et son affectation.
Scénario d'exception	6.a Le format du fichier n'est pas admis ou sa taille excède la limite autorisée. 6.b Le système refuse le dépôt et affiche un message précisant la cause du rejet. 3.a La mission est clôturée et son dossier archivé. 3.b Le système refuse toute opération de dépôt, de modification ou de suppression sur un dossier archivé. 1.a L'utilisateur n'est pas affecté à la mission sélectionnée. 1.b Le système refuse l'accès au dossier.
Post condition de succès	Le document est enregistré dans le dossier de mission, référencé selon la nomenclature et accessible aux membres autorisés de l'équipe.

Post condition d'échec	Aucun document n'est déposé ; le dossier de mission demeure inchangé.
Exigences non fonctionnelles	Le code de référence est engendré automatiquement selon la nomenclature en vigueur. Aucune version antérieure n'est écrasée. Un document archivé ne peut plus être modifié ni supprimé. Tout dépôt, toute consultation et tout téléchargement sont horodatés et rattachés à leur auteur.

g. Description textuelle du cas : Gérer les missions d'audit

Tableau 15 description textuelle du cas : Gérer les missions d'audit

Gérer les missions d'audit	
Résumé	Permet au Chef de Division et au Chef de Cellule de piloter le cycle de vie d'une mission d'audit : création de la mission avec rattachement des risques significatifs qu'elle doit couvrir, composition de l'équipe chargée de la conduire, désignation du chef de mission, suivi de l'avancement et clôture.
Acteurs	Chef de Division, Chef de Cellule
Date de création	29 juillet 2026
Responsable	NKONO NDEME Miguel
Version	1.0
Précondition	Être authentifié avec un profil Chef de Division ou Chef de Cellule. Un exercice d'audit doit être ouvert.
Scénario nominal	1. L'utilisateur accède au module de gestion des missions. 2. Le système affiche la liste des missions de l'exercice en cours. 3. L'utilisateur demande la création d'une nouvelle mission. 4. Le système affiche le formulaire de création. 5. L'utilisateur saisit l'intitulé, la nature, l'entité auditée, les objectifs, le périmètre et les dates prévisionnelles. 6. L'utilisateur sélectionne les risques significatifs que la mission doit couvrir. 7. Le système vérifie qu'au moins un risque significatif a été retenu. 8. Le système engendre le code de la mission et l'enregistre au statut « Planifiée ». 9. L'utilisateur compose l'équipe en désignant les auditeurs affectés et leur rôle respectif. 10. Le système enregistre la composition de l'équipe et notifie les auditeurs affectés.

	11. L'utilisateur consulte à tout moment l'état d'avancement de la mission. 12. Le système affiche le taux de réalisation des travaux et l'état des pièces du dossier. 13. Au terme des travaux, l'utilisateur demande la clôture de la mission. 14. Le système vérifie la complétude du dossier de mission. 15. Le système enregistre la mission au statut « Clôturée » et la rend accessible en lecture seule.
Scénarios alternatifs	3.a L'utilisateur choisit d'inscrire la mission au Plan Annuel d'Audit. 3.b Le système vérifie qu'un plan validé existe pour l'exercice en cours et rattache la mission à ce plan. 3.c Lorsque la mission est créée hors plan, aucun rattachement n'est effectué et la mission est signalée comme mission hors plan. 9.a L'utilisateur est un Chef de Cellule. 9.b Le système restreint la liste des auditeurs proposés aux membres de sa cellule. 10.a L'utilisateur désigne parmi les membres de l'équipe celui qui tiendra le rôle de chef de mission. 10.b Le système enregistre la désignation et notifie l'auditeur concerné. 10.c Lorsqu'aucun chef de mission n'est désigné à ce stade, la mission demeure enregistrée et la désignation pourra intervenir ultérieurement.
Scénario d'exception	7.a Aucun risque significatif n'a été retenu. 7.b Le système affiche un message d'erreur et maintient le formulaire à l'écran sans enregistrer la mission. 10.a Un chef de mission est déjà désigné pour cette mission. 10.b Le système refuse la seconde désignation et invite l'utilisateur à retirer la première. 14.a Le dossier de mission est incomplet : une pièce

	obligatoire est absente ou n'a pas été validée. 14.b Le système affiche la liste des éléments manquants et refuse la clôture. 13.a L'utilisateur est un Chef de Cellule. 13.b Le système refuse la demande : seul le Chef de Division peut prononcer la clôture d'une mission.
Post condition de succès	La mission est enregistrée, couvre au moins un risque significatif, est dotée d'une équipe, et progresse dans son cycle de vie jusqu'à la clôture.
Post condition d'échec	Aucune mission n'est créée, ou la mission demeure dans son statut antérieur.
Exigences non fonctionnelles	Le code de mission est engendré automatiquement et demeure unique. Une mission comporte au plus un chef de mission. Toute création, modification de composition d'équipe et clôture est horodatée et rattachée à son auteur dans le journal applicatif. Une mission clôturée ne peut plus être modifiée.

2. Diagramme de communication

Un diagramme de communication dans le langage de modélisation unifié fait référence à un graphique qui représente le flux de messages dans un système. En un mot, il montre comment les parties d'un système interagissent ou, dans ce cas, communiquent entre elles.

Comme tout autre diagramme, le diagramme de communication UML comporte également plusieurs composants qui constituent son intégralité. La bonne chose à propos de ce schéma est qu'il peut être réalisé avec seulement quelques composants.

Le formalisme est le suivant :

Tableau 16 formalisme du diagramme de communication

Composant	Description	Représentation
Acteur	C'est une entité qui interagit avec le système ou qui est extérieur à lui. Sa représentation est identique à celui d'un diagramme de cas d'utilisation.	 Acteur
Objet	Il est représenté par un rectangle contenant le nom et le type de l'instance.	 Objet
Les connecteurs	Les relations entre les lignes de vie sont appelées connecteurs et se représentent par un trait plein reliant deux lignes de vie.	
Les messages	Désignent une communication particulière entre les lignes de vie, et sont généralement ordonnés selon un ordre de numéro croissant.	

1. Diagramme de communication du cas

S'authentifier

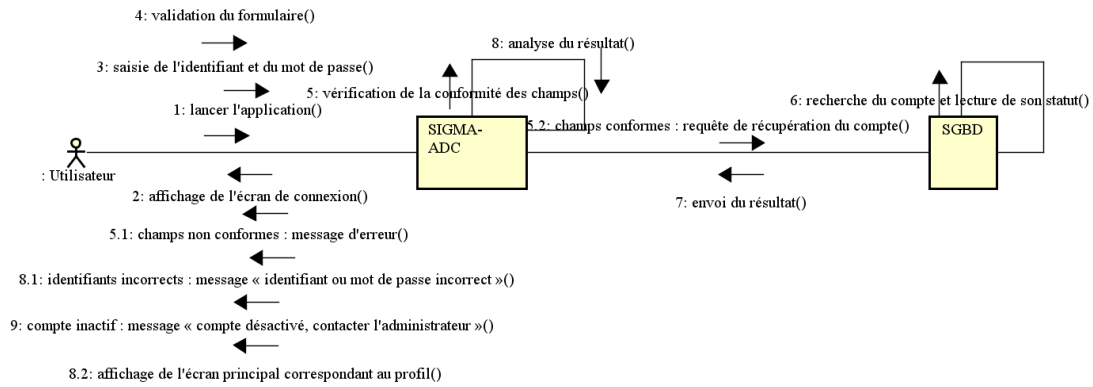


Figure 11 diagramme de communication du cas s'authentifier

2. Diagramme de communication du cas : Gérer les missions d'audit

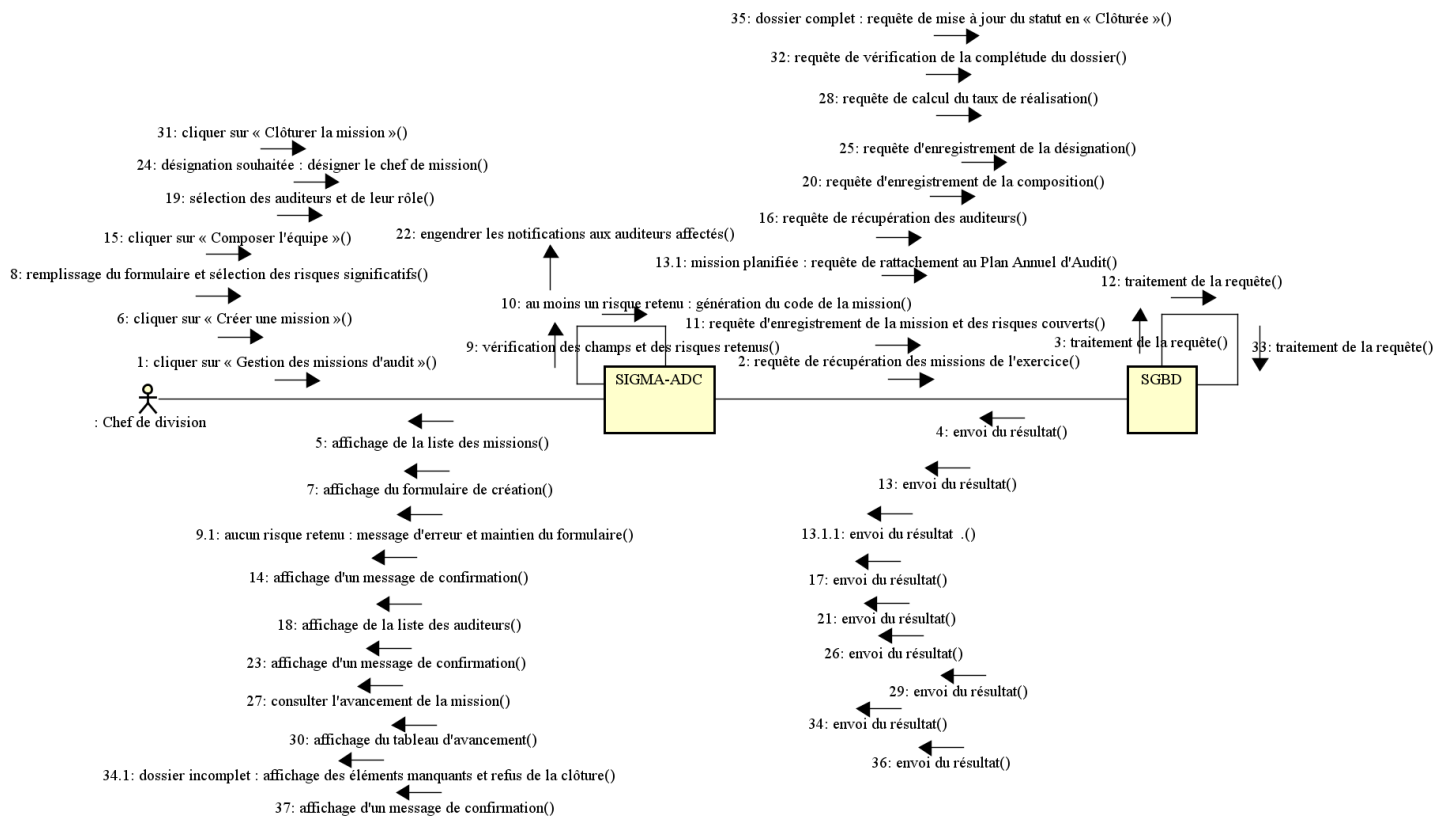


Figure 12 diagramme de communication du cas : Gérer les missions d'audit

3. Diagramme de communication du cas : Gérer l'exécution des missions d'audit

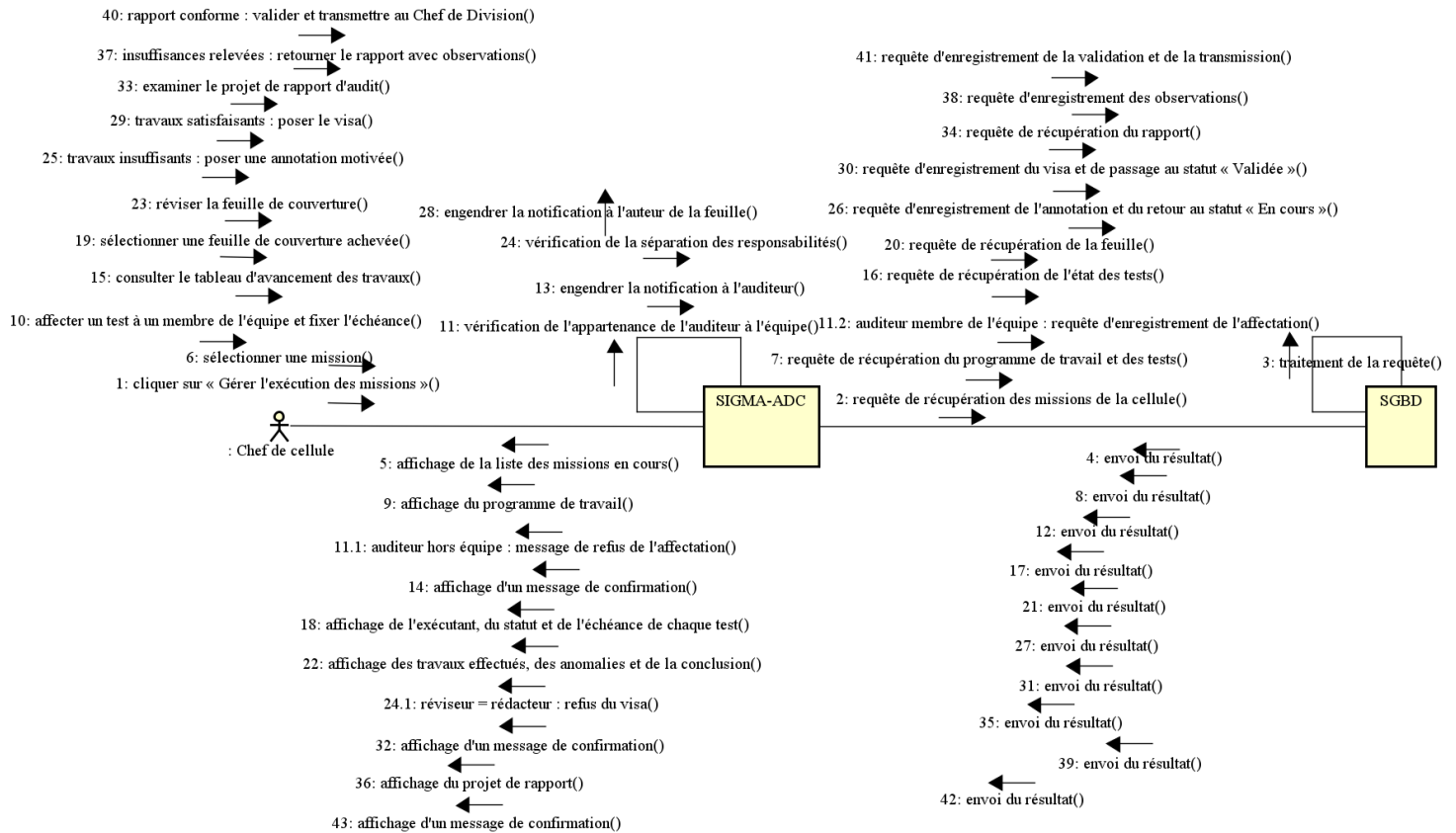


Figure 13 diagramme de communication du cas : gérer l'exécution des missions d'audit

4. Diagramme de communication du cas : Gérer la documentation de la mission

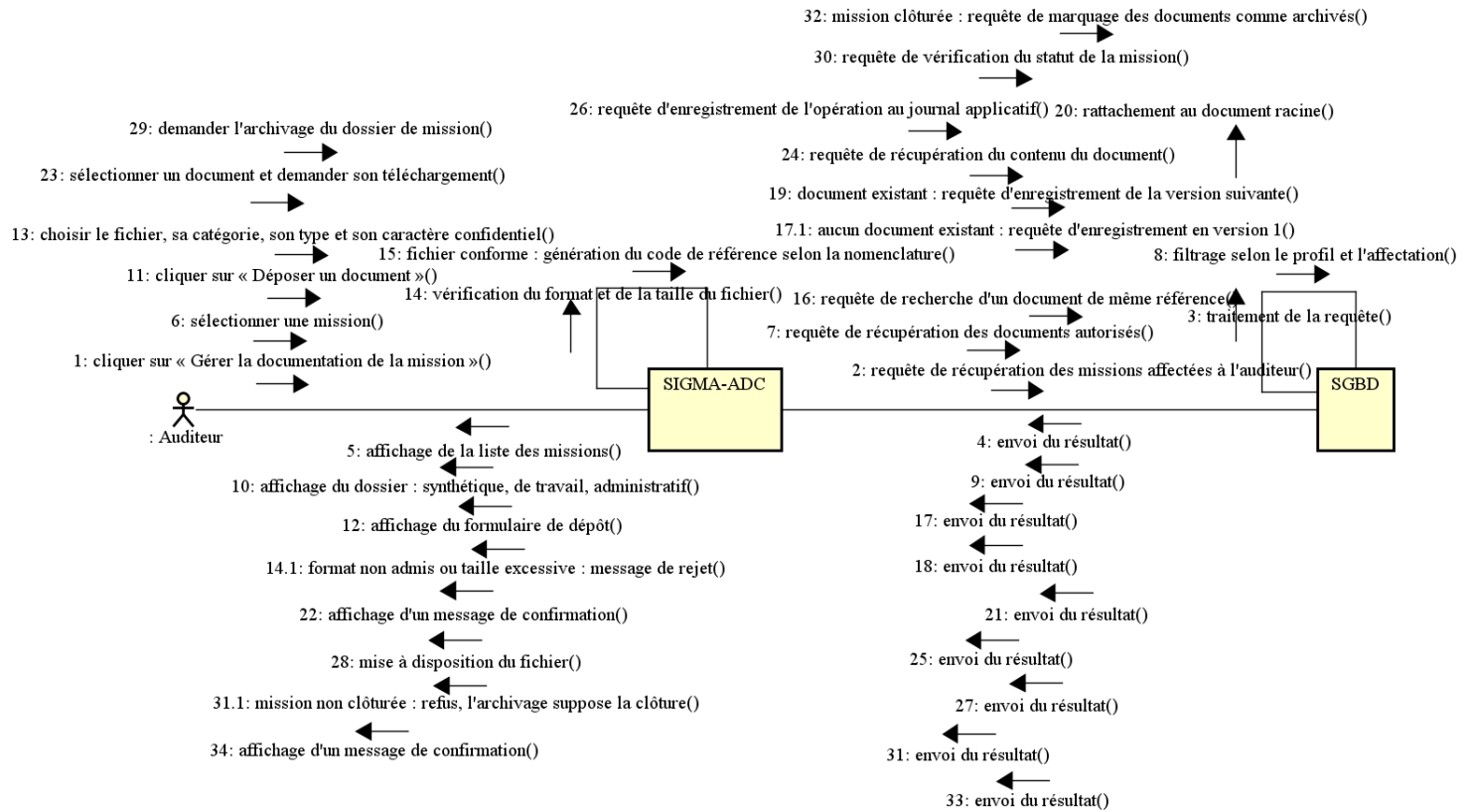


Figure 14 diagramme de communication du cas : gérer la documentation de la mission.

3. Diagramme de séquence

Les diagrammes de séquences sont des représentations graphiques des interactions entre les acteurs et le système selon un ordre chronologique dans la formulation UML. Le diagramme de séquences permet de décrire les interactions entre les objets au sein d'un diagramme des cas d'utilisation.

Le diagramme de séquence énumère les objets horizontalement et le temps verticalement. Il modélise l'exécution des différents modèles en fonction du temps. Dans ce diagramme, les objets et les acteurs sont énumérés en colonnes avec leur ligne de vie verticale indiquant la durée de vie de l'objet.

Le formalisme est le suivant :

Tableau 17 formalisme du diagramme de séquence

Composant	Description	Représentation
Objet	Se décrit comme étant l'instance d'une classe	
Acteur	Il s'agit d'une personne qui interagit et communique avec le système.	
Ligne de vie	La ligne de vie, représente le déroulement temporel d'un processus.	
Activation	Il s'agit ici du temps nécessaire pour qu'un objet ou un acteur accomplisse une tâche, elle indique quand l'objet effectue une action.	
Message synchrone	On utilise ce symbole lorsqu'un expéditeur doit attendre une réponse à un message avant de continuer. Le diagramme doit montrer à la fois l'appel et la réponse	

Messages de retour asynchrones	Les messages asynchrones ne nécessitent pas de réponse avant que l'expéditeur ne continue. Seul l'appel doit être inclus dans le diagramme.	
Message de retour	ces messages sont des réponses aux appels.	

Exemple de diagramme de séquence :

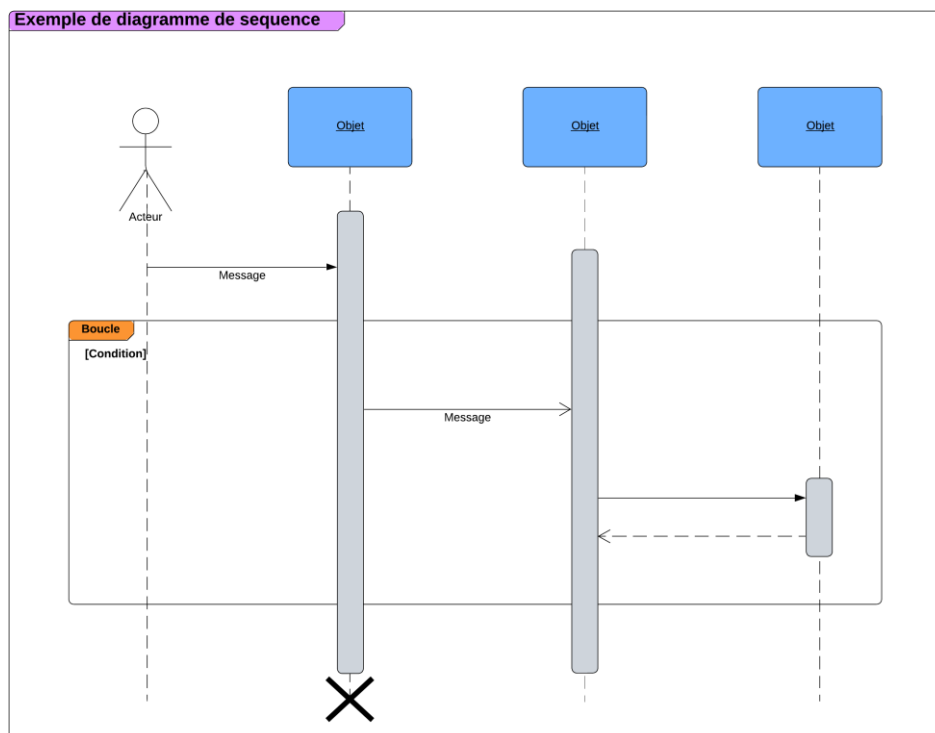


Figure 15 exemple de diagramme de séquence.

1. Diagramme de séquence : S'authentifier

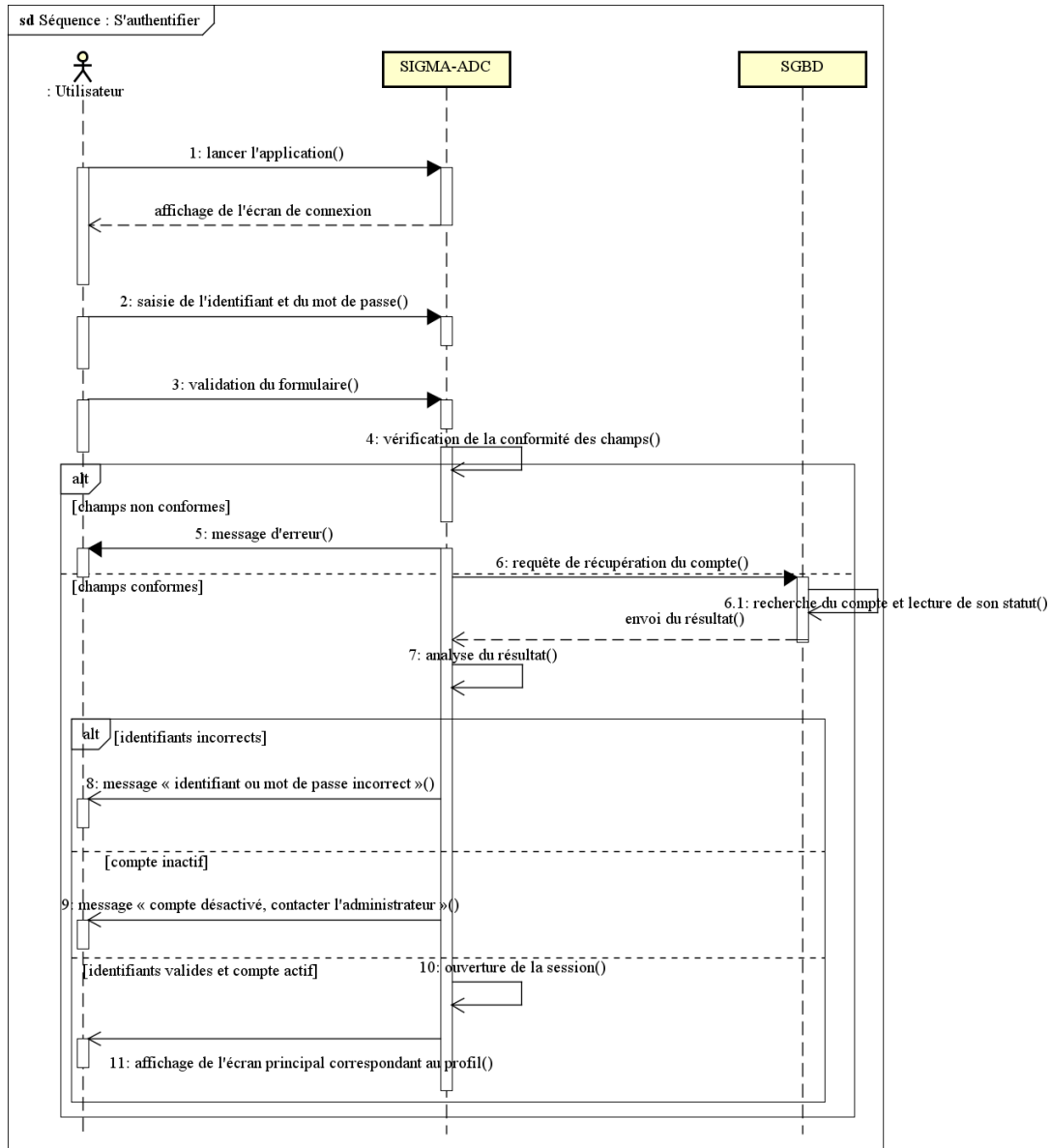


Figure 16 diagramme de séquence : s'authentifier

2. Diagramme de séquence : Gérer les missions d'audit

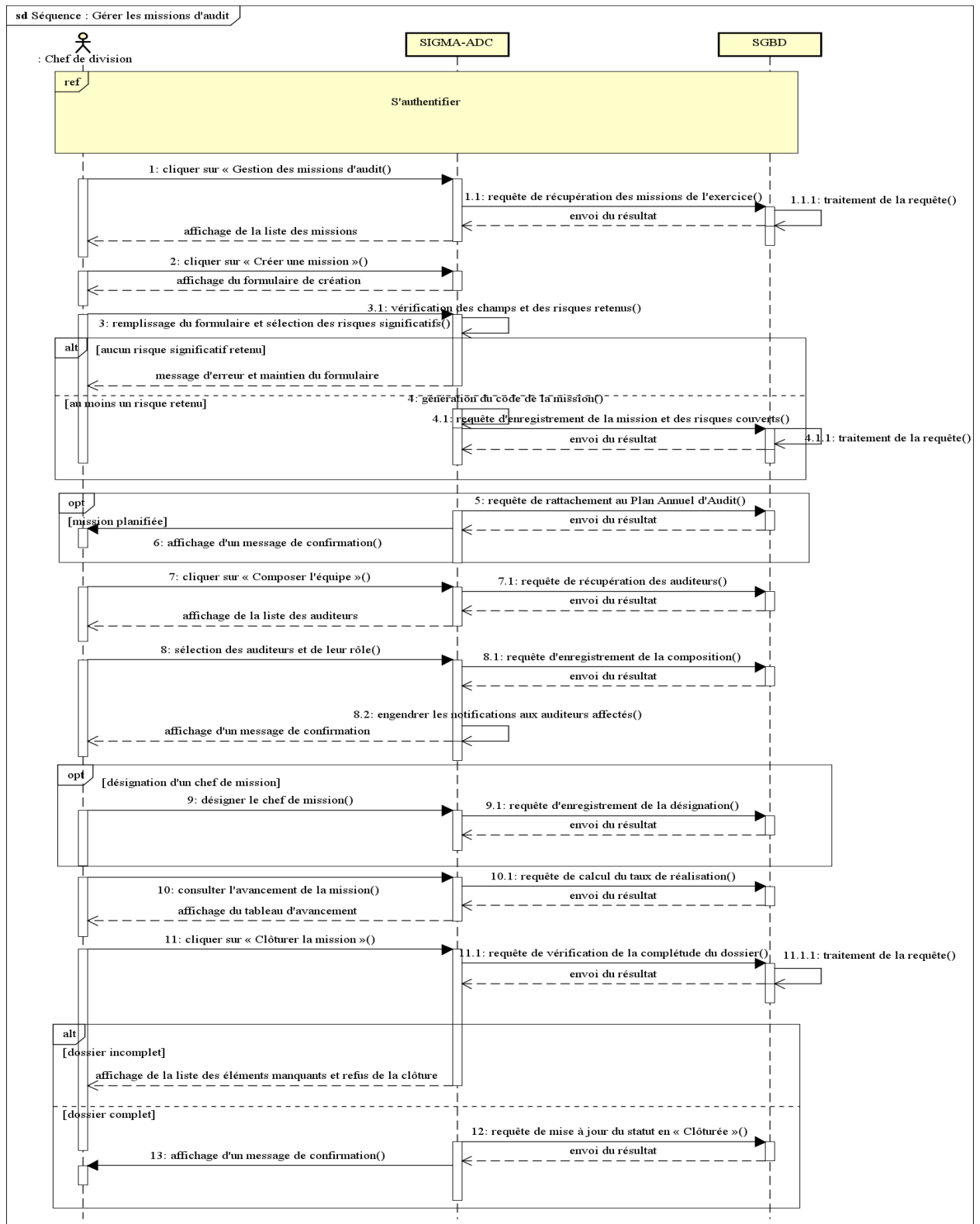


Figure 17 diagramme de séquence : Gérer les missions d'audit

3. Diagramme de séquence : Gérer l'exécution des missions d'audit

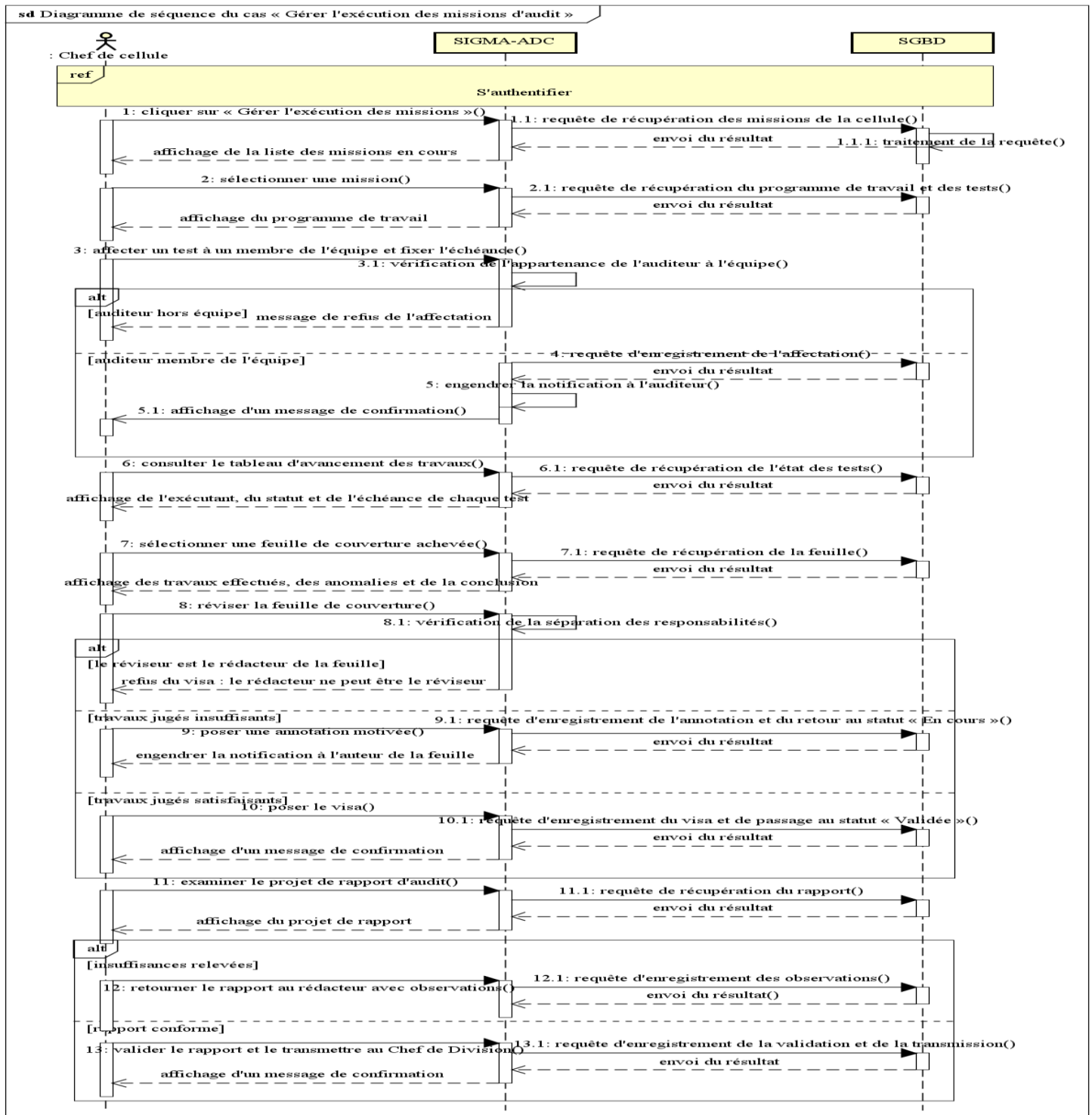


Figure 18 diagramme de séquence : gérer l'exécution des missions

4. Diagramme de séquence : Gérer la documentation de la mission

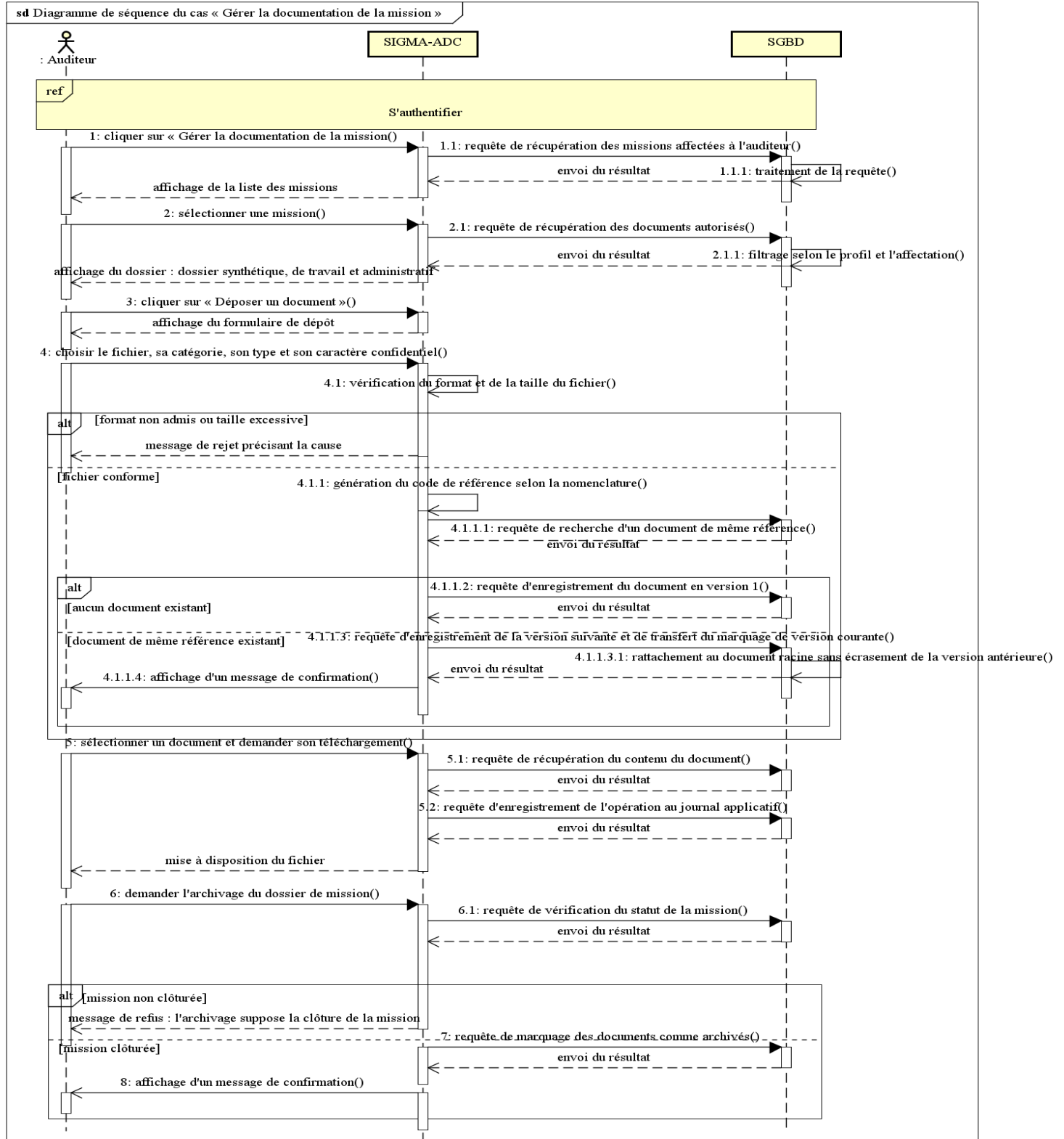


Figure 19 diagramme de séquence : Gérer la documentation de la mission.

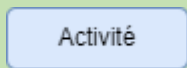
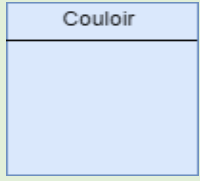


Conception et développement d'une application de gestion des missions d'audit interne : SIGMA-ADC



4. Diagramme d'activité

Tableau 18 tableau de formalisme d'activité

Composant	Description	Représentation
Une action	Rectangle aux coins arrondis qui représente une opération élémentaire et instantanée	
Une transition	Ligne qui représente le passage d'une activité à une autre	
Nœud initial	Cercle plein à partir duquel un flot débute dans un diagramme d'activité	
Nœud final	Un nœud de fin d'activité : un petit cercle plein marquant l'arrêt l'exécution de l'activité enveloppante s'arrête	
	Un nœud de fin de flot : cercle vide barré d'une croix marquant l'arrêt d'un flot à l'intérieur du diagramme	
Couloir	Les diagrammes d'activités font intervenir les acteurs de chaque activité. Chaque activité sera placée dans une colonne (couloir) qui correspond à l'acteur qui l'effectue.	
Barre de synchronisation	Barre horizontale sur laquelle convergent les activités qui sont exécutées en parallèle.	

Les diagrammes d'activités sont particulièrement adaptés à la description des cas d'utilisation. Plus précisément, ils viennent illustrer et consolider la description textuelle des cas d'utilisation.

L'usage général du diagramme d'activité permet de mettre l'accent sur les traitements et de formaliser graphiquement la séquence d'actions réalisées dans un cas d'utilisation.

Exemple de diagramme d'activité :

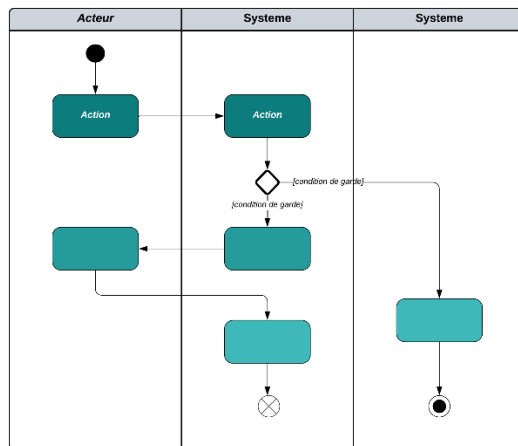


Figure 20 formalisme d'un diagramme d'activité

1. Cas S'authentifier

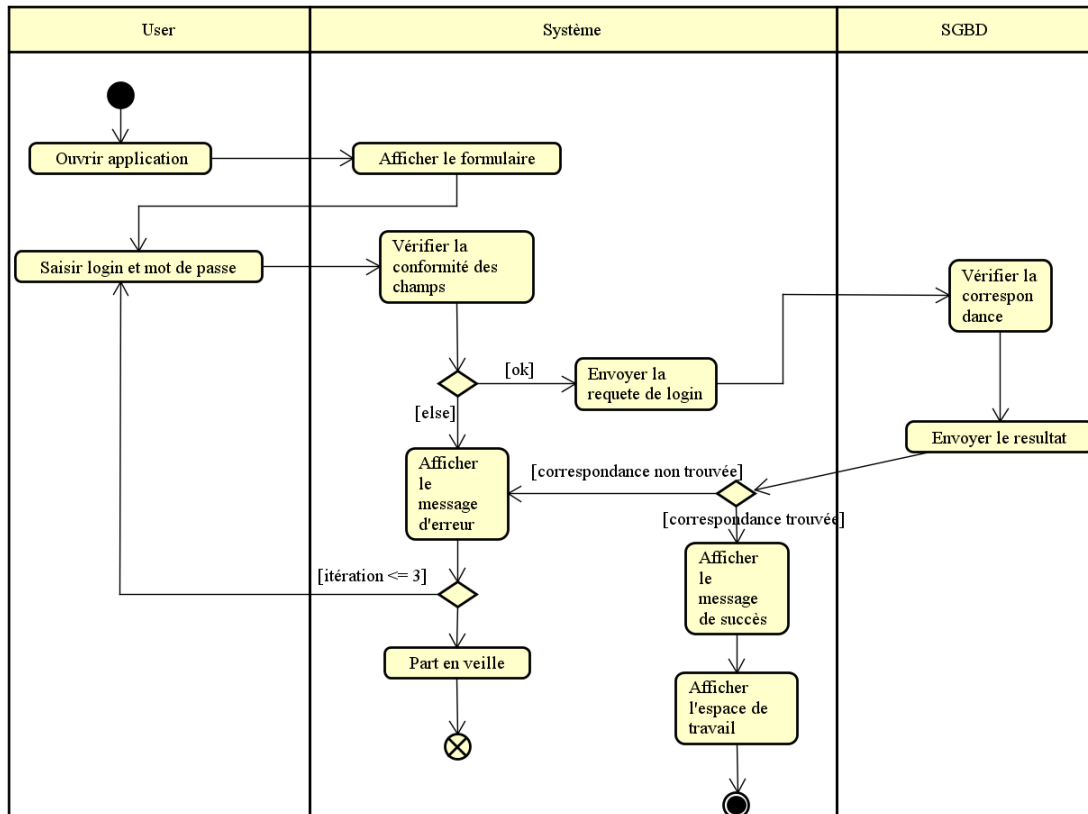


Figure 21 diagramme d'activité du cas s'authentifier

2. Diagramme d'activité du cas : Gérer les missions d'audit

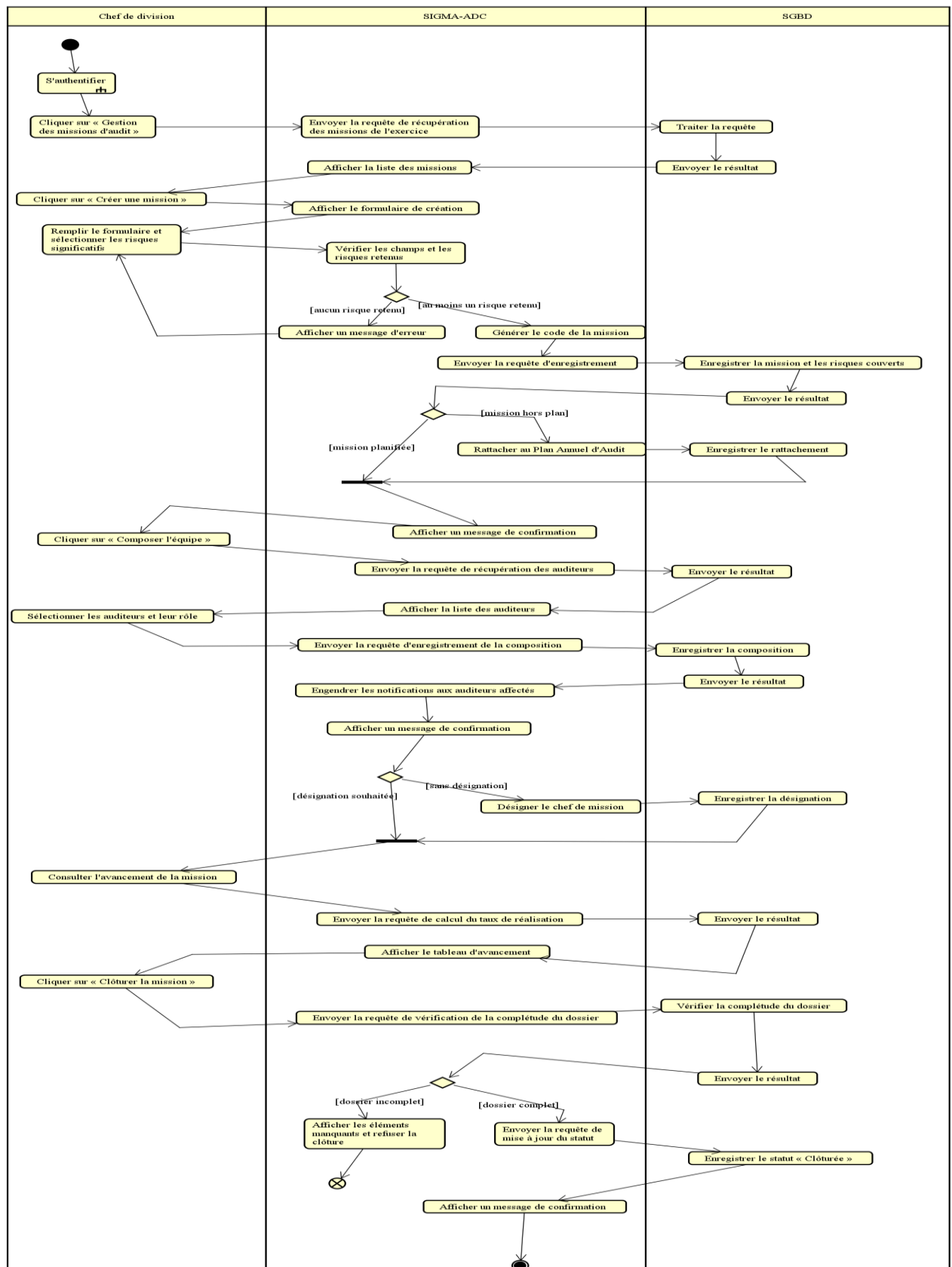


Figure 22 diagramme d'activité du cas : Gérer les missions d'audit

3. Diagramme d'activité du cas : Gérer l'exécution des missions d'audit

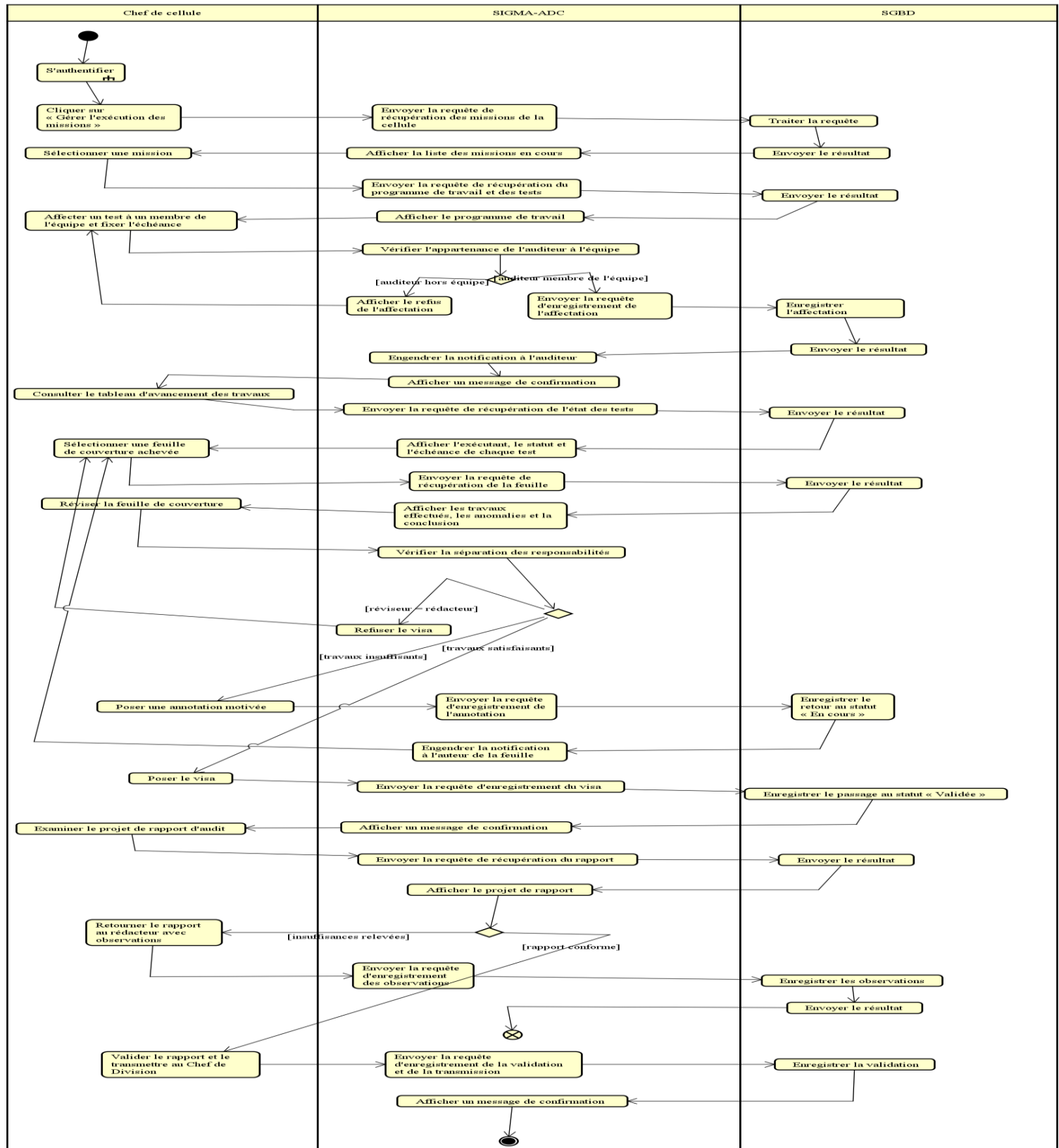


Figure 23 diagramme d'activité du cas : Gérer l'exécution des missions d'audit

4. Diagramme d'activité du cas : Gérer la

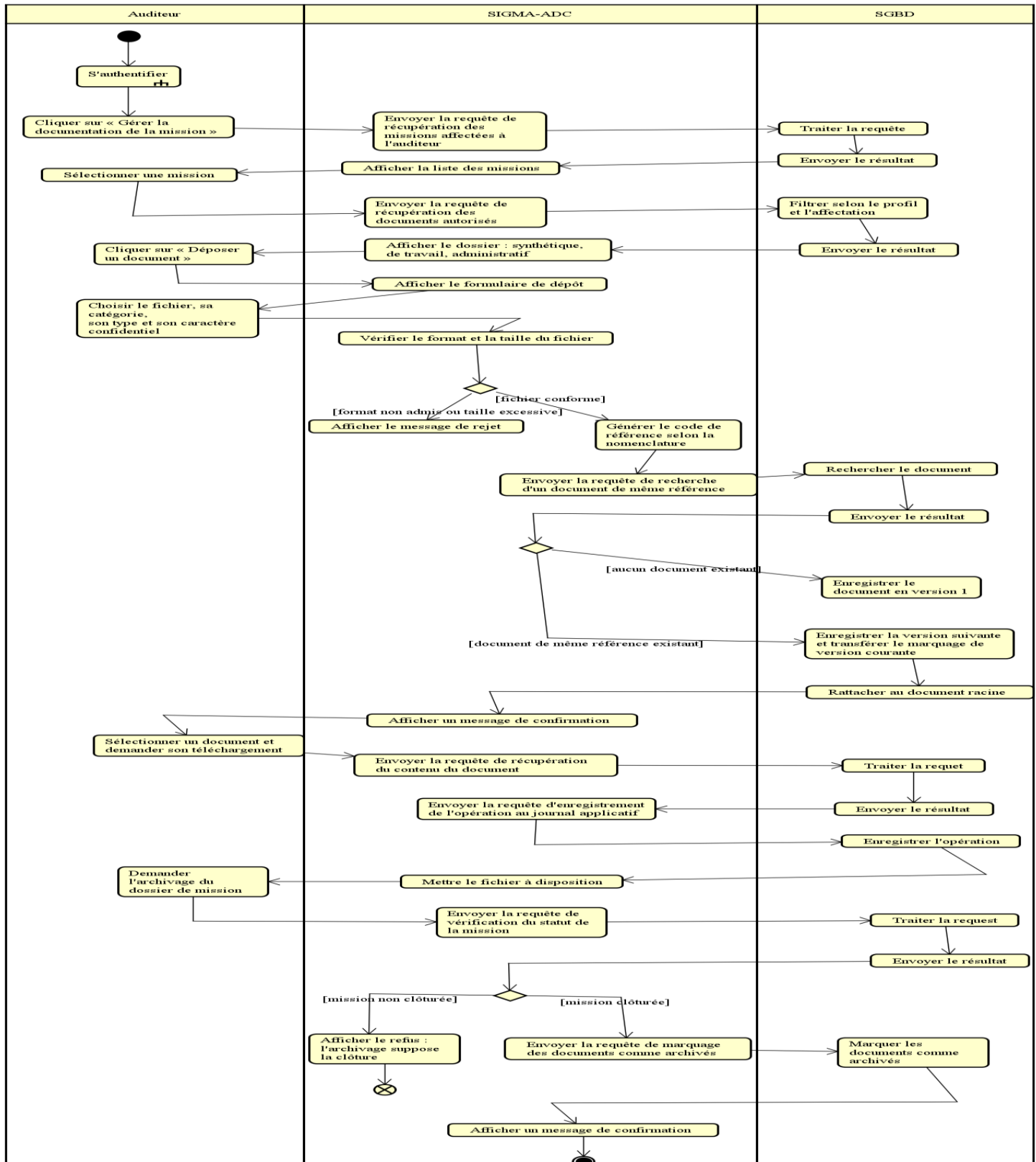


Figure 24 diagramme d'activité du cas : Gérer la documentation de la mission

documentation de la mission

Conclusion

Dans ce dossier, on a présenté une étude du système existant, les lacunes qu'il comprend ainsi que les solutions que nous proposons pour pallier ces problèmes. Ce dossier d'analyse nous a permis d'avoir un aperçu détaillé du système à mettre sur pied. Toujours en exploitant le langage UML et le processus associé 2TUP, nous poursuivrons avec le dossier de conception dans lequel nous représenterons les diagrammes de la branche technique.

DOSSIER 4 : DOSSIER DE CONCEPTION

Résumé :

Le dossier de conception permet de modéliser dans son ensemble la solution proposée et de recueillir les informations nécessaires à la mise sur pied d'une base de données complexes et efficaces. De ce fait il prévoit le système futur.

Aperçu :

Introduction

1. Diagramme de classe
2. Diagramme d'état transition
3. Diagramme de paquetage

Conclusion

Introduction

Une bonne méthodologie de la réalisation d'un logiciel suppose une bonne maîtrise d'analyse et conception. Cette dernière nous offre tous les modèles destinés à assurer le fonctionnement du logiciel. Dans le dossier de conception, il est question de ressortir à travers les diagrammes UML, l'architecture statique de notre futur logiciel. De manière globale, le dossier de conception offre une vue panoramique sur l'ensemble des éléments et les interactions prise en compte dans le dossier d'analyse.

I. DIAGRAMME DE CLASSE

Le diagramme de classe met en évidence d'éventuelles relations entre ces classes. Le diagramme de classes comporte 6 concepts : **classe**, **attribut**, **identifiant**, **relation**, **opération**, **généralisation/spécialisation**. Ce diagramme est constitué des éléments suivants :

- **Classe** : il s'agit de la description abstraite à un ensemble d'objet : elle définit leurs structures, leurs comportements et leurs relations.
- **L'attribut** : Il représente la modélisation d'une information élémentaire représentée par son nom et format.

UML définit 3(trois) niveaux de visibilité pour les attributs :

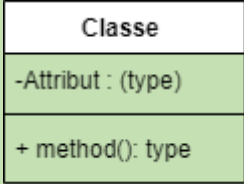
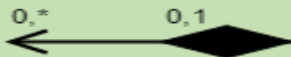
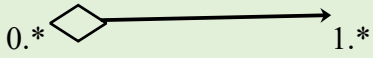
- **Public (+)** : l'élément est visible par tous les clients de la classe.
- **Protégé (#)** : l'élément est visible que par les sous-classes ou les classes filles.
- **Privé (-)** : l'élément n'est visible que par les objets de la classe auxquels il appartient.
- **L'identifiant** : c'est un attribut particulier, qui permet de repérer de façon unique chaque objet, instance de la classe.
- **Multiplicité** : elle définit le nombre d'instance de la classe, elle est définie par un nombre entier ou un intervalle de valeur (1...1, 0...1, 1...*, 0...*).
- **Associations** : Une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions structurelles entre leurs instances.

Il existe plusieurs types de relations dont les plus connues sont les suivants :

- **Généralisation** : Relation d'héritage, dans laquelle les objets de l'élément spécialisé (Classe enfant) peuvent remplacer les objets de l'élément général (Classe parent)
- **Agrégation** : utilisé pour les relations de types « contenant / contenue »
- **La composition** : La composition est un cas particulier de l'agrégation dans laquelle la vie des composants est liée à celle des agrégats.

Le tableau suivant nous donne une liste assez détaillée du formalisme du diagramme de classe en un tableau :

Tableau 19 formalisme du diagramme de classe

Elément	Représentation
Classe	
Association	
Composition	
Agrégation	
Généralisation	
Multiplicité	0..*, 0...1, 1...1, 1...*
Modificateur d'accès	+, -, #, ~, /

Exemple de diagramme de classe :

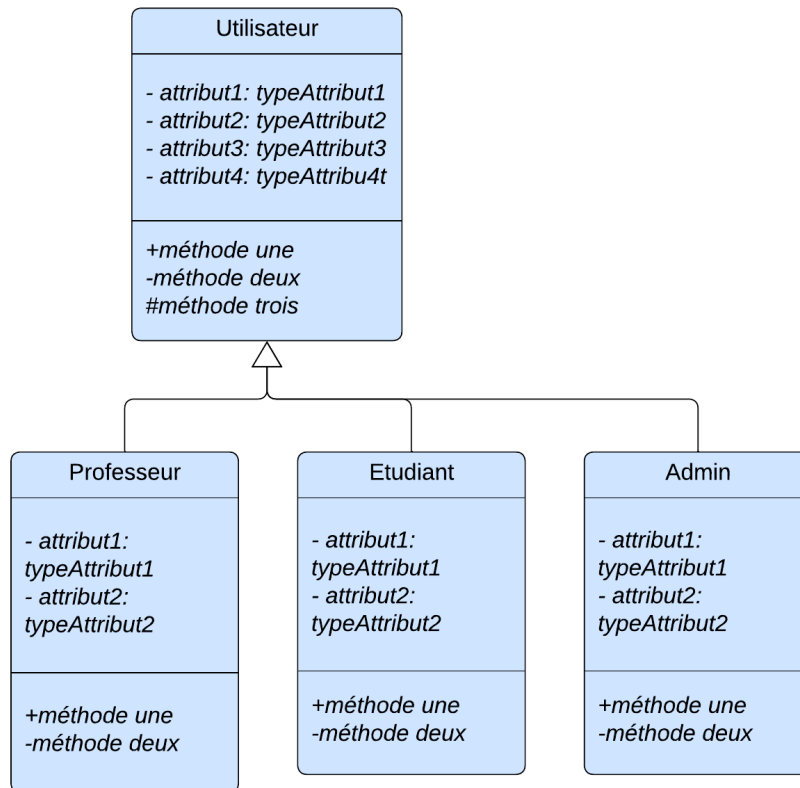


Figure 25 exemple du diagramme de classe.

Le diagramme de classe de notre projet est le suivant :

1. Règles de métiers de notre diagramme de classe :

Dans le cadre de programmation Orienté Objet, chaque classe doit être responsable de tâches précises, laissant à d'autres classes le soin de prendre en charge d'autres tâches. Les relations entre classes qui ressortent de notre analyse sont les suivantes :

Tableau 20 description textuelle.

Relation	Description
R1	Tout utilisateur du système possède un et un seul profil parmi les six suivants : Administrateur, Chef de Division, Chef de Cellule, Auditeur Sénior, Auditeur, Direction Générale.
R2	Un Auditeur Sénior est un Auditeur : il en possède toutes les prérogatives et y ajoute celles de rédaction et de révision. De même, un Chef de Cellule est un Auditeur Sénior et en possède toutes les prérogatives.
R3	Le Chef de Division n'hérite d'aucun profil d'exécution. Il ne peut exécuter un travail qu'il aurait ensuite à valider, conformément au principe de séparation des responsabilités.
R4	L'Administrateur ne dispose d'aucun accès au contenu métier des missions. La Direction Générale ne dispose que d'un accès en lecture aux tableaux de bord et aux rapports définitifs.
R5	Aucun accès au système n'est possible sans authentification préalable par identifiant et mot de passe.
R6	Un compte utilisateur est soit actif, soit inactif. Un compte inactif ne peut ouvrir de session.
R7	Une cellule regroupe zéro ou plusieurs auditeurs ; un auditeur appartient à au plus une cellule.
R8	Une cellule est dirigée par un et un seul chef de cellule ; un chef de cellule dirige au plus une cellule.
R9	Une cellule conduit zéro ou plusieurs missions ; une mission est conduite par au plus une cellule.

R10	Un exercice d'audit couvre au plus un plan annuel d'audit ; un plan annuel d'audit se rapporte à un et un seul exercice et ne peut exister indépendamment de lui.
R11	Un exercice rattache zéro ou plusieurs missions ; toute mission se rattache à un et un seul exercice.
R12	Un plan annuel inscrit zéro ou plusieurs missions ; une mission est inscrite à au plus un plan. Une mission hors plan n'est rattachée à aucun plan annuel.
R13	Un plan annuel est arrêté par un et un seul Chef de Division ; un Chef de Division arrête zéro ou plusieurs plans.
R14	Un exercice ne peut comporter qu'un seul plan annuel à l'état validé.
R15	Une mission est ouverte par un et un seul Chef de Division.
R16	Une mission mobilise au moins un auditeur ; un auditeur participe à zéro ou plusieurs missions. Chaque participation précise un rôle et une date d'affectation.
R17	Le rôle tenu dans une mission est l'un des suivants : chef de mission, auditeur sénior, auditeur, observateur. Une mission comporte un et un seul chef de mission.
R18	Seuls le Chef de Division et le Chef de Cellule peuvent composer l'équipe d'une mission. Le Chef de Cellule ne dispose de cette prérogative que pour les missions relevant de sa propre cellule.
R19	Une mission suit un cycle de vie ordonné : planifiée, en préparation, en cours, en rédaction, validée, clôturée. Aucun retour en arrière n'est admis une fois la mission clôturée.
R20	Seul le Chef de Division peut prononcer la clôture d'une mission.
R21	Un risque évalué est coté par une probabilité et un impact. Sa criticité est le produit des deux et n'est jamais saisie directement.
R22	Un risque évalué est soit un risque significatif du référentiel permanent, soit un risque recensé au registre d'une mission.
R23	Une mission couvre au moins un risque significatif ; un risque significatif est couvert par zéro ou plusieurs missions.
R24	Une mission recense zéro ou plusieurs risques dans son registre ; ces risques n'existent pas indépendamment de la mission qui les a recensés.

R25	Une feuille de couverture couvre zéro ou plusieurs risques significatifs ; un risque significatif est couvert par zéro ou plusieurs feuilles.
R26	Seul le Chef de Division peut mettre à jour la cartographie des risques significatifs.
R27	Toute pièce appartient à une et une seule mission et disparaît avec elle.
R28	Une pièce de mission est d'une et une seule nature parmi les six suivantes : document d'orientation, programme de travail, feuille de couverture, FRAP, rapport, document du dossier.
R29	Une mission comporte au plus un document d'orientation et au plus un programme de travail.
30	Toute pièce est rédigée par un et un seul auditeur, ou par le Chef de Division lorsqu'il s'agit d'un document du dossier.
31	Une pièce est révisée par au plus un auditeur sénior, et validée par au plus un chef de division.
32	Le rédacteur d'une pièce ne peut en être le réviseur, ni le valideur.
33	Une pièce ne peut être validée que si elle a préalablement été révisée.
34	Toute pièce porte un numéro de version. Toute reprise après refus engendre une nouvelle version sans écrasement de la précédente.
35	Le document d'orientation ne prend effet qu'après signature par le Chef de Division.
36	Un document du dossier se rattache à au plus un document racine dont il constitue une version. Une seule version est marquée comme courante à un instant donné.
37	Un document du dossier relève d'une et une seule catégorie : dossier synthétique, dossier de travail ou dossier administratif, et porte le code de référence correspondant.
38	Un document archivé ne peut plus être modifié ni supprimé.
39	Un programme de travail contient zéro ou plusieurs tâches ; une tâche appartient à un et un seul programme et disparaît avec lui.
40	Une tâche se rattache à une ligne du document d'orientation par une référence stable. Elle conserve une copie de l'objectif d'audit, du risque couvert et du contrôle à tester tels qu'ils ont été arrêtés, de sorte qu'une

	modification ultérieure de l'orientation n'altère pas les travaux déjà exécutés.
41	Une tâche est exécutée par au plus un auditeur et révisée par au plus un auditeur sénior. L'exécutant d'une tâche ne peut en être le réviseur.
42	Une tâche est justifiée par au plus une feuille de couverture ; une feuille de couverture justifie une et une seule tâche.
43	Une feuille de couverture est soit une feuille de test, soit une feuille d'entretien.
44	Une feuille de couverture fonde zéro ou plusieurs FRAP ; une FRAP se fonde sur au plus une feuille de couverture.
45	Une FRAP ne peut être visée qu'après recueil de la réponse de l'audité.
46	Une FRAP engendre zéro ou plusieurs recommandations ; une recommandation provient d'au plus une FRAP.
47	Un rapport formule zéro ou plusieurs recommandations ; une recommandation appartient à un et un seul rapport et disparaît avec lui.
48	Une recommandation ne peut être enregistrée qu'à partir d'un rapport définitif validé.
49	Une recommandation désigne un responsable métier chargé de sa mise en œuvre et un référent audit chargé de son suivi, ce dernier étant nécessairement un auditeur.
50	Une recommandation non acceptée doit obligatoirement porter un motif de non-acceptation.
51	Une recommandation fait l'objet de zéro ou plusieurs campagnes de suivi ; un suivi porte sur une et une seule recommandation et disparaît avec elle.
52	Toute campagne de suivi se déroule dans le cadre d'une mission de nature Suivi, distincte de la mission qui a formulé la recommandation.
53	Le taux d'avancement d'une recommandation est compris entre 0 et 100 %. Une recommandation déclarée mise en œuvre présente un taux de 100 % et doit être validée par le référent audit.
54	Un acte de supervision se rattache à une et une seule mission et disparaît avec elle.

55	Un acte de supervision désigne la pièce sur laquelle il porte par son type et son identifiant. Il peut ainsi viser indifféremment un document, une feuille de couverture, une FRAP, une tâche ou un rapport.
56	Un acte de supervision est soit une annotation, soit une validation.
57	Un acte de supervision est posé par un auditeur sénior, un chef de cellule ou un chef de division.
58	Une validation porte l'un des trois niveaux suivants : sénior, cellule, division, et l'une des trois décisions suivantes : visé, refusé, à reprendre.
59	Toute annotation engendre au moins une notification à destination des membres de l'équipe concernés.
60	Une notification est destinée à un et un seul auditeur ; un auditeur reçoit zéro ou plusieurs notifications.
61	Toute opération de création, de modification ou de validation est horodatée et rattachée à son auteur, sans possibilité de suppression ni d'altération de la trace.

II. DIAGRAMME D'ÉTAT TRANSITION

Le diagramme d'états-transition est comme son nom l'indique décrit les transitions entre les états et les actions que le système ou ses parties réalisent en réponse à un événement. Les diagrammes d'états-transitions permettent justement de spécifier les réactions d'une partie du système à des événements discrets. Un événement se produit à un instant précis et est dépourvu de durée.

Le diagramme d'états-transition décrit l'état ou le comportement d'un objet à un événement précis du système. A cet effet il est composé des éléments suivants :

Tableau 21 formalisme d'un diagramme de transition

Elément	Description	Représentation
Etat initial	Il s'agit de l'état dans lequel il se trouve lors de sa création. Est représenté par un point noir.	
Etat final	Correspond à une étape où l'objet n'est plus nécessaire dans le système et où il est détruit. Il se représente par un point entouré d'un cercle.	
Etat intermédiaire	Est une situation stable qui possède une certaine durée pendant laquelle un objet exécute une activité ou attend un Événement. Il est représenté par un rectangle arrondi contenant son nom.	
Transition	Une transition définit la réponse d'un objet à l'arrivée d'un événement. Elle est représentée par une flèche.	

Exemple de diagramme d'état transition :

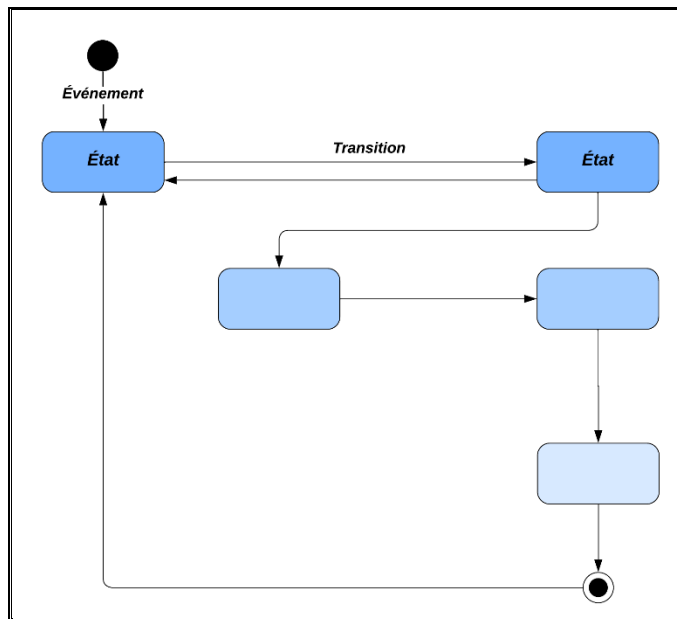


Figure 27 exemple de diagramme d'état transition

Quelques diagrammes d'état-transition de notre projet ci-dessous :

- Cas de la connexion :

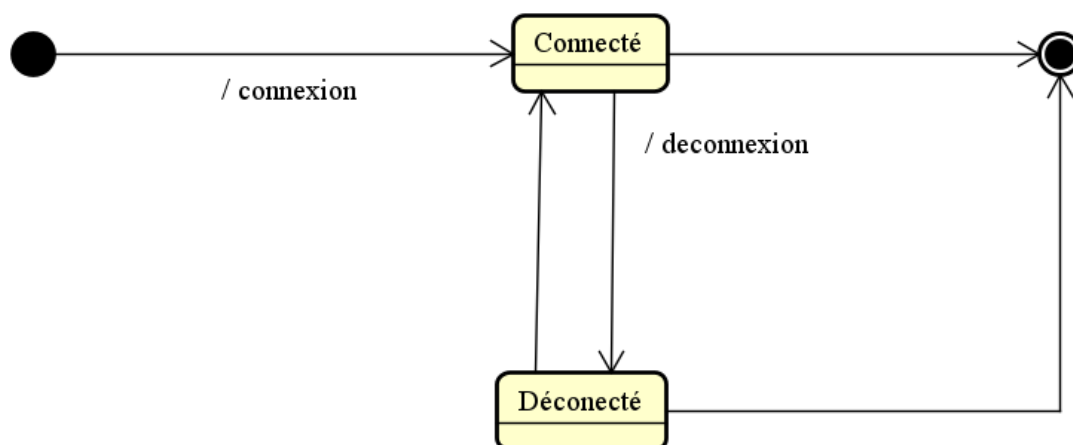


Figure 28 diagramme d'état transition du cas de la connexion

- Cas d'un utilisateur :

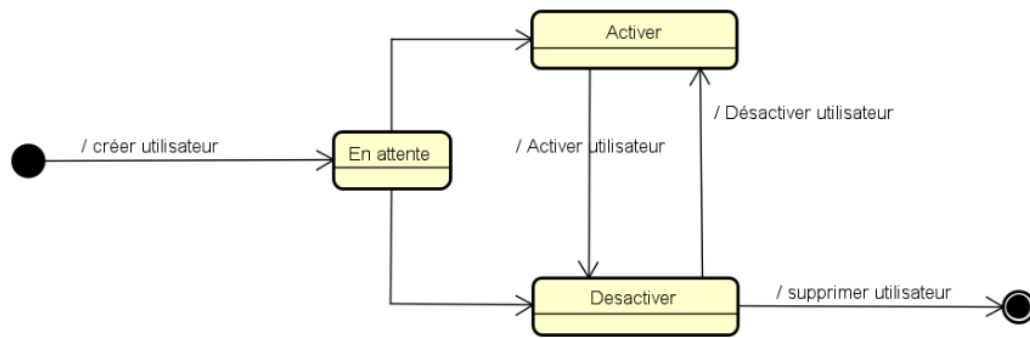


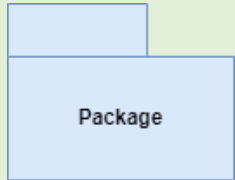
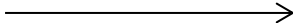
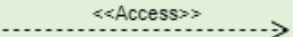
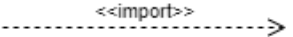
Figure 29 diagramme d'état transition du cas d'un utilisateur

III. DIAGRAMME DE PAQUETAGE

Les diagrammes de package (ou diagramme de paquetages) permettent de décomposer le système en catégories ou parties plus facilement observable s'appelés « packages ». L'utilisation la plus courante pour les diagrammes de paquetages est d'organiser des diagrammes de Cas d'Utilisation et des diagrammes de Classes. Bien que l'utilisation des Diagrammes de Paquetages ne se limite pas à ces éléments UML.

Le formalisme est le suivant :

Tableau 22 formalisme du diagramme de paquetage

Composant	Description	Représentation
Package	Regroupement des cas d'utilisation en catégorie	
Merge	Représente la fusion de deux packages. Le contenu du package cible est intégré au package source.	
Access	Indique que les éléments d'un package ont accès aux éléments publics d'un autre package.	
Import	Permet à un package d'importer les éléments publics d'un autre package.	

Le diagramme paquetage de notre projet est le suivant :

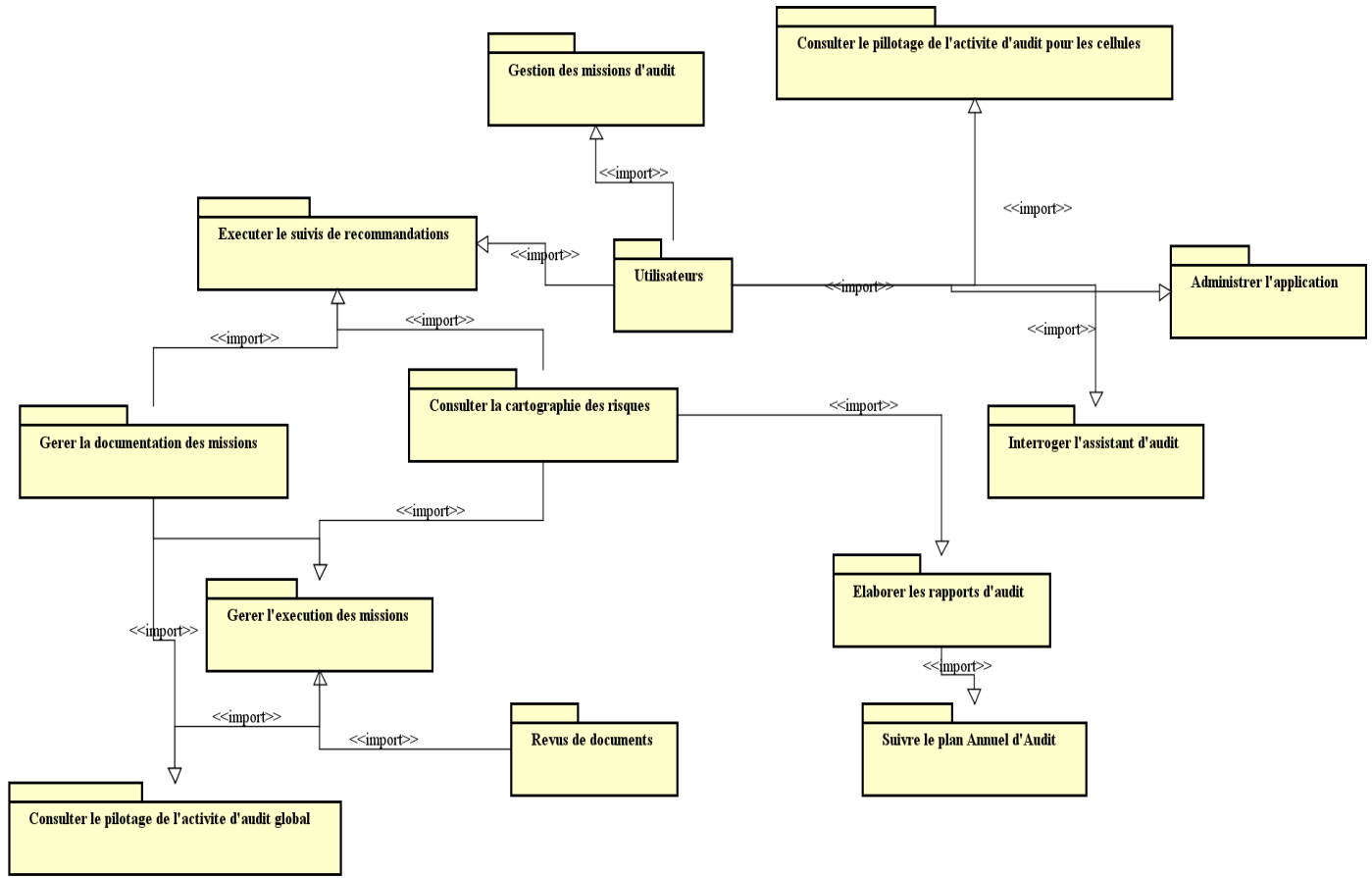


Figure 30 diagramme de paquetage.

CONCLUSION

En sommes, ce dossier analyse nous a permis de bien délimiter les besoins fonctionnels de notre application afin d'avoir une étude existentielle, sa critique et d'avoir un aperçu détaillé du nouveau système à mettre en place à travers la méthode d'analyse UML associé à son processus 2TUP.

DOSSIER 5 : DOSSIER DE RÉALISATION

Résumé :

La réalisation consiste à la mise en œuvre de projets dans un langage de programmation conformément aux spécifications définies dans le dossier précédent. Elle renferme en son sein les diagrammes de composant et de déploiement. A la sortie de cette partie, il sera produit une documentation de programmation expliquant l'architecture de la base de données et l'architecture de notre code.

Aperçu :

Introduction

1. Choix des technologies
2. Architecture physique et logique du système
3. Diagramme de déploiement
4. Diagramme de composant

Conclusion

INTRODUCTION

Fortement influencé par la phase de conception mais aussi d'analyse, la réalisation s'attaque au domaine physique et concret du projet. Tout au long de cette phase, la maîtrise d'œuvre met en place les différents codes et architecture utilisée dans la mise sur pied de la solution. Il sera donc question pour nous de présenter dans un premier temps le langage de programmation utilise, puis les outils de développement, enfin l'architecture de notre application.

I. CHOIX DES TECHNOLOGIES

1. Environnement logiciel

Différents logiciels utilisés lors de la réalisation de notre solution :

Tableau 23 environnement logiciel utilisée

NOM DU LOGICIEL	DESCRIPTION
	Git : Système de contrôle de version pour gérer les modifications du code et faciliter la collaboration entre développeurs.
	Gantt Project : Outil de gestion de projet pour créer des diagrammes de Gantt, planifier des tâches et suivre l'avancement.
	Astar : Outil de modélisation pour concevoir des bases de données et des systèmes d'information (modèles UML, MCD, MLD).
	IntelliJ IDEA est un environnement de développement intégré (IDE) puissant et populaire, conçu principalement pour programmer en Java et en Kotlin.
	Scene Builder est un outil visuel de conception d'interfaces graphiques (GUI) par glisser-déposer pour la technologie JavaFX

2. Technologies Desktop utilisée

Technologies utilisées pour le développement de l'application Desktop.

Tableau 24 technologies desktop utilisée

Technologie utilisée	
	JavaFX est un framework open-source permettant de créer des applications graphiques (interfaces utilisateur) modernes, performantes et riches en fonctionnalités pour les ordinateurs de bureau (Desktop), le web et les mobiles
	AtlantaFX est une collection de thèmes CSS modernes et open-source pour le framework de développement d'interfaces graphiques JavaFX.

3. Technologies Mobile utilisée

Technologies utilisées pour le développement de l'application de mobile.

Tableau 25 technologies mobile utilisée

Technologies mobile utilisée	
	Flutter est un framework de développement logiciel open-source créé par Google en 2017. Il permet aux développeurs de concevoir des applications performantes et esthétiques pour plusieurs plateformes (iOS, Android, Web, Windows, MacOS et Linux) à partir d'un seul et unique code de base .

4. Technologies backend utilisée

Technologies utilisées pour le développement du backend.

Tableau 26 technologies backend utilisée

Technologies backend utilisée	
	Python : Langage de programmation polyvalent utilisé pour le développement web, l'automatisation, la data science, et l'intelligence artificielle.
	FastAPI : est un framework web moderne, rapide (haute performance) et performant, utilisé pour bâtir des API avec le langage Python.
	Spring Boot : est un framework open-source basé sur Java qui permet de créer des applications autonomes et prêtes pour la production de manière simple et rapide.
	Java : est un langage de programmation de haut niveau, orienté objet et multiplateforme, créé par Sun Microsystems (aujourd'hui détenu par Oracle) et publié pour la première fois en 1995.
	PostgreSQL : est un système de gestion de base de données relationnelle et objet (SGBDRO) open-source, réputé pour sa robustesse, sa flexibilité et sa conformité aux normes SQL.

II. ARCHITECTURE PHYSIQUE ET LOGIQUE DU SYSTÈME

1. Architecture physique

Notre application utilise une architecture N-tiers. L'architecture N-tier (anglais tier : étage, niveau), ou encore appelée multi-tier, est une architecture client-serveur dans laquelle une application est exécutée par plusieurs composants logiciels distincts. Dans cette architecture, le lien entre les niveaux est défini et limité à des interfaces et interfaces assurent la modularité et l'indépendance technologique et topologique de chaque niveau.

Les différentes couches d'une architecture 4-tier sont :

- La couche de présentation contient les différents types de clients, léger (ASP, JSP) ou lourd (Applet) ;
- La couche applicative contient les traitements représentant les règles métier ;
- La couche d'objets métier est représentée par les objets du domaine, c'est à dire l'ensemble des entités persistantes de l'application ;
- La couche d'accès aux données contient les usines d'objets métier, c'est-à-dire les classes chargées de créer des objets métier de manière totalement transparente, indépendamment de leur mode de stockage (SGBDR, Objet, Fichiers, ...).

Cette séparation par couches de responsabilités sert à découpler au maximum une couche de l'autre afin d'éviter l'impact d'évolutions futures de l'application. Par exemple : si l'on est amené à devoir changer de base de données relationnelle, seule la couche d'accès aux données sera impactée, la couche de service et la couche de présentation ne seront pas concernées car elles auront été découplées des autres.

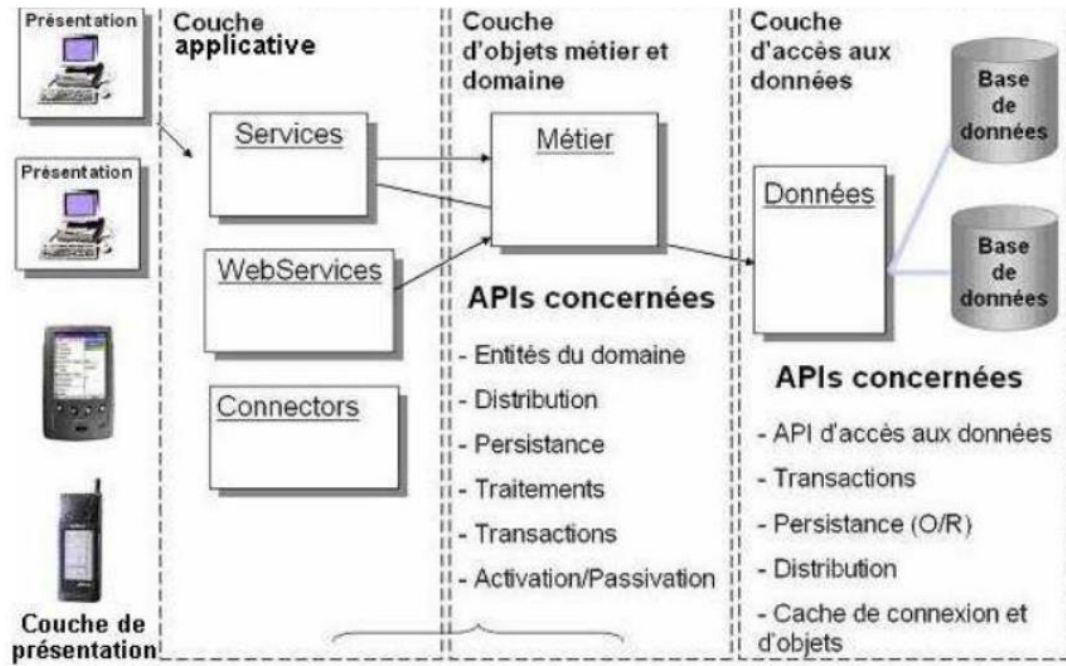


Figure 31 architecture physique n-tiers

2. Architecture logique

Nous basons la logique applicative sur le patron d'architecture Modèle-Vue-Contrôleur (MVC), largement utilisé aussi bien pour les interfaces desktop en JavaFX que pour les applications serveur développées avec Spring Boot. Le MVC est un patron d'architecture logicielle qui divise l'application en trois parties distinctes, favorisant ainsi la modularité, la réutilisation du code et la maintenabilité. Il se compose de trois types de modules :

- **Modèle (Model) :** Il représente les données et les règles métier de l'application : missions d'audit, risques significatifs, Plan Annuel d'Audit, constats, recommandations, etc. Ces données sont persistées dans la base PostgreSQL centralisée. Le modèle est indépendant de la vue et du contrôleur, ce qui permet de le modifier sans affecter les autres composants.
- **Vue (View) :** Elle est responsable de l'affichage des données et de l'interaction avec l'utilisateur. Dans notre application, elle est décrite en FXML (fenêtres, formulaires, tableaux de bord) et stylisée en CSS. La vue ne contient aucune logique métier : elle affiche uniquement les données qui lui sont fournies.
- **Contrôleur (Controller) :** Il fait le lien entre la vue et le modèle. Il reçoit les actions de l'utilisateur, déclenche les opérations métier correspondantes, puis met à jour la vue avec les résultats

Ce patron est appliqué sur les deux parties de l'application :

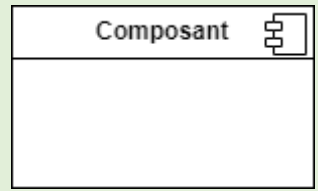
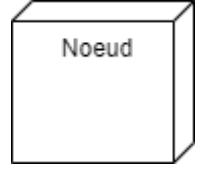
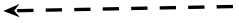
- **Côté client (JavaFX) :** les vues FXML sont associées à des contrôleurs JavaFX qui traitent les actions de l'utilisateur et communiquent avec le serveur.
- **Côté serveur (Spring Boot) :** l'application est organisée en couches : les contrôleurs REST exposent les services aux clients, la couche service porte les règles métier, et la couche d'accès aux données interagit avec PostgreSQL.

III. DIAGRAMME DE DÉPLOIEMENT

Le diagramme de déploiement est une vue statique qui sert à représenter l'utilisation de l'infrastructure physique par le système. Et la manière dont les composants du système sont répartis ainsi que les relations entre eux.

Le formalisme est le suivant :

Tableau 27 formalisme du diagramme de déploiement

Elément	Description	Représentation
Les composants	Il représente une entité logicielle du système (Fichier de code source, programmes, documents, fichiers de ressource, etc.). Un composant est représenté par une boîte rectangulaire avec deux rectangles dépassant du côté gauche portant le nom du composant.	
Les nœuds	Un nœud représente un ensemble d'éléments matériels du système. Cette entité est représentée par un cube tridimensionnel.	
Les Associations	Une association, représentée par une ligne pleine entre deux nœuds, indique une ligne de communication entre les éléments matériels.	
Les dépendances	Une dépendance est utilisée pour mobiliser la relation entre deux composants. La notation utilisée pour cette relation de dépendance est une flèche de pointillés.	

Exemple de diagramme de déploiement :

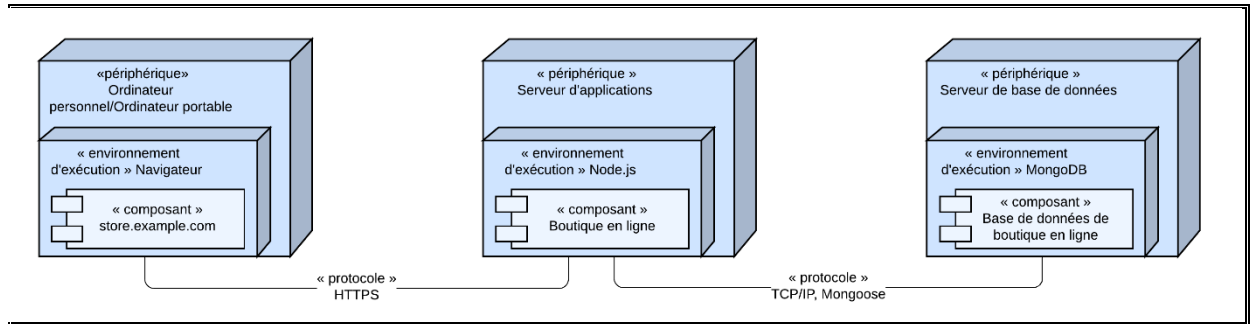


Figure 32 exemple de diagramme de déploiement

Le diagramme suivant est une architecture utilisée pour le déploiement de notre application :

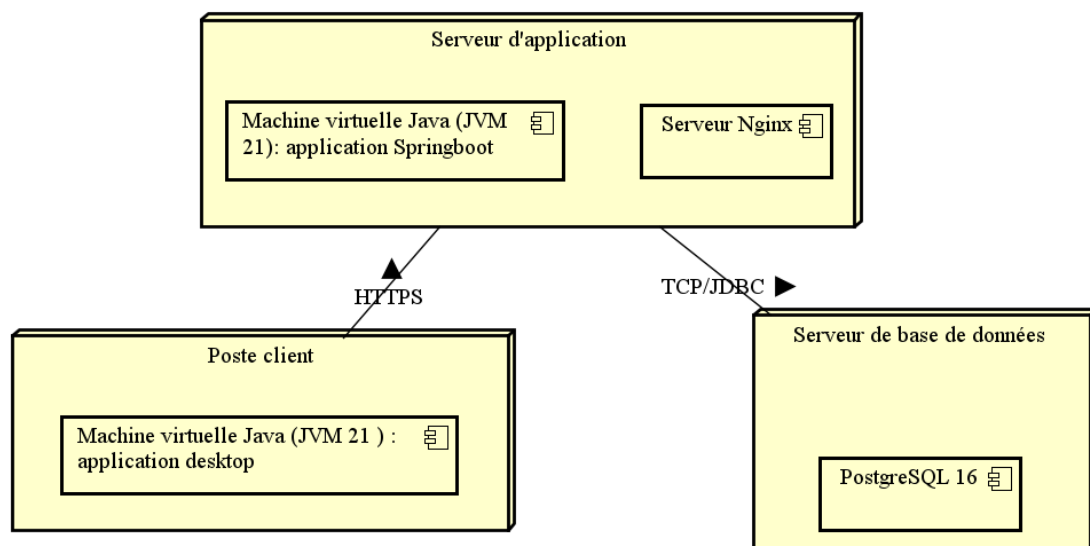


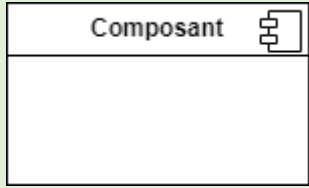
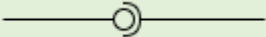
Figure 33 diagramme de déploiement

IV. DIAGRAMME DE COMPOSANT

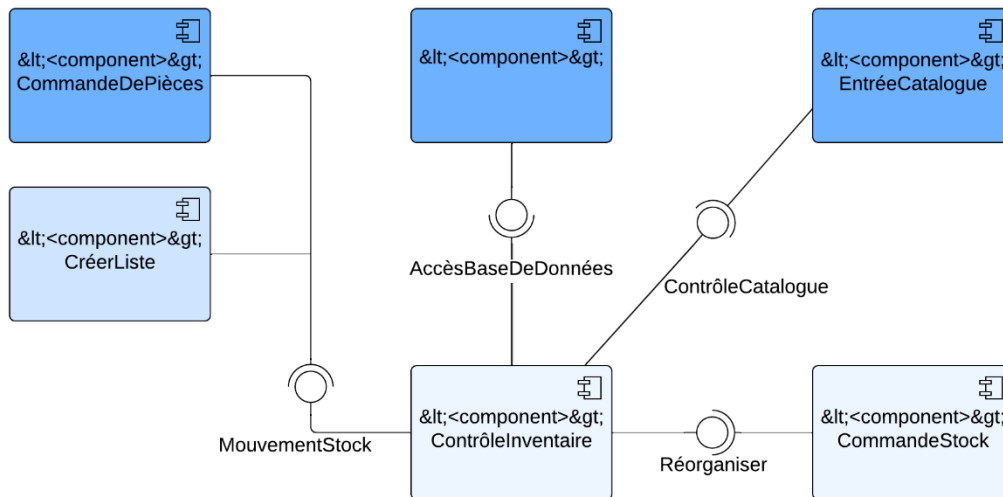
Le diagramme de composants est principalement employé pour décrire les dépendances entre divers composants (les dépendances entre fichiers exécutables et les fichiers sources). Le diagramme de composants modélise les composants logiciels utilisés pour implémenter un système et l'association entre ces composants

Le diagramme de composant décrit les composants et leurs dépendances dans leurs environnements de réalisation. A cet effet il est composé des éléments suivants :

Tableau 28 formalisme du diagramme de composant

Elément	Description	Représentation
Les Composants	Un composant représente une entité logicielle d'un système. Un composant est représenté par une boîte rectangulaire, avec deux rectangles dépassant du côté gauche.	
Dépendances	Elles sont utilisées pour modéliser la relation entre deux composants. La notation utilisée pour cette relation de dépendance est une flèche pointillée, se dirigeant d'un composant donné au composant dont il dépend.	
Interface (Nœud)	Lien entre deux composants logiciels. Il en existe 2 : Interface fournie Interface requise	

Exemple de diagramme de composants :



Le diagramme suivant montre la structure des composants logiciels du système :

Insérez votre diagramme ici...

CONCLUSION

Le dossier de réalisation nous a permis de présenter de façon générale les outils utilisés pour la réalisation de notre application. Cependant, connaître toutes les fonctionnalités du système est d'une importance capitale. La prochaine partie sera donc dédiée au test de fonctionnalités.

DOSSIER 6 : TEST DE FONCTIONNALITÉS

Résumé :

Un test désigne une procédure de vérification partielle d'un système. Son objectif principal est d'identifier un nombre maximal de comportements problématiques du logiciel. Il permet ainsi, dès lors que les problèmes identifiés seront corrigés, d'en augmenter la qualité.

Aperçu :

Introduction

1. Fonctionnalités
2. Tests

Conclusion

INTRODUCTION

Les tests de fonctionnalités sont une étape cruciale dans le cycle de vie du développement logiciel. Ils consistent à vérifier que les différentes fonctionnalités d'une application ou d'un système fonctionnent comme prévu. Ces tests se concentrent sur l'interaction entre l'utilisateur et le système et visent à valider que les fonctionnalités répondent aux exigences spécifiées. Contrairement aux tests unitaires, qui examinent chaque composant individuellement, les tests fonctionnels couvrent un ensemble de composants ou de modules intégrés pour s'assurer que l'application fonctionne correctement dans son ensemble. Ces tests aident à détecter les défauts qui peuvent avoir été introduits pendant le processus de développement, garantissent la conformité aux besoins des utilisateurs et identifient les éventuelles déviations par rapport aux spécifications. Dans un contexte où la qualité du logiciel est primordiale.

I. FONCTIONNALITÉS

Les fonctionnalités de notre application desktop sont définies en prenant en compte les besoins et les attentes des utilisateurs de la Division de l'Audit Interne. Elles désignent les différentes actions du système qui permettent à l'utilisateur d'effectuer des tâches spécifiques. Ainsi, les fonctionnalités de notre application sont les suivantes :

❖ **Cartographie des risques significatifs** : l'utilisateur identifie, évalue et classe les risques des différentes directions selon leur probabilité et leur impact, afin de disposer d'une base de décision pour la planification des missions.

❖ **Élaboration du Plan Annuel d'Audit** : l'utilisateur programme les missions à réaliser sur l'année en fonction des risques identifiés et des ressources disponibles, puis suit l'avancement du plan.

❖ **Gestion des missions d'audit** : l'utilisateur planifie, exécute et suit chaque mission, de la préparation à la restitution des constats, en centralisant les informations et les travaux réalisés pour garantir leur traçabilité.

❖ **Production de rapports** : l'utilisateur génère les rapports d'audit à partir des constats et des travaux enregistrés dans l'application.

❖ **Suivi des recommandations** : l'utilisateur suit l'état de mise en œuvre des recommandations formulées à l'issue des missions et identifie celles qui sont en retard ou non appliquées.

❖ **Pilotage de la performance par tableaux de bord** : l'utilisateur consulte des indicateurs synthétiques (avancement du plan, missions réalisées, recommandations mises en œuvre) pour piloter la performance de la fonction d'audit interne.

II. TESTS

1. Les types de tests fonctionnels

- ✚ Tests unitaires : Les tests unitaires vérifient le bon fonctionnement de composants individuels ou de petites unités de code, généralement des fonctions ou des méthodes. L'objectif est de s'assurer que chaque unité de code fonctionne comme prévu, indépendamment des autres parties du système. Ces tests sont généralement écrits et exécutés par les développeurs et permettent de détecter des erreurs dès le début du cycle de développement, ce qui facilite le débogage et améliore la qualité du code ;
- ✚ Tests d'acceptation : Ces tests vérifient si le système répond aux exigences et aux attentes des utilisateurs finaux. Ils sont souvent réalisés par les utilisateurs eux-mêmes pour s'assurer que le produit est conforme à leurs besoins ;
- ✚ Tests d'intégration : Ces tests se concentrent sur l'interaction entre différents modules ou composants d'une application. L'objectif est de s'assurer que les modules fonctionnent ensemble comme prévu et que les données circulent correctement entre eux ;
- ✚ Tests de système : Les tests de système évaluent l'application dans son ensemble, en vérifiant que toutes les fonctionnalités fonctionnent comme spécifier dans les exigences. Cela inclut des tests sur l'interface utilisateur, la performance et la sécurité ;
- ✚ Tests de régression : Ces tests sont effectués après des modifications du code (comme des corrections de bugs ou des mises à jour) pour s'assurer que les nouvelles modifications n'ont pas introduit de nouveaux défauts dans les fonctionnalités existantes ;
- ✚ Tests de performance : Ces tests mesurent la réactivité, la rapidité et la stabilité d'une application sous diverses charges. Ils visent à s'assurer que le système peut gérer le volume prévu d'utilisateurs et de transactions ;
- ✚ Tests de compatibilité : Ces tests vérifient que l'application fonctionne correctement sur différentes plateformes, navigateurs, systèmes d'exploitation et appareils. Cela garantit une expérience utilisateur cohérente quel que soit le contexte d'utilisation ;

- ✚ Tests de sécurité : Ces tests évaluent la sécurité de l'application, en vérifiant les vulnérabilités et les failles potentielles qui pourraient être exploitées par des attaquants. Ils incluent des tests d'authentification, d'autorisation et de protection des données ;
- ✚ Tests d'usabilité : Ces tests examinent l'expérience utilisateur et la facilité d'utilisation de l'application. Ils sont souvent réalisés avec de vrais utilisateurs qui interagissent avec le système, permettant d'identifier des problèmes d'ergonomie ou de navigation ;
- ✚ Tests de localisation : Ces tests s'assurent que l'application est adaptée à différents marchés et cultures, en vérifiant les traductions, les formats de date, les symboles monétaires, et d'autres éléments spécifiques à chaque région ;
- ✚ Tests de montée en charge : Ces tests vérifient comment l'application se comporte sous une charge croissante. Ils aident à identifier les points de rupture et à s'assurer que le système peut évoluer avec la demande des utilisateurs.

2. Procédure pour effectuer des tests en Springboot

Arrivés à l'étape cruciale des tests dans le développement de notre application de gestion des missions d'audit interne, nous avons mis en place une stratégie de test s'appuyant sur l'écosystème de test de Spring Boot, combiné à notre client JavaFX. Cette procédure s'est articulée autour de plusieurs principes fondamentaux :

- **Création de classes de test avec JUnit 5 et Spring Boot Test** : Nous avons développé des classes de test annotées avec `SpringBootTest`, `DataJpaTest` ou `WebMvcTest` selon la couche testée. Elles permettent de bénéficier du contexte Spring, de l'injection de dépendances et de la configuration automatique d'une base de données dédiée aux tests, distincte de la base PostgreSQL de production.
- **Utilisation de BeforeEach pour initialiser l'environnement de test** : Cette annotation a été employée pour préparer l'environnement avant l'exécution de chaque méthode de test. Elle nous a permis de créer des objets fictifs avec **Mockito**, de peupler la base avec des données de test (missions, risques, recommandations) et de simuler des utilisateurs authentifiés. Chaque test démarre ainsi avec un contexte propre et reproductible.
- **Implémentation de méthodes de test spécifiques** : Chaque méthode de test est annotée `Test` et conçue pour vérifier une fonctionnalité précise : les entités et les repositories pour la persistance des données, les services pour les règles métier, les contrôleurs REST pour l'exposition des services, et les contrôleurs JavaFX pour le traitement des actions de l'utilisateur. Ces méthodes utilisent des assertions spécialisées comme `assertEquals()` et `assertThat()` (JUnit, AssertJ), ainsi que `status().isOk()` et `jsonPath()` avec **MockMvc** pour valider le comportement attendu des API.
- **Exécution des tests via Maven** : La commande `mvn test` a été utilisée pour lancer la suite de tests complète. Maven exécute alors tous les tests, gère automatiquement le cycle de vie du build et génère un rapport indiquant les tests réussis et ceux ayant échoué, avec des messages d'erreur explicites pour faciliter le débogage.

Pour les composants graphiques JavaFX, nous avons complété cette approche avec **TestFX**, qui permet de simuler les interactions réelles de l'utilisateur (clics, saisies, navigation entre les vues) et de valider le rendu et le comportement des interfaces.

3. Différents tests effectués :

1. Tests unitaires (tests de vue) :

Ces tests vérifient le bon fonctionnement des vues JavaFX.

```
@Test @miguelnkono
@DisplayName("L'Administrateur n'a aucun accès métier, pas une seule entrée")
void lAdministrateurNaAucunAccesMetier() {
    List<String> vues = vuesDe(Profil.ADMINISTRATEUR);

    for (String metier : VUES_METIER) {
        assertFalse(vues.contains(metier),
            message: "L'Administrateur ne doit jamais pouvoir influencer le contenu d'un audit, "
                + "or son menu porte " + metier);
    }
}

@Test @miguelnkono
@DisplayName("Mon bureau s'ouvre à qui conduit des travaux, et à personne d'autre")
void monBureauSouvreAQuiConduitDesTravaux() {
    // L'écran ne montre que ce qui porte le nom de l'appelant. Il n'a donc de sens que
    // pour les profils qui reçoivent des tests, préparent des pièces ou posent des visas.
    for (Profil profil : List.of(Profil.CHEF_DE_DIVISION, Profil.CHEF_DE_CELLULE,
        Profil.AUDITEUR_SENIOR, Profil.AUDITEUR)) {
        assertTrue(vuesDe(profil).contains("bureau-view.fxml"),
            message: profil + " conduit des travaux : sans cet écran, il doit ouvrir ses "
                + "missions une par une pour savoir ce qui l'attend");
    }

    // La Direction Générale n'est jamais affectée à une mission : les six listes lui
    // reviendraient vides, et une entrée de menu qui ne montre jamais rien fait douter de
    // l'application plutôt que d'un droit qu'on n'a pas.
    assertFalse(vuesDe(Profil.DIRECTION_GENERALE).contains("bureau-view.fxml"));
    assertFalse(vuesDe(Profil.ADMINISTRATEUR).contains("bureau-view.fxml"));
}
```

Figure 34 premier test sur les rôles

```
@Test @miguelnkono
@DisplayName("L'adresse est lue dans le fichier pose à côté de l'application")
void lAdresseEstLueDansLeFichier(@TempDir Path repertoire) throws IOException {
    Path fichier = repertoire.resolve( other: "sigma-adc.properties");
    Files.writeString(fichier, csq: """
        # Adresse du serveur pour ce poste
        sigma.api.url=http://192.168.1.20:8080
        """);

    assertEquals( expected: "http://192.168.1.20:8080", ApiConfig.lireDans(fichier));
}

@Test @miguelnkono
@DisplayName("Un fichier absent ne fait rien échouer")
void unFichierAbsentNeFaitRienEchouer(@TempDir Path repertoire) {
    // L'application doit démarrer sur un poste ou l'administrateur n'a pas encore pose le
    // fichier : elle se rabat alors sur la valeur embarquée.
    assertNull(ApiConfig.lireDans(repertoire.resolve( other: "absent.properties")));
    assertNull(ApiConfig.lireDans( fichier: null));
}
```

Figure 35 test d'appel d'api

```
@Test @miguelnkono
@DisplayName("Seul l'Administrateur administre les comptes, les postes et les cellules")
void ladministrationNestOuverteQuaLAdministrateur() {
    for (Profil profil : Profil.values()) {
        if (profil == Profil.ADMINISTRATEUR) {
            continue;
        }
        List<String> vues = vuesDe(profil);
        for (String administration : VUES_ADMINISTRATION) {
            assertFalse(vues.contains(administration),
                message: "le profil " + profil + " ne doit pas administrer : " + administration);
        }
    }
}

@Test @miguelnkono
@DisplayName("La Direction Générale lit les dossiers, et n'annote pas")
void laDirectionGeneraleEstStrictementEnLecture() {
    List<String> vues = vuesDe(Profil.DIRECTION_GENERALE);

    // Pas de « Mes remarques » : la Direction Générale n'annote pas, ne répond pas, et n'en
    // recevrait donc jamais. L'entrée lui aurait promis une liste toujours vide.
    assertFalse(vues.contains("notifications-view.fxml"),
        message: "la Direction Générale n'annote pas et n'en recevrait jamais");
    assertFalse(vues.contains("journal-view.fxml"),
        message: "le journal appartient à ses deux lecteurs, pas à la gouvernance");
    for (String administration : VUES_ADMINISTRATION) {
        assertFalse(vues.contains(administration),
            message: "la Direction Générale n'administre pas : " + administration);
    }
}
```

Figure 36 deuxième test sur le rôle s'identifier

```
@Test // miguelnkono
@DisplayName("Chaque type d'échange porte les mêmes composants des deux cotes")
void lesTypesDEchangeNontPasDerive() throws IOException {
    Assumptions.assumeTrue(Files.isDirectory(DTO_SERVEUR),
        message: "dépôt du serveur absent à côté du client : vérification sautee");

    Map<String, String> ecarts = new LinkedHashMap<>();
    int compares = 0;

    try (Stream<Path> fichiers = Files.list(MODELES_CLIENT)) {
        for (Path cote : fichiers.filter( Path f -> f.toString().endsWith(".java")).sorted().toList()) {
            String nom = cote.getFileName().toString().replace( target: ".java", replacement: "");
            Path serveur = DTO_SERVEUR.resolve( other: nom + ".java");

            List<String> ici = composantsDuClient(nom);
            if (ici.isEmpty() || !Files.exists(serveur)) {
                continue; // pas un record, ou type propre au client : rien à confronter
            }

            List<String> laBas = composantsDeLaSource(Files.readString(serveur));
            if (laBas.isEmpty()) {
                continue;
            }

            compares++;
            if (!ici.equals(laBas)) {
                ecarts.put(nom, "client " + ici + " / serveur " + laBas);
            }
        }
    }

    // Sans ce garde-fou, une extraction cassée ferait passer le test sur zéro comparaison :
    // il resterait vert en ne verifiant plus rien, ce qui est pire que rouge.
}
```

Figure 37 tests sur les modèles

CONCLUSION

Les tests de fonctionnalités sont essentiels pour garantir la qualité globale du logiciel. Ils valident non seulement que les fonctionnalités clés fonctionnent comme prévu, mais aussi que l'expérience utilisateur répond aux attentes. En identifiant les défauts et les lacunes, ils contribuent à améliorer la robustesse, la fiabilité et les performances du logiciel avant sa publication. Les tests fonctionnels contribuent ainsi à la satisfaction des utilisateurs et sont un élément central du processus de développement logiciel.

DOSSIER 7 : GUIDE D'INSTALLATION ET GUIDE D'UTILISATEUR

Résumé :

Le manuel d'installation et d'utilisation est le document établi après réalisation d'une application, d'un logiciel ou d'une plateforme. Il renseigne sur la question « comment installer et utiliser le logiciel, l'application ou la plateforme que l'on a devant soi ? »

Aperçu :

Introduction

1. Guide d'installation
2. Guide utilisateur

Conclusion

INTRODUCTION

Ce manuel propose des directives exhaustives et opérationnelles pour vous accompagner à travers les étapes d'installation, de configuration et de lancement de l'application. Son but est d'assurer une expérience sans accroc pour les utilisateurs à venir et de faciliter la prise en main de l'application, en expliquant de manière concise et accessible ses fonctionnalités, son utilisation, et les étapes à suivre pour tirer le meilleur parti de ses capacités. Dans cette section, nous vous offrirons une vue d'ensemble des principales phases requises pour installer l'application, en mettant en évidence les éléments essentiels qui simplifieront la marche à suivre du système ensuite nous présenterons un aperçu des principales caractéristiques de l'application et des actions que les utilisateurs peuvent entreprendre, leur permettant ainsi de profiter pleinement de cet outil de recherche de logement convivial et efficace.

I. GUIDE D'INSTALLATION

Afin de réaliser une installation adéquate de l'application en local, nous nécessitons des outils suivants pour être installé en dure dans notre machine :

- ✓ Installation de Java ;
 - Rendez sur le site d'Oracle (<https://www.oracle.com/java/technologies/downloads/>) pour télécharger Java ;

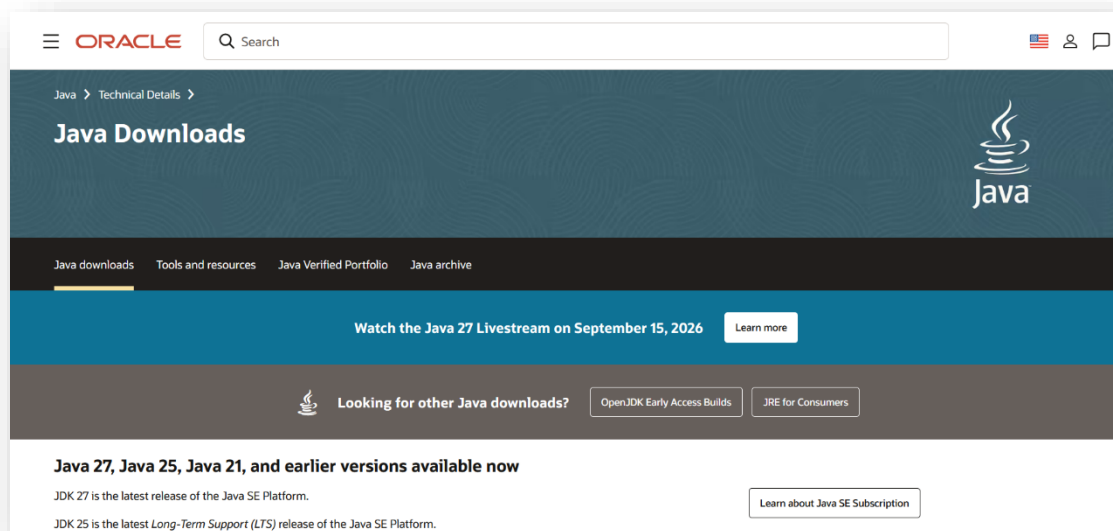


Figure 38 page d'accueil du site d'Oracle pour télécharger Java.

- Lancez l'installateur, puis suivre les tapes ;
- ✓ Pour vérifier que tout fonctionne bien, lancez ces commandes :
 - **Java -version**
 - **Javac -version**
- ✓ Si tout c'est bien passe, votre machine devrais déjà être bien configure pour lance l'application.

II. GUIDE UTILISATEUR

Pour accéder à l'application et commencé à l'utiliser suivez les différentes étapes ci-dessous :

- Ouvrez un terminal Powershell et rendez vous dans le dossier qui contient le code du projet que vous vous seriez appropriée a préalable ;
- Puis rendez vous dans le dossier **sigma-adc-backend** ;
- Dans ce dossier lancez la commande : **./gradlew clean bootRun**, qui est la commande qui vas lancer le serveur d'application ;
- Sortez de ce dossier et rendez vous dans le dossier qui contient la vue et qui s'appel **sigma_adc_client** ;
- Dans ce dossier, lancez la commande : **./mvnw clean javafx:run**, qui est la commande qui vas lance l'application desktop ;
- Après avoir lancée l'application desktop, vous verrez cette page si :



The screenshot shows a web browser window titled "SIGMA-ADC — Connexion". The page features the ADC logo (Aéroports Du Cameroun S.A.) and the title "SIGMA-ADC". Below the title, it reads "Système Intégré de Gestion des Missions d'Audit" and "Division de l'Audit Interne". There are two input fields: "Matricule" and "Mot de passe". A blue button labeled "Se connecter" is positioned below the password field. At the bottom, a note states: "Ce poste doit avoir été approuvé par un administrateur."

Figure 39 page de connexion

- Après s'être connecté, on arrive sur la page d'accueil qui dépend du rôle de l'utilisateur :
 - Pour l'admin de l'application :

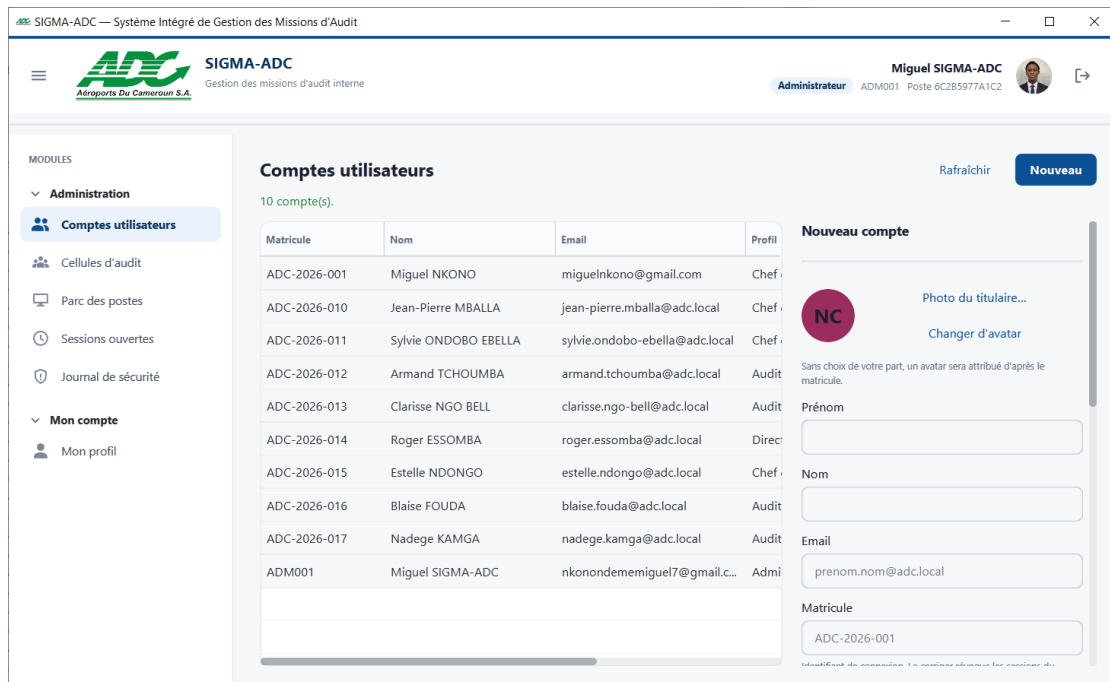


Figure 40 page d'accueil de l'administrateur

- Pour le chef de division :

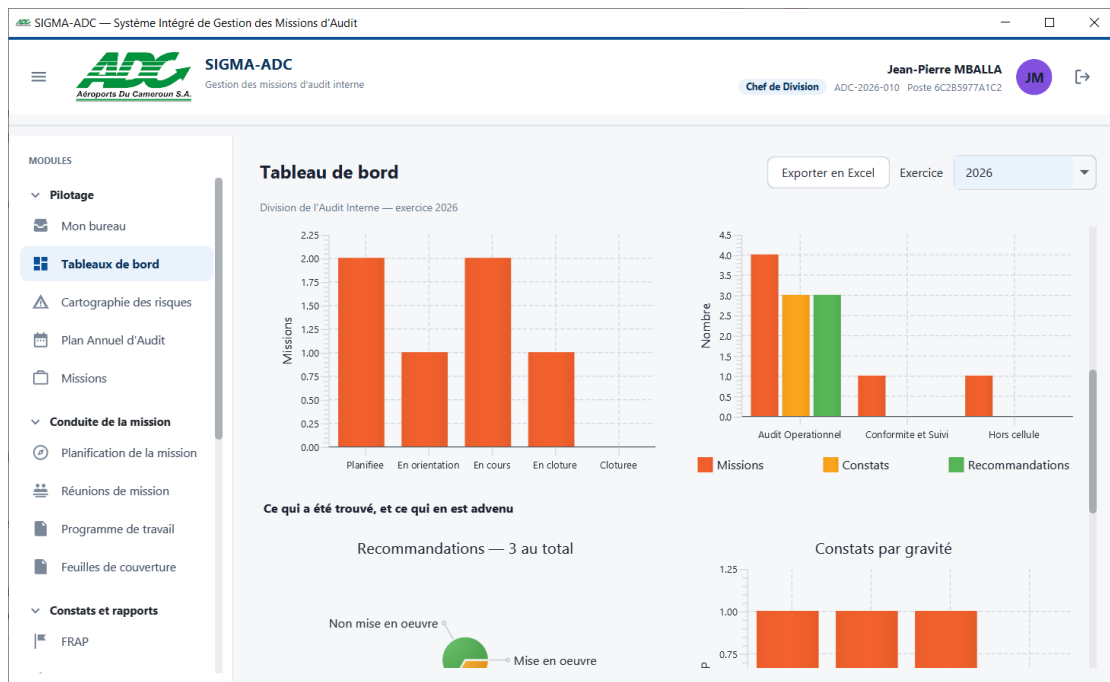


Figure 41 page d'accueil du chef de division

- Après être arrivé à la page de connexion en fonction de votre rôle, vous pourriez utiliser l'application à votre guise.

CONCLUSION

Malgré la simplicité de notre application nous avons tenu à établir ce manuel utilisateur afin de présenter de façon claire et précise le processus d'installation puis d'utilisateur. Ce guide contient toutes les informations permettant aux futurs utilisateurs de connaître l'environnement et les fichiers nécessaires au bon fonctionnement du logiciel.

CONCLUSION GÉNÉRALE

Arrivés au terme de ce projet de fin de stage, nous avons eu l'opportunité de concevoir et de développer une solution répondant à un besoin concret de la Division de l'Audit Interne d'Aéroports Du Cameroun S.A. : disposer d'un outil dédié à la gestion des missions d'audit interne, en remplacement d'outils bureautiques dispersés qui limitaient la centralisation de l'information et la traçabilité des travaux. Notre démarche s'est articulée autour de plusieurs phases essentielles : l'insertion en entreprise, qui nous a permis de nous familiariser avec la structure d'accueil et le fonctionnement de la fonction d'audit ; l'analyse des besoins, qui a défini précisément les objectifs et les fonctionnalités attendues ; la conception de la solution, qui a établi les fondations d'une architecture client-serveur fiable et modulaire ; et enfin la réalisation de l'application, à l'aide de technologies telles que Java, Spring Boot pour le serveur, JavaFX pour le client desktop et PostgreSQL pour la base de données centralisée.

Cette expérience nous a permis de mettre en pratique nos connaissances théoriques et de développer nos compétences techniques dans un contexte professionnel, tout en nous confrontant aux réalités de la gestion de projet informatique et à l'écoute des utilisateurs. Nous avons ainsi conçu une application qui couvre l'ensemble du cycle de vie d'une mission d'audit : la cartographie des risques significatifs, l'élaboration du Plan Annuel d'Audit, la gestion des missions, la production des rapports et le suivi de la mise en œuvre des recommandations, le tout complété par des tableaux de bord destinés au pilotage de la performance de la fonction d'audit interne.

De ce fait, comment garantir que l'application s'adapte durablement à l'évolution des besoins et des méthodes de travail de la Division de l'Audit Interne, tout en assurant la sécurité et la confidentialité des informations sensibles qu'elle centralise ? Et comment faire évoluer cette solution, par exemple vers une version web ou mobile, l'interconnexion avec les autres systèmes d'information de l'entreprise, ou l'exploitation de l'analyse de données pour renforcer l'aide à la décision, afin qu'elle accompagne la fonction d'audit dans le temps ?

BIBLIOGRAPHIE

Audit interne et gestion des risques

- The Institute of Internal Auditors (2024). *Global Internal Audit Standards*. IIA. → Référentiel international qui définit les exigences de la pratique de l'audit interne, sur lequel s'appuie le cycle de vie des missions géré par l'application.
- Renard, J. (2016). *Théorie et pratique de l'audit interne* (9e éd.). Eyrolles. → Ouvrage de référence en français sur la planification, la conduite des missions, les constats et le suivi des recommandations.
- COSO (2013). *Internal Control – Integrated Framework*. Committee of Sponsoring Organizations of the Treadway Commission. → Cadre de référence du contrôle interne, sur lequel repose l'évaluation des dispositifs de contrôle des directions.
- COSO (2017). *Enterprise Risk Management – Integrating with Strategy and Performance*. → Fondement méthodologique de l'identification et de l'évaluation des risques utilisées dans la cartographie.
- ISO (2018). *ISO 31000:2018 – Management du risque : lignes directrices*. → Principes de gestion des risques (probabilité, impact, hiérarchisation) appliqués dans le module de cartographie.

Architecture et conception logicielle

- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. → Référence pour l'architecture en couches (présentation, service, accès aux données) des applications d'entreprise, utilisée pour notre serveur.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. → Fondamental pour la conception orientée objet et l'application de patrons tels que le MVC.
- Tardieu, H., Rochfeld, A., & Coletti, R. (2000). *La méthode MERISE, Tome 1 : Principes et outils*. Éditions d'Organisation. → Base méthodologique pour la modélisation des données et la conception de la base de données.

Technologies de développement

- Walls, C. (2022). *Spring in Action* (6e éd.). Manning Publications. → Guide de référence pour le développement du serveur avec Spring Boot (API REST, injection de dépendances, accès aux données).
- Bloch, J. (2018). *Effective Java* (3e éd.). Addison-Wesley. → Bonnes pratiques de programmation Java appliquées au client et au serveur.
- Spring Boot Reference Documentation (2025). [En ligne]. Disponible à : <https://docs.spring.io/spring-boot/> → Documentation officielle du framework serveur.

- OpenJFX Project (2025). *JavaFX Documentation* [En ligne]. Disponible à : <https://openjfx.io> → Documentation officielle pour la construction du client desktop (FXML, contrôleurs, CSS).
- PostgreSQL Global Development Group (2025). *PostgreSQL Documentation* [En ligne]. Disponible à : <https://www.postgresql.org/docs/> → Documentation officielle du SGBD centralisant les données de l'application.

Tests

- JUnit Team (2025). *JUnit 5 User Guide* [En ligne]. Disponible à : <https://junit.org/junit5/docs/current/user-guide/> → Référence pour l'écriture des tests unitaires et d'intégration.
- TestFX (2025). *TestFX Documentation* [En ligne]. Disponible à : <https://github.com/TestFX/TestFX> → Bibliothèque de tests automatisés des interfaces JavaFX

WEBOGRAPHIE

- <https://www.theiia.org/en/standards/> : Site de l'Institute of Internal Auditors présentant les *Global Internal Audit Standards*, référentiel international de la pratique de l'audit interne. Il sert de base au cycle de vie des missions géré par l'application (planification, réalisation, communication des résultats, suivi) (visité le .../.../2025 à ... : ...).
- <https://www.ifaci.com> : Site de l'Institut Français de l'Audit et du Contrôle Interne, qui propose des ressources en français sur les normes, la méthodologie et les bonnes pratiques de l'audit interne, utiles pour la cartographie des risques et le suivi des recommandations (visité le .../.../2025 à ... : ...).
- <https://www.coso.org> : Site du Committee of Sponsoring Organizations, éditeur des référentiels de contrôle interne et de gestion des risques d'entreprise. Il éclaire l'évaluation des dispositifs de contrôle des directions (visité le .../.../2025 à ... : ...).
- <https://www.iso.org/iso-31000-risk-management.html> : Page officielle de la norme ISO 31000 sur le management du risque, dont les principes (identification, évaluation, hiérarchisation) sont appliqués dans le module de cartographie des risques (visité le .../.../2025 à ... : ...).
- <https://docs.spring.io/spring-boot/> : Documentation officielle de Spring Boot couvrant les API REST, l'injection de dépendances, l'accès aux données et le système de tests, essentiels pour le développement du serveur de l'application (visité le .../.../2025 à ... : ...).
- <https://openjfx.io> : Site officiel d'OpenJFX présentant la documentation de JavaFX (FXML, contrôleurs, CSS), fondamentale pour la construction de l'interface du client desktop (visité le .../.../2025 à ... : ...).
- <https://www.postgresql.org/docs/> : Documentation officielle de PostgreSQL, système de gestion de bases de données relationnelles qui centralise les données de l'application (missions, risques, recommandations) (visité le .../.../2025 à ... : ...).
- <https://junit.org/junit5/docs/current/user-guide/> : Guide officiel de JUnit 5, utilisé pour l'écriture des tests unitaires et d'intégration du serveur et du client (visité le .../.../2025 à ... : ...).
- <https://github.com/TestFX/TestFX> : Dépôt de TestFX, bibliothèque de tests automatisés des interfaces JavaFX, utilisée pour simuler les interactions de l'utilisateur (visité le .../.../2025 à ... : ...).



ANNEXES

TABLE DES MATIÈRES

DÉDICACE	II
REMERCIEMENTS	III
SOMMAIRE	IV
LISTE DES TABLEAUX	V
LISTE DES FIGURES	VI
SIGLES ET ABBREVIATIONS	VIII
GLOSSAIRE	IX
RESUME	XI
ABSTRACT	XII
INTRODUCTION GÉNÉRALE	1
1ère PARTIE : PHASE D'INSERTION	2
I. ACCUEIL ET INTÉGRATION EN ENTREPRISE	4
1. Accueil en entreprise	4
2. Intégration en entreprise	5
II. PRÉSENTATION DE L'ENTREPRISE	6
1. Historique	6
2. Situation géographique	7
3. Organigramme	8
4. Mission	10
5. Activités	10
III. RESSOURCES MATÉRIELLES ET LOGICIELLES	11
1. Ressources matérielles	11
2. Ressources logicielles	12
2ème PARTIE : PHASE TECHNIQUE	14
DOSSIER 1 : L'EXISTANT	16
I. PRÉSENTATION DU THÈME/PROJET	18
II. ETUDE DE L'EXISTANT	19
1. Situation actuelle au sein d'Aéroports Du Cameroun S.A.	19
2. Recherche sur les solutions existantes sur le marché	19
III. CRITIQUE DE L'EXISTANT	21

IV. PROBLÉMATIQUE	22
V. PROPOSITION DE SOLUTION	22
DOSSIER 2 : CAHIER DES CHARGES.....	24
I. CONTEXTE ET JUSTIFICATION DE L'ÉTUDE DU PROJET	26
1. Contexte	26
2. Justification	27
II. BÉNÉFICIAIRE ET CIBLE	29
1. Bénéficiaire	29
2. Cible.....	29
III. LES OBJECTIFS DE L'ÉTUDE PROJET	30
1. OBJECTIF GÉNÉRAL.....	30
2. OBJECTIFS SPÉCIFIQUES	30
IV. EXPRESSIONS DES BESOINS DE L'UTILISATEUR.....	32
1. BESOINS FONCTIONNELS.....	32
2. Besoins non fonctionnels	34
V. PLANIFICATION DU PROJET / GANTT PROJECT	36
1. Intervenants.....	36
2. Diagramme de Gantt	37
VI. ESTIMATION DU COÛT DU PROJET	38
1. Ressources Matérielles.....	38
2. Ressources logicielles	39
3. Ressources humaines	41
4. Cout total.....	41
VII. LES CONTRAINTES DU PROJET.....	42
VIII. LES LIVRABLES	44
DOSSIER 3 : DOSSIER D'ANALYSE.....	46
I. Méthode et langage d'analyse.....	48
1. Etude comparative UML et MERISE	48
2. Choix et présentation de la démarche d'analyse.....	50
3. Etude comparative des processus unifiés.....	51
4. Choix et présentation du processus unifié	52
II. Modélisation	54
1. Diagramme des cas d'utilisation.....	54

2. Diagramme de communication	71
3. Diagramme de séquence	76
4. Diagramme d'activité.....	83
DOSSIER 4 : DOSSIER DE CONCEPTION	91
I. DIAGRAMME DE CLASSE	93
1. Règles de métiers de notre diagramme de classe :.....	97
II. DIAGRAMME D'ÉTAT TRANSITION	102
III. DIAGRAMME DE PAQUETAGE.....	105
DOSSIER 5 : DOSSIER DE RÉALISATION.....	108
I. Choix des technologies	110
1. Environnement logiciel.....	110
2. Technologies Desktop utilisée	111
3. Technologies Mobile utilisée	111
4. Technologies backend utilisée	112
II. Architecture physique et logique du système	113
1. Architecture physique	113
2. Architecture logique.....	115
III. Diagramme de déploiement	116
IV. Diagramme de composant.....	118
DOSSIER 6 : TEST DE FONCTIONNALITÉS	121
I. Fonctionnalités.....	123
II. Tests	124
1. Les types de tests fonctionnels.....	124
2. Procédure pour effectuer des tests en Springboot.....	126
3. Différents tests effectués :.....	127
DOSSIER 7 : GUIDE D'INSTALLATION ET GUIDE D'UTILISATEUR.....	131
I. Guide d'installation.....	133
II. Guide utilisateur.....	134
CONCLUSION GÉNÉRALE.....	138
BIBLIOGRAPHIE	2
WEBOGRAPHIE	4
ANNEXES.....	5
TABLE DES MATIÈRES	6



Conception et développement d'une application de gestion des missions d'audit interne : SIGMA-ADC

